Especificación formal y verificación de modelos con TLA+/PlusCal

v1.1.3

Pedro de las Heras Quirós, Eva Castro Barbero
Escuela de Ingeniería de Fuenlabrada, EIF/URJC

Octubre de 2024

Este documento enseña a escribir especificaciones de sistemas digitales software usando el lenguaje de especificación formal TLA+. Posteriormente enseña a utilizar PlusCal, un lenguaje algorítmico que se parece a los lenguajes de programación imperativos y que sirve principalmente para describir algoritmos concurrentes y distribuidos.

Índice

Introducción 3
Especificaciones de sistemas digitales 6
TLA+: Definiciones 7
Instalación de las herramientas de TLA+/PlusCal 8
Primera especificación con TLA+: un reloj que sólo marca las horas 9
Verificación de modelos con TLC 12
Propiedades de las especificaciones 13
TLC: verificación de propiedades de tipo Safety del reloj que sólo marca las horas 15
TLC: verificación de propiedades de tipo Liveness del reloj que sólo marca las horas 19
TLC: verificación de propiedades de acciones del reloj que sólo marca las horas 23
Refactorizaciones de la especificación del reloj que sólo marca las horas 25
Cambio en los requisitos del reloj que sólo marca las horas: nuevo estado inicial 29
Especificación de un termómetro en TLA+ 30
Ejercicio 1: Verificación de propiedades del termómetro 36
Ejercicio 2: Especificación de reloj con minutos y AM/PM 39
Ejercicio 3: Especificación de reloj con minutos y AM/PM más termómetro 39
Especificación de un reloj que sólo marca las horas en PlusCal 40
Especificación de un termómetro en PlusCal 47
Refactorización de la especificación del termómetro en PlusCal 53
Ejercicio 4: Termómetro y Reloj que marca las horas en PlusCal 56
Ejercicio 5: Termómetro y Reloj que marca los minutos y AM/PM en PlusCal 56
Ejercicio 6: Condiciones de carrera 57
Uso de TLA+/Pluscal para analizar el programa benari.c del Ejercicio 6 61
Ejercicio 7: rango de valores de n en benari.tla 64
Especificación de un sistema para la evaluación de prácticas de estudiantes 66
Primera versión 66
Segunda versión 70
Tercera versión 72
Ejercicio 8 76

Especificación del sistema de control semafórico para un puente de un sólo carril 77
La Máquina y el Mundo 78
Abstracciones 82
Constantes y variables 84
Organización del código de la especificación: 87
Exploración de los comportamientos de la especificación con TLC 88
Exploración de los comportamientos de la especificación usando el grafo de estados 90
Ejercicio 9 92
Ejercicio 10 92
Secuencias o tuplas en TLA+ y módulo `Sequences` 93
Ejercicio 11 94
Ejercicios de Conjuntos 98
Ejercicios de Lógica 98
Ejercicios con los cuantificadores existencial y universal 100
Verdades y falsedades por vacuidad 102
Verdad por vacuidad 102
Falsedad por vacuidad 102
Operadores con parámetros y operadores recursivos 104
Protocolo de encaminamiento basado en vectores de distancias 106
Constantes 107
Procesos y variables 107
Verificación con TLC 111
Propiedades y aserciones (encaminamiento basado en vectores de distancias) 112
Propiedades de tipo safety*:* `TypeOK` 112
Propiedades de tipo safety*:* `RoutesFound` 114
Aserciones para las constantes 115
Reflexiones sobre la especificación 117
Ejercicio 12 117
Ejercicio 13 117
Ejercicio 14 117
Ejercicio 15 118
Ejercicio 16 118
Ejercicio 17 118
Protocolo de inundación controlada usado en el encaminamiento basado en el estado de los enlaces 119
Algoritmo de inundación controlada 119
Inundación controlada de los mensajes LINKSTATE 121
Verificación de propiedades 122
Ejercicio 18 123
Ejercicio 19 123
Ejercicio 20 123
Lecturas recomendadas 124
Recursos sobre TLA+/PlusCal 124

Introducción

En muchas profesiones como la arquitectura o la ingeniería, previamente a la construcción de los edificios o las máquinas con las que se pretende dar solución a un problema, se utilizan herramientas como los bocetos, maquetas, planos, simuladores, modelos computacionales, especificaciones formales, etc.

Estas herramientas ayudan a los ingenieros a pensar en el problema y a pensar en el sistema o máquina que se pretende construir para resolver dicho problema. Se trabaja en estas fases iniciales con abstracciones en las que se detallan sólo algunos aspectos del comportamiento de la máquina, dejando de lado otros detalles que no se considera necesario contemplar hasta más adelante.

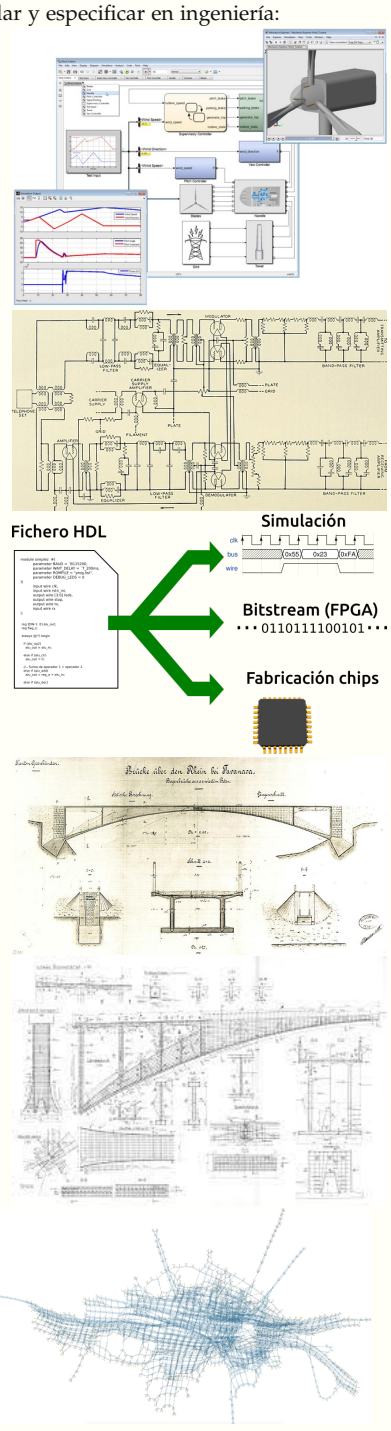
Al realizar la especificación de un sistema el ingeniero puede pensar en el problema que trata de resolver, y puede pensar en diseños alternativos, facilitando el descubrimiento de fallos de diseño en las fases iniciales del proyecto antes de comenzar a construir el sistema. Descubrir un fallo en esta fase inicial del diseño en lugar de descubrirlo cuando el sistema ya está siendo construido es considerado generalmente una buena práctica debido al ahorro de coste que consigue.

Si bien esta práctica no está tan generalizada en la industria del software como en otras áreas de la ingeniería, cada vez es más habitual realizar especificaciones antes de comenzar la implementación, especialmente para construir sistemas complejos (sistemas con requisitos de confiabilidad, seguridad, tolerancia a fallos, sistemas concurrentes y sistemas distribuidos, etc.)

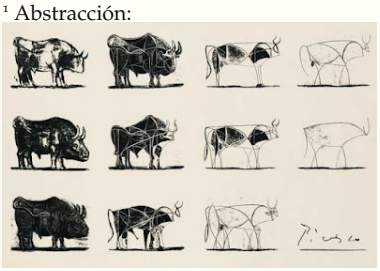
En la industria del software hubo una reacción hace varias décadas en contra de esta práctica habitual en ingeniería, promoviéndose como alternativa la realización de implementaciones de sistemas programando desde el comienzo del proyecto una solución inicial que luego se va refinando. Con demasiada frecuencia esta aproximación, que es muy valiosa, se ha llevado al límite, haciendo creer erróneamente a muchos profesionales que pensar sobre el problema y sus soluciones alternativas, diseñar y especificar partes del software antes de programarlo, es una mala práctica.

Sin embargo, en muchos casos pensar al principio sobre el problema y diseñar soluciones alternativas, si no de todo el sistema sí de las partes más complejas o críticas, puede reducir los costes del proyecto porque al especificar se trabaja en un nivel de abstracción[1] en el que no se tienen en cuenta muchos detalles del código que no son necesarios para encontrar la mejor solución, lo que facilita la exploración de diseños alternativos y el descubrimiento de errores de diseño.

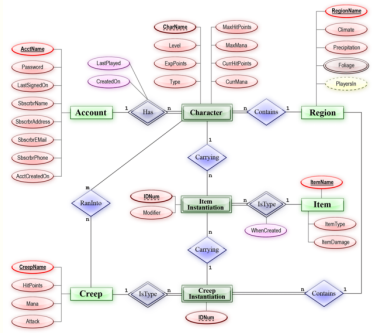
Ejemplos de herramientas para modelar y especificar en ingeniería:



[1] Abstracción:

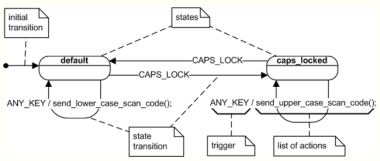


EXISTEN MUCHOS TIPOS DE HERRAMIENTAS PARA AYUDAR EN EL DISEÑO DE SISTEMAS SOFTWARE. Las herramientas de modelado de datos permiten realizar modelos de la estructura de la información que tiene que gestionar un sistema informático. Estas herramientas permiten describir las entidades existentes en el dominio de la aplicación y cómo se relacionan estas entidades entre sí. Los diagramas Entidad / Relación son una herramienta de este tipo[2].



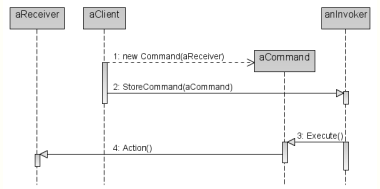
2

Cuando se está concibiendo el software de un sistema se realizan también especificaciones del comportamiento funcional en las que se describen las acciones que deberá llevar a cabo el sistema para satisfacer los requisitos funcionales identificados. Se detalla qué eventos debe aceptar el sistema (p.ej. recibir datos de periféricos o adquirir señales de sensores), qué acciones debe generar el sistema (p.ej. transformar datos, enviar mensajes, escribir datos en periféricos o enviar señales a actuadores), y cuándo deben aceptarse los eventos y generarse las acciones.



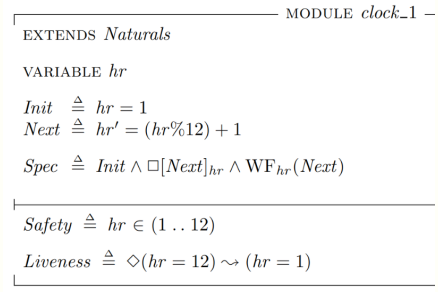
3

Estas especificaciones del comportamiento funcional o lógico se suelen escribir informalmente en lenguaje natural, o gráficamente mediante diagramas como los de estados[3], o los diagramas de secuencia[4], etc.



4

LAS ESPECIFICACIONES FORMALES son una alternativa a las especificaciones informales que se escriben en lenguaje natural o con notaciones gráficas. En una especificación formal se utiliza un lenguaje formal como las matemáticas u otros lenguajes de especificación más parecidos a los lenguajes de programación. La especificación formal detalla el comportamiento del sistema diseñado[5]. Las especificaciones formales permiten describir con mayor precisión y sin ambigüedades el comportamiento funcional o lógico del sistema que se pretende realizar.


MODULE *clock_1* –

EXTENDS *Naturals*

VARIABLE *hr*

$Init \triangleq hr = 1$
$Next \triangleq hr' = (hr\%12) + 1$

$Spec \triangleq Init \wedge \Box[Next]_{hr} \wedge \mathrm{WF}_{hr}(Next)$

$Safety \triangleq hr \in (1 \,..\, 12)$

$Liveness \triangleq \Diamond(hr = 12) \rightsquigarrow (hr = 1)$
5

A partir de los requisitos identificados que debe satisfacer el sistema que se desea construir, se pueden escribir, usando también un lenguaje formal, las propiedades/predicados que debe satisfacer cualquier ejecución correcta del sistema.

Al estar especificado el sistema y las propiedades en un lenguaje formal, se pueden utilizar herramientas para automatizar la tarea de verificar o comprobar[6] que la especificación satisface las propiedades que expresan formalmente los requisitos. Los verificadores de modelos (*model checkers*[7]) son programas que generan todas las ejecuciones posibles de un modelo o instancia del sistema especificado en la especificación formal, y comprueban si dichas ejecuciones satisfacen las propiedades que expresan formalmente los requisitos.

También pueden realizarse demostraciones formales que prueben que las propiedades funcionales que deben cumplir las ejecuciones del sistema especificado no sólo las cumplen las ejecuciones

[6] Usaremos la palabra verificar en este texto. https://dle.rae.es/verificar

verificar Conjugar

Del lat. tardío *verificāre*.

1. tr. Comprobar o examinar la verdad de algo.
2. tr. Realizar, efectuar. U. t. c. prnl. *Las elecciones se verificaron en marzo.*
3. prnl. Salir cierto y verdadero lo que se dijo o pronosticó.

[7] Verificadores de modelos. https://es.wikipedia.org/wiki/Verificaci%C3%B3n_de_modelos

de algún modelo o instancia concreto, como hacen los verificadores de modelos, sino que las cumple cualquier ejecución de cualquier modelo o instancia que se pueda generar a partir de la especificación. Sin embargo las demostraciones formales, a diferencia de la verificación de modelos que está automatizada, requieren mucho esfuerzo humano, lo que es mucho más costoso, haciendo inviable su uso salvo en pequeños sistemas críticos en algunos sectores de la industria del software. Ver https://backend.orbit.dtu.dk/ws/portalfiles/portal/137536957/FormalMethodsNoteTS.pdf. No explicamos nada de demostración formal en estos apuntes.

TLA+[8] es un lenguaje de especificación formal que utiliza la lógica proposicional, la teoría de conjuntos y una lógica temporal para describir el comportamiento del sistema y las propiedades funcionales que las ejecuciones correctas del sistema deben satisfacer. En TLA+ no se especifican otras propiedades importantes distintas de las funcionales, como las relativas al tiempo de ejecución medio del sistema, aunque sí se pueden especificar propiedades relativas al rendimiento del sistema como el caso peor en cuanto a tiempo de ejecución.

[8] TLA+ es el nombre del lenguaje de especificación que utiliza la *Temporal Logic of Actions* (TLA).

PlusCal es un lenguaje algorítmico que extiende TLA+, facilitando la descripción de sistemas con múltiples flujos de ejecución fácilmente, como los sistemas concurrentes y los sistemas distribuidos. El código escrito en PlusCal se traduce automáticamente a especificaciones TLA+. Las especificaciones escritas en TLA+, y por tanto las escritas en PlusCal se pueden analizar y depurar utilizando el verificador de modelos (*model checker*) TLC.

Un verificador de modelos como TLC permite verificar si todos los comportamientos o ejecuciones posibles de una especificación satisfacen las propiedades especificadas. Esta capacidad de los verificadores de modelos como TLC es muy valiosa: el poder comprobar si todas las ejecuciones posibles de un sistema satisfacen o no unas propiedades, permite al ingeniero ganar en poco tiempo confianza sobre la corrección del sistema especificado, antes de comenzar a realizar su implementación. Las especificaciones informales en lenguaje natural o gráficas, no pueden ser procesadas de este modo[9].

[9] Esto justifica aunque sólo en parte el rechazo que algunos profesionales aun siguen teniendo a este tipo de herramientas, debido en muchos casos a su desconocimiento de la tecnología de los verificadores de modelos.

En sectores críticos con requisitos de seguridad o confiabilidad elevados (sector del transporte, centrales nucleares, sistemas financieros, etc.) se han seguido realizando especificaciones antes de comenzar la implementación[10]. Cada vez hay más sectores en la industria en los que se sigue esta práctica utilizando herramientas como TLA+/PlusCal. Empresas como Intel, Amazon, Microsoft u Oracle han introducido en los últimos años el uso de TLA+/PlusCal[11].

[10] Ver secciones 5,6,7 de https://backend.orbit.dtu.dk/ws/portalfiles/portal/137536957/FormalMethodsNoteTS.pdf

[11] Lee este breve y reciente artículo de IEEE Spectrum para saber más al respecto: https://spectrum.ieee.org/tla
También puedes leer este artículo del autor de TLA+/PlusCal Leslie Lamport: http://mobiquo.gsyc.es/isi/wbahwdb-Lamport.pdf



Especificaciones de sistemas digitales

Estamos acostumbrados a modelar el estado de sistemas físicos usando variables cuyos valores reflejan la magnitud de ciertos aspectos de interés del sistema que se quiere modelar. Por ejemplo el estado de un sistema compuesto por la tierra y la luna[12] puede describirse con los valores de cuatro variables que modelen las posiciones y las velocidades de la tierra y de la luna.

La elección de las variables con las que modelar el estado del sistema depende de los aspectos del sistema que resultan de interés, abstrayéndose el resto de detalles del sistema. Cuando especificamos el sistema tierra-luna con 2 variables para las posiciones y 2 variables para las velocidades estamos trabajando en un nivel de abstracción muy elevado, en el que consideramos la tierra y la luna como 2 partículas. No estamos teniendo en cuenta detalles importantes de la realidad modelada, como por ejemplo que la tierra no es una sóla partícula, sino que está compuesta en realidad por $1,3 * 10^{50}$ de partículas llamadas átomos. Sin embargo estos detalles importantes son irrelevantes para calcular trayectorias de proyectiles enviados desde la tierra para alcanzar la luna[13].

Mediante ecuaciones diferenciales podemos describir la dinámica o el comportamiento del sistema físico, es decir, cómo evolucionará el estado del sistema en función del tiempo. El comportamiento del sistema tierra-luna modelado con variables para la posición y la velocidad de ambas partículas se puede describir mediante ecuaciones. Estas ecuaciones expresan cómo van cambiando de manera continua los valores de las variables de la posición y velocidad de la tierra y la luna, en función del tiempo y de ciertas constantes como su masa y las posiciones y velocidades iniciales.

Un sistema digital, software o hardware se puede modelar también mediante variables que reflejan los aspectos de interés, abstrayéndose otros detalles que no son considerados importantes.

Los cambios en el estado del sistema se modelan también mediante asignaciones de nuevos valores a dichas variables. Pero, a diferencia de un sistema físico, cuando modelamos un sistema discreto asumimos que sus estados van cambiando en transiciones o pasos discretos en lugar de hacerlo de modo continuo[14]. Por ello no utilizamos ecuaciones diferenciales para describir su comportamiento. En su lugar, podemos representar el comportamiento de un sistema digital, hardware o software, como una secuencia de estados discretos.

[12] El sistema tierra-luna, imagen NASA, The Harvest Moon, realizada el 29-09-2023:



[13] La elección del nivel de abstracción depende del objetivo que se persigue. El objetivo de la abstracción no es ser menos concreto, sino crear un nuevo nivel semántico en el que se pueda ser absolutamente preciso. Esta es la traducción de una frase del discurso de Edsger W. Dijkstra en su conferencia ACM Turing Lecture, 1972: *"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."* `https://www.cs.utexas.edu/~EWD/transcriptions/EWD03xx/EWD340.html`.

[14] Evidentemente un sistema digital es un sistema físico y por tanto su estado varía de modo continuo. Pero en los sistemas digitales abstraemos este hecho y consideramos que los cambios son discretos. P.ej., el voltaje y la intensidad de los circuitos electrónicos digitales varía de modo continuo pero sólo prestamos atención a los valores alcanzados en ciertos umbrales, considerando de interés sólo 2 estados a los que denominamos cero y uno.

TLA+: Definiciones

Vuelve a esta sección siempre que encuentres alguno de los términos que se definen a continuación.

Un *estado* es una asignación de valores a variables.

Un *comportamiento* o *behavior* es una secuencia infinita de estados del universo en el que se encuentra el sistema que queremos diseñar.

Un *paso* o transición es un pareja de estados contiguos de un comportamiento.

Un *predicado sobre estados* es una fórmula[15] cuyo significado es un predicado (función booleana) que opera sobre estados.

[15] Una fórmula es una expresión que se evalúa a un valor TRUE o a un valor FALSE.

Una *acción* es una fórmula cuyo significado es un predicado que opera sobre pasos o transiciones. En TLA+ las acciones se escriben como fórmulas que contienen variables primadas (ej: `hr'`) y no primadas (ej: `hr`). Las variables no primadas se refieren a los valores de las variables en el primer estado de un paso o transición; las variables primadas se refieren a los valores de las variables en el segundo estado de un paso. Los predicados sobre estados no contienen variables primadas.

Una *fórmula temporal* contiene operadores de lógica temporal como siempre `[]`, eventualmente `<>`, etc. El significado de una fórmula temporal es un predicado sobre comportamientos (*behaviors*), es decir, se evalúa como TRUE o FALSE cuando se aplica a un comportamiento.

Una especificación de un sistema en TLA+ es una fórmula normalmente llamada ***Spec*** compuesta por la conjunción de un predicado sobre estados que determina el estado inicial normalmente denominado ***Init***, y por una fórmula temporal que determina el conjunto de comportamientos que puede tener el sistema a partir del estado inicial: `[](Next)`, siendo `Next` una acción.

El significado de la fórmula *Spec* define por comprensión el conjunto de los comportamientos del universo (secuencias infinitas de estados discretos) que puede experimentar el sistema especificado.

Los nombres de las fórmulas `Init, Next` y `Spec` se utilizan por convenio si bien no tienen por qué llamarse así.

Instalación de las herramientas de TLA+/PlusCal

Se pueden utilizar las herramientas de TLA+ desde la línea de comandos o desde el *TLA Toolbox*[16]. Nosotros utilizaremos las herramientas de TLA+ desde Visual Studio Code (VS Code) por ser una alternativa más flexible y sencilla de utilizar, sin pérdida de funcionalidad.

[16] TLA Toolbox es un IDE (Integrated Development Environment) basado en Eclipse.

Lo PRIMERO que tienes que hacer es instalar el editor Visual Studio Code (VS Code)[17]. En los laboratorios de la EIF ya está instalado.

[17] Puedes descargarlo desde `https://code.visualstudio.com/`

A CONTINUACIÓN tienes que instalar un par de extensiones de VS Code para poder utilizar las herramientas de TLA+ y para poder visualizar diagramas de estados con Graphviz.

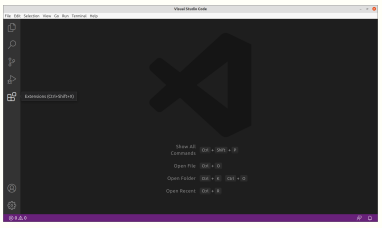
Puedes ver el vídeo con el proceso de instalación[18]. En el vídeo aparece un texto que tienes que escribir para configurar la extensión TLA+ (Temporal Logic of Actions), pero es mejor que lo cortes y lo pegues de la siguiente descripción detallada de los pasos que aparecen en el vídeo:

[18] `http://mobiquo.gsyc.es/isi/install-tla-vscode.mp4`

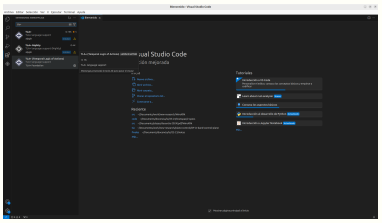
1. Arranca Visual Studio ejecutando `code` en la línea de comandos. Alternativamente puedes pulsar el botón de Ubuntu `Mostrar aplicaciones`[19] y escribe `code` en la caja de texto para que salga el icono de VS Code; luego haz *click* sobre su icono o pulsa la tecla `enter` después para que arranque la aplicación.


[19]

2. Instalación y configuración de las herramientas TLA+/PlusCal dentro de VS Code:


[20]

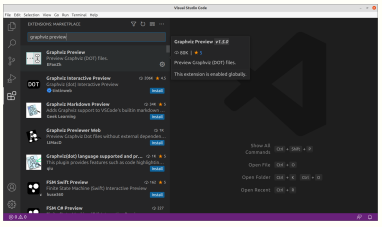
   - Pincha en el icono para instalar extensiones que aparece en la columna de iconos de la izquierda[20] y escribe en la caja de texto `TLA+`
   - En el menú de extensiones que aparece haz click en el icono de `TLA+ (Temporal Logic of Actions)` para que se instale[21].
   - Haz click en el icono de configuración que aparece en el panel que representa la extensión TLA+. En el menú que se despliega haz click en `Extension Settings`. Se abrirá un panel con cajas de texto para configurar parámetros de la extensión de TLA+.


[21]

     Ve a `Tlaplus > Tlc > Model Checker: Options` y escribe:

```
-workers auto -fpmem .9 -deadlock -dump dot,colorize,actionlabels output.dot
```

3. Instalación de la extensión `Graphviz preview` para visualizar los estados generados por TLC para las especificaciones TLA+:


[22]

   - Pincha en el icono para instalar extensiones y busca en la caja de texto la extensión `Graphviz preview`
   - En el menú de extensiones que aparece haz click en el icono de `Graphviz preview` para que se instale[22].

Primera especificación con TLA+: un reloj que sólo marca las horas

Vamos a escribir la especificación en TLA+ de un sistema muy simple: un reloj que marca las horas.

Tras una fase inicial de captura de requisitos, hemos identificado informalmente estos requisitos funcionales:

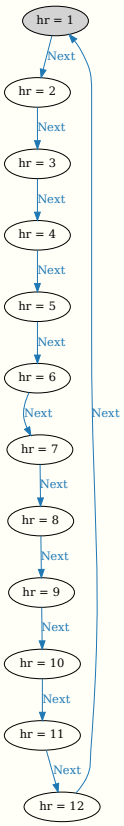
1. el reloj deberá mostrar sólo las horas, números naturales de 1 a 12
2. inicialmente comienza marcando la 1
3. el reloj va marcando las horas a partir de la 1, y tras llegar a las 12 vuelve a marcar la 1
4. va incrementando de una en una unidad la hora y nunca retrocede, salvo cuando pasa de marcar las 12 a marcar la 1.

Para especificar el comportamiento de un reloj que marque las horas vamos a trabajar en un nivel de abstracción muy elevado, olvidando detalles de un reloj real como la pantalla o la relación de las horas que marque con la hora real. La especificación de un reloj que marca sólo las horas es una abstracción que simplemente muestra los valores de 1 a 12 cíclicamente.

Sólo necesitamos una variable para modelar el estado del reloj: la variable `hr` guardará el valor de la cantidad de horas transcurridas, que será un número natural comprendido entre el 1 y el 12.

En la figura de la derecha se muestra un diagrama o grafo de los estados (asignaciones de valores a variables) y los pasos o transiciones entre estados (parejas de estados contiguos en la ejecución) de cualquier ejecución del reloj que queremos construir. Las flechas entre pasos están etiquetadas con el nombre de las acciones de la especificación que provocan ese paso o transición. Este diagrama representa visualmente todos los comportamientos correctos del reloj de horas, comenzando siempre por un estado inicial en el que `hr = 1`, estado que aparece sombreado en gris en la figura.

Diagrama de estados del reloj que marca las horas:



En el fichero `clock_2.tla` que acompaña a estos apuntes y cuyo contenido mostramos un poco más abajo hay una especificación de un reloj que marca las horas comenzando por la 1, como el que pretendemos construir. Esta especificación es una forma alternativa de expresar con el lenguaje formal TLA+ el diagrama de estados de la figura.

La especificación no es un programa. No tiene sentencias que tiene que ejecutar un microprocesador para generar las horas. Contiene fórmulas cuyo significado son predicados que se evalúan a `TRUE` o a `FALSE`, con las cuales se especifica **qué debe hacer** un reloj que marque las horas, **no cómo debe hacerlo**. La especificación especifica el conjunto de comportamientos que puede tener el sistema:

clock_2.tla

```
1 ---- MODULE clock_2 ----
2 EXTENDS Naturals
3
3 VARIABLE hr
4
4 Init == hr = 1
5 Next == hr' = (hr + 1) % 12
6
6 Spec == Init /\ [][Next]_hr
7 ====
```

Se puede generar una versión de las especificaciones en la que los símbolos matemáticos aparecen con la tipografía habitual. Se hace presionando `Ctrl+Shift+P` en VS Code y seleccionando el comando `TLA+: Export module to PDF`.

MODULE *clock_2*

EXTENDS *Naturals*

VARIABLE *hr*

$$Init \triangleq hr = 1$$
$$Next \triangleq hr' = (hr + 1)\%12$$
$$Spec \triangleq Init \wedge \Box[Next]_{hr}$$

- En la línea 1 aparece el nombre del módulo tras la palabra `MODULE`. Tiene que ser el mismo nombre que el del fichero pero sin el sufijo `.tla`. Por delante y por detrás tiene que tener al menos 4 guiones. El módulo termina en la línea 9 con al menos cuatro caracteres `=`.
- En la línea 2 se importa el módulo `Naturals` para poder operar con números naturales en la línea 5, donde se suman y se usa el operador módulo `%`. `EXTENDS` tiene un efecto parecido a `import` en lenguajes como Python o Java, o a `#include` en C/C++.
- En la línea 3 se declaran las variables del reloj, en este caso sólo `hr` después de la palabra reservada `VARIABLE`. TLA+ es un lenguaje no tipado: no especificamos de qué tipo son los valores que pueden almacenarse en las variables.
- En la línea 4 se define la fórmula `Init` cuyo significado es un predicado sobre estados (lo sabemos porque no tiene variables primadas) que define el estado inicial: `hr = 1`. No es una asignación a la variable `hr`, sino una expresión matemática que compara `hr` con el valor 1, por lo que expresa que `Init` es `TRUE` cuando la expresión `hr = 1` es `TRUE`.

  El símbolo `==` sirve para definir la fórmula[23].
- En la línea 5 se define la fórmula `Next`, que es una acción (lo sabemos porque aparecen variables primadas (`hr'`) y no primadas). Su significado es un predicado sobre pasos o transiciones, refiriéndose la variable primada `hr'` al valor de la variable en el segundo estado de un paso o transición, y la variable no primada `hr` a su valor en el primer estado de un paso o transición.

  `Next` es una fórmula que define la única forma en que pueden cambiar los valores de las variables de la especificación en cada transición entre dos estados contiguos de una ejecución[24]. El valor de la variable en el nuevo estado (`hr'`) ha de ser igual al valor en el anterior estado (`hr`) más 1, módulo 12.
- Finalmente, en la línea 8 aparece la especificación del reloj: la fórmula **`Spec`**, que explicamos a continuación.

[23] En cualquier lugar donde aparezca el nombre que está a la izquierda de `==`, este se sustituirá por la evaluación de la expresión a la derecha de `==`.

[24] Sólo aquellos comportamientos o ejecuciones cuyas transiciones satisfagan este predicado forman parte de la especificación.

La especificación es la fórmula `Spec`: `Spec` está definida como la conjunción lógica[25] de la fórmula `Init` y de la fórmula temporal `[][Next]_hr`[26]. El símbolo `[]` es un operador de lógica temporal que significa "siempre". `[][Next]_hr` significa que siempre la acción `Next` es cierta en todos los pasos o transiciones de los comportamientos definidos por esta especificación. `[Next]_hr` se puede escribir así también: `Next \/ hr' = hr`. Significa que, o bien la acción `Next` es `TRUE` y por tanto se ejecuta el paso o transición que describe la acción `Next`, o bien la variable `hr` no cambia, produciéndose un paso tartamudeante o *stuttering*[27].

El significado de la fórmula temporal `Spec` es un predicado que es `TRUE` para un comportamiento[28] dado σ si y solo si `Init` es `TRUE` en el primer estado de σ y todo paso o transición de σ es, o una transición *Next* (una que satisface la formula *Next*) o una transición tartamudeante que deja todas las variables con el mismo valor que tenían. Entre dos estados cualesquiera puede haber un número indeterminado de estados tartamudeantes. En las ejecuciones de una especificación de un sistema que tiene un final, al terminar un comportamiento se asume que hay una secuencia infinita de transiciones tartamudeantes, es decir, que no cambian los valores asignados a las variables que definen el estado de la especificación. Un comportamiento que termina así representa una ejecución que ha concluido.

Nuestro reloj, idealmente, no se debe parar nunca, debiendo marcar la hora como un valor que comienza valiendo 1, se va incrementando, y cuando llega a 12 vuelve a 1. Hablaremos más adelante sobre relojes que nunca se paran.

Podemos entender la especificación `Spec` como una forma de definir por comprensión el conjunto de comportamientos o ejecuciones que puede tener el sistema especificado.

Por ejemplo, estos comportamientos pertenecen al conjunto de los especificados por `Spec`:

- $[hr = 1] \longrightarrow [hr = 2] \longrightarrow [hr = 2] \longrightarrow ... \longrightarrow [hr = 2]$...
- $[hr = 1] \longrightarrow [hr = 2] \longrightarrow [hr = 3] \longrightarrow [hr = 4] \longrightarrow [hr = 5] \longrightarrow [hr = 6] \longrightarrow [hr = 7] \longrightarrow [hr = 8] \longrightarrow [hr = 9] \longrightarrow [hr = 10] \longrightarrow [hr = 11] \longrightarrow [hr = 12] \longrightarrow [hr = 1] \longrightarrow [hr = 2] \longrightarrow ... \longrightarrow [hr = 12] \longrightarrow [hr = 1]$...
- $[hr = 1] \longrightarrow [hr = 1] \longrightarrow [hr = 1] \longrightarrow [hr = 2] \longrightarrow [hr = 3] \longrightarrow [hr = 4] \longrightarrow [hr = 5] \longrightarrow [hr = 6] \longrightarrow [hr = 7] \longrightarrow [hr = 8] \longrightarrow [hr = 9] \longrightarrow [hr = 10] \longrightarrow [hr = 11] \longrightarrow [hr = 12] \longrightarrow [hr = 1] \longrightarrow [hr = 2] \longrightarrow ... \longrightarrow [hr = 12] \longrightarrow [hr = 1] \longrightarrow [hr = 2]$...

Sin embargo, estos comportamientos NO pertenecen al conjunto de los especificados por `Spec`:

- $[hr = 1] \longrightarrow [hr = 2] \longrightarrow [hr = 5] \longrightarrow ...$
- $[hr = 1] \longrightarrow [hr = 2] \longrightarrow [hr = 3] \longrightarrow [hr = 2]$...
- $[hr = 3] \longrightarrow [hr = 4] \longrightarrow [hr = 5] \longrightarrow [hr = 6] \longrightarrow [hr = 7] \longrightarrow [hr = 8] \longrightarrow [hr = 9] \longrightarrow [hr = 10] \longrightarrow [hr = 11] \longrightarrow [hr = 12] \longrightarrow [hr = 1] \longrightarrow [hr = 2] \longrightarrow ... \longrightarrow [hr = 12] \longrightarrow [hr = 1]$...

[25] El operador `/\` es la conjunción lógica, como el operador `and` o `&&` en lenguajes de programación).

El operador `\/` es la disyunción lógica, es decir, el equivalente a `or` o `||` en lenguajes de programación.

[26] Recordamos que una fórmula temporal tiene como significado un predicado sobre comportamientos, es decir, sobre secuencias de estados.

[27] La traducción literal sería pasos tartamudeantes o pasos de tartamudo, haciendo alusión a que la especificación permite que haya transiciones en las que no se modifiquen los valores de `hr`, repitiéndose su valor en el nuevo estado. Permitir este tipo de transiciones tartamudeantes en la especificación del reloj de horas permite reutilizar la especificación para (1) especializarla y para (2) componerla con otras:

1. Podríamos construir una nueva especificación `SpecMin` de un reloj que además de marcar las horas marcase los minutos, especializando la especificación de nuestro reloj de horas. La especificación del nuevo reloj añadiría una variable `min` para contar los minutos. Una transición del nuevo reloj podría modificar la variable `min`, pero dejando sin modificar la variable `hr`:
$$\begin{bmatrix} \mathbf{hr = 3} \\ min = 56 \end{bmatrix} \longrightarrow \begin{bmatrix} \mathbf{hr = 3} \\ min = 57 \end{bmatrix}$$
2. También podríamos componer la especificación de nuestro reloj que marca horas con una nueva especificación de un termómetro en la que se usase una nueva variable `t` para la temperatura. La composición de ambas especificaciones podría tener transiciones entre estados en las que se modificase la temperatura `t` pero no se modificarse la hora `hr`:
$$\begin{bmatrix} \mathbf{hr = 12} \\ t = 19 \end{bmatrix} \longrightarrow \begin{bmatrix} \mathbf{hr = 12} \\ t = 22 \end{bmatrix}$$

[28] Recuerda, un comportamiento o *behavior* es una secuencia de estados

Verificación de modelos con TLC

La especificación del fichero `clock_2.tla` define un conjunto de comportamientos. Al ser una especificación formal escrita en TLA+ podemos utilizar un verificador de modelos (*model checker*) como TLC para que genere todos los comportamientos de una especificación y para verificar si estos satisfacen ciertas propiedades.

CREAMOS UN FICHERO DE CONFIGURACIÓN PARA NUESTRA ESPECIFICACIÓN al que llamamos `clock_2.cfg`:

clock_2.cfg

```
1 SPECIFICATION
2     Spec
```

Tras la palabra reservada de TLC `SPECIFICATION` de la línea 1 escribimos el nombre de la fórmula de nuestra especificación, `Spec`, en la línea 2.

ARRANCAMOS AHORA TLC con la combinación de teclas `Ctrl-Shift-P`, haciendo *click* en la opción `TLA+: Check model with TLC`.

Si hubiera errores sintácticos en el fichero, el nombre del fichero en VS Code se pondría en rojo, y en el menú `View > Problems` podrías desplegar un panel en el que se explican los problemas encontrados en el texto de la especificación. Estos pueden ser errores sintácticos como que no se haya definido una variable, una palabra reservada mál escrita, etc.

Si por el contrario todo ha ido bien, en un nuevo panel se mostrará la salida de TLC[29]. TLC nos informa de que no hay errores (indica `Success` en verde) y de que ha encontrado 12 estados distintos (suponemos que uno por cada hora, de 1 a 12).

NOS HACEMOS AHORA VARIAS PREGUNTAS. Trata de pensar en cada una de ellas antes de ver las respuestas a la derecha.

¿Qué significa exactamente que TLC no ha encontrado errores?[30] ¿Es esta una especificación de un reloj que marque las horas como el que anhelamos especificar, cuyos requisitos definimos más arriba?[31] ¿Genera esta especificación comportamientos correctos como los que expresa el diagrama de estados que mostramos antes, o como los ejemplos que hemos puesto antes de comportamientos correctos?[32]

Como hemos comentado antes, tener una especificación formal nos permite utilizar verificadores de modelos para comprobar propiedades de la especificación. Comenzamos la siguiente sección definiendo los tipos de propiedades que se pueden utilizar para verificar una especificación.

[29] Salida de TLC para `clock_2.tla`:



[30] Significa que la especificación no tiene errores sintácticos. Es como cuando un programa compila sin errores. Pero que no haya errores no significa que la especificación satisfaga los requisitos. No hay nada que ligue los requisitos con esta especificación, por lo que TLC no puede aún comprobar si la especificación satisface los requisitos.

[31] TLC no dice ni que sí ni que no, por ahora.

[32] De nuevo, TLC no dice, por ahora, nada al respecto.

Propiedades de las especificaciones

A partir de los requisitos identificados para el reloj que marca las horas, expresados en lenguaje natural (castellano) más arriba, definiremos ahora en TLA+ las propiedades que debe cumplir el diseño del reloj que marca las horas expresado por la especificación que hemos escrito en TLA+.

Las propiedades son fórmulas cuyo significado puede ser un predicado sobre estados, sobre pasos o sobre comportamientos, dependiendo del tipo de propiedad. TLC, tras generar todos los comportamientos de la especificación, verificará si los estados, los pasos y los comportamientos cumplen las propiedades. Y entonces, TLC sí que podrá ayudarnos a determinar si la especificación satisface o no los requisitos expresados en las propiedades. Evidentemente, para ello es necesario estar seguros de que las propiedades expresadas formalmente en TLA+ reflejan correctamente los requisitos.

Tipos de propiedades: safety, liveness, propiedades de acciones. Una propiedad de tipo *safety* se diseña para detectar "comportamientos malos" no deseados que nunca deben presentarse durante la ejecución del sistema. Las propiedades de tipo *safety* se expresan con fórmulas cuyo significado es un predicado sobre estados. Las propiedades de tipo *safety* son las más usadas para verificar especificaciones. Ejemplo: el sistema nunca alcanza cierto estado malo.

Una propiedad de tipo *liveness* especifica qué "comportamientos buenos" debe tener la especificación. Se expresan mediante fórmulas temporales cuyo significado es un predicado sobre comportamientos. Ejemplo: en algún momento el sistema alcanza cierto estado bueno.

En TLA+ existen también las *propiedades de acciones*, que sirven para comprobar si las transiciones de las acciones se realizan correctamente. Son propiedades que, como las de tipo *safety*, sirven para expresar comportamientos malos que no deben ocurrir nunca, pero se expresan con una fórmula temporal concreta, cuyo significado es un predicado sobre pasos que deben cumplir todos los pasos de cualquier comportamiento.

Si TLC comprueba que la especificación satisface las propiedades de tipo *safety*, *liveness* y de acciones que hemos definido a partir de los requisitos, entonces se incrementará nuestra confianza en el sistema diseñado. Dicho de otro modo, estaremos algo más seguros de que el diseño expresado por la especificación que hemos diseñado hace lo que de manera informal recogen los requisitos.

En caso contrario, si TLC detecta que no se satisface algu-

na propiedad, pueden ocurrir dos cosas: las propiedades podrían estar incorrectamente expresadas, y quizá la especificación en realidad puede que sí satisfaga los requisitos, aunque no satisfaga las incorrectas propiedades. O bien puede ocurrir que tanto las propiedades como la especificación sean incorrectas. Por ello, si usando el verificador de modelos TLC se detecta que la especificación no satisface las propiedades, hay que revisar en primer lugar las propiedades, y si se considera que son correctas, revisar la especificación y corregirla para que satisfaga las propiedades, y por tanto los requisitos.

Relación entre las Propiedades y los Requisitos. Por ahora no es completamente automatizable la traducción de los requisitos expresados en lenguaje natural a las propiedades que tratan de recoger esos mismos requisitos pero expresados en un lenguaje formal como TLA+.

Si bien los LLMs (ChatGPT, Claude, Gemni,...) pueden ayudarnos hoy ya a automatizar la traducción, sigue siendo por ahora necesario que un experto humano determine si las propiedades reflejan correctamente o no los requisitos.

TLC: verificación de propiedades de tipo Safety del reloj que sólo marca las horas

Requisitos:

1. el reloj deberá mostrar sólo las horas, números naturales de 1 a 12
2. inicialmente comienza marcando la 1
3. el reloj va marcando las horas a partir de la 1, y tras llegar a las 12 vuelve a marcar la 1
4. va incrementando de una en una unidad la hora y nunca retrocede, salvo cuando pasa de marcar las 12 a marcar la 1.

Recordamos los requisitos identificados a la derecha.

¿Satisface nuestra especificación estos requisitos? Como hemos visto más arriba, si definimos propiedades formales, el verificador de modelos TLC sí nos podrá ayudar.

Vamos a definir una primera propiedad de tipo *safety*. Para ello escribimos una fórmula cuyo significado sea un predicado sobre estados (no contiene variables primadas):

```
TypeOK == hr ∈ {1,2,3,4,5,6,7,8,9,10,11,12}
```

escrito así en el código fuente:

```
TypeOK == hr \in {1,2,3,4,5,6,7,8,9,10,11,12}
```

El predicado definido en la fórmula `TypeOK` será `TRUE` cuando `hr` tenga un valor que pertenece (`\in`) al conjunto definido entre llaves por extensión (nombrando todos sus elementos).

El módulo `Naturals` que estamos utilizando en nuestra especificación nos permite escribir esta fórmula de este otro modo más cómodo:

```
TypeOK == hr \in (1..12)
```

Es habitual en todas las especificaciones escribir una propiedad de tipo *safety* llamada `TypeOK` como esta.

En el caso del reloj que sólo marca las horas, esta propiedad recoge el requisito 1: cualquier estado de cualquier comportamiento de nuestro reloj de horas debería satisfacer este predicado. Es decir, el valor de `hr` no puede ser otro que un valor comprendido entre 1 y 12. Esta es una propiedad de tipo *safety* pues excluye "comportamientos malos" no deseados: aquellos en los que el valor de `hr` no pertenece al conjunto `(1..12)`.

La forma de verificar las propiedades de tipo *safety* es comprobando si el predicado expresado en la fórmula `TypeOK` es una invariante. El predicado es una invariante si es `TRUE` en todos los estados de todos los comportamientos que satisfacen la especificación `Spec`.

Si al menos en un estado de un comportamiento de `Spec` esta fórmula no es `TRUE`, entonces no es una invariante y por tanto la especificación no satisface el requisito expresado por la propiedad de tipo *safety* que se recoge en la fórmula `TypeOK`.

Para verificarlo usamos TLC: añadimos `TypeOK` al fichero de nuestra especificación y lo guardamos con el nuevo nombre `clock_3.tla`, cuyo contenido mostramos en la siguiente página:

clock_3.tla

```
1 ---- MODULE clock_3 ----
2 EXTENDS Naturals
3
4 VARIABLE hr
5
6 Init == hr = 1
7 Next == hr' = (hr + 1) % 12
8
9 Spec == Init /\ [][Next]_hr
10 ----
11 TypeOK == hr \in (1..12)
12 ====
```

La línea 7 es decorativa, sirve para separar nuestra especificación de las líneas 1-6 (la misma que teníamos en `clock_2.tla`) de la propiedad de tipo *safety* `TypeOK`.

A continuación creamos el fichero de configuración para TLC que se corresponde con esta nueva especificación, `clock_3.cfg`:

clock_3.cfg

```
1 SPECIFICATION
2   Spec
3
4 INVARIANTS
5   TypeOK
```

Hemos añadido en las líneas 3 y 4 la fórmula `TypeOK`. Al ponerla debajo de la palabra reservada `INVARIANTS` le estamos pidiendo a TLC que compruebe que el predicado definido por la fórmula `TypeOK` es `TRUE` en todos los estados de todos los comportamientos de la especificación Spec del fichero `clock_3.tla`.

ARRANCAMOS DE NUEVO TLC con la combinación de teclas `Ctrl-Shift-P`, y eligiendo en el menú desplegado la opción `TLA+: Check model with TLC`.

En un nuevo panel se mostrará la salida de TLC[33].

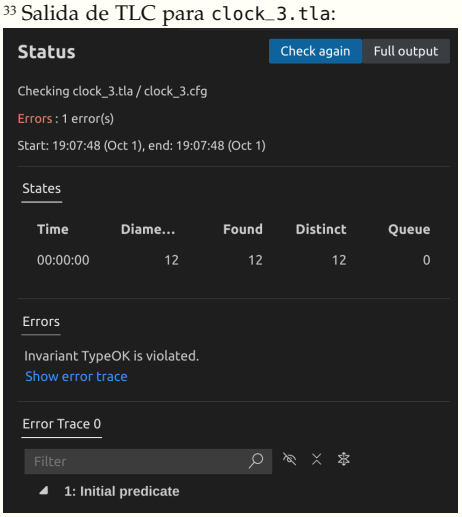
Como no hemos modificado la especificación, ya que simplemente hemos añadido la fórmula `TypeOK`, TLC vuelve a indicar que ha encontrado 12 estados distintos.

Pero en esta ocasión TLC nos informa de que ha encontrado errores (indica `Errors: 1 error`). Además sale este texto en el que nos advierte de que ha encontrado un comportamiento en el que se viola (no se satisface) la propiedad de tipo *safety* que se expresa en la invariante de la fórmula `TypeOK`:

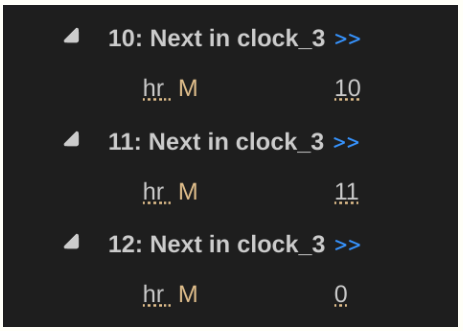
```
Errors

Invariant TypeOK is violated.
Show error trace
```

[33] Salida de TLC para `clock_3.tla`:



Y debajo aparece la traza de uno de los comportamientos que ha encontrado TLC en el que no se ha cumplido la propiedad `TypeOK` (TLC dice que se ha "violado" la invariante `TypeOK`). Los estados del comportamiento aparecen numerados hasta el estado en el que se ha detectado el incumplimiento de la propiedad. En la figura de la derecha[34] aparecen los últimos 3 estados (10, 11, 12) de la traza que muestra TLC.



[34]

En cada estado del comportamiento que ha encontrado TLC aparece una `M` al lado de la variable modificada en cada paso.

En el último estado del comportamiento mostrado por la traza, el 12 en la figura, aparece el estado en el que no se satisfizo `TypeOK`. Vemos que la variable `hr` tiene el valor 0 en el estado 12, después de tener el valor 11 en el estado anterior. Evidentemente ese valor hace que el predicado definido en `TypeOK` no sea `TRUE`, pues el valor 0 de `hr` no pertenece al conjunto `(1..12)`.

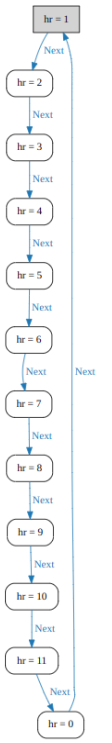
Cada vez que se ejecuta TLC genera un fichero llamado `output.dot` en el que se guarda el diagrama de los estados que ha generado. Vuelve a usar TLC para verificar la especificación `clock_2.tla`[35]. Después abre el fichero `output.dot` y pulsa en el icono que sale arriba a la derecha con una lupa[36]. Verás la figura[37] que mostramos a la derecha con el diagrama de estados. Se aprecia que este diagrama de estados no refleja el reloj de horas que queremos construir: después del estado en el que `hr = 11` transita al estado en el que `hr = 0`.

Que nuestra especificación tuviese un fallo, y que TLC lo haya detectado gracias a una propiedad de tipo *safety* que hemos expresado en la fórmula `TypeOK` y que expresa el requisito 1, **es una gran noticia**: TLA+ nos permite diseñar sistemas más confiables.

Piensa por qué falla la especificación antes de seguir leyendo la siguiente página.

[35] Si usas TLC para verificar `clock_3.tla` obtendrás el diagrama de la traza en la que falla. Preferimos que muestres la generada por `clock_2.tla`, que al no tener la invariante, muestra todos los estados y transiciones.

[36] Si no te sale el icono de la lupa puedes ejecutar el siguiente comando: `Ctrl-Shift-P` y escribir `Graphviz: Open Preview to the Side`



[37]

Podemos localizar fácilmente el fallo de la especificación que ha detectado TLC: la fórmula `Next` está mal definida:

```
Next == hr' = (hr + 1) % 12
```

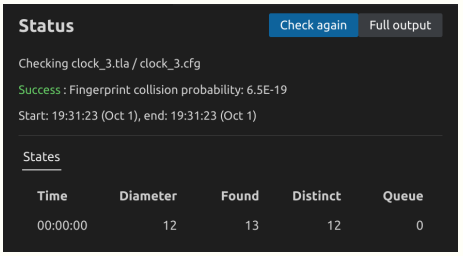
Cuando en el estado `11` de la traza mostrada por TLC el valor de `hr` es `11`, en el siguiente paso `hr'` pasa a valer `(11 + 1) % 12`, que es 0, como muestra en el estado `12` la traza de TLC.

Corregimos en `clock_3.tla` esa línea:

```
Next == hr' = (hr % 12) + 1
```

Ahora cuando `hr` valga `11`, `hr'` valdrá `(11 % 12)+1`, que es `12`. Y cuando `hr` valga `12`, `hr'` valdrá `(12 % 12) + 1`, que es `1`.

Si volvemos a ejecutar TLC esta vez no detecta errores: verifica que la invariante se cumple en todos los estados de todos los comportamientos expresados por `Spec`[38].



[38]

Ahora TLC no sólo no detecta errores sintácticos en la especificación, sino que además ha comprobado que todos los estados de todos los comportamientos posibles expresados por la especificación, cumplen la propiedad de tipo *safety*. Es decir, esta propiedad es una invariante.

ESTA EXPLORACIÓN SISTEMÁTICA DE TODOS LOS ESTADOS DE TODAS LOS COMPORTAMIENTOS de una especificación, y la comprobación de que los estados y las transiciones de todos los comportamientos cumplen o no ciertas propiedades, es lo que hace de la verificación de modelos (*model checking*) una tecnología tan útil. En caso de que una invariante no se cumpla en algún estado de alguno de los comportamientos, TLC interrumpirá la exploración de estados y nos mostrará la traza concreta, lo que facilita detectar problemas de diseño antes de implementar en hardware o en software.

En la siguiente sección estudiamos las propiedades de tipo *liveness*.

TLC: verificación de propiedades de tipo Liveness del reloj que sólo marca las horas

Una propiedad de tipo *liveness* expresa "comportamientos buenos" deseados que debe tener el sistema.

Las propiedades de tipo *liveness* se expresan mediante fórmulas temporales, es decir, fórmulas cuyo significado es un predicado sobre comportamientos (secuencias infinitas de estados). Son fórmulas en las que aparecen los símbolos de la lógica temporal como $\Diamond A$ [39] (eventualmente, ahora o en algún momento del futuro A es cierto), $\Box A$ [40] (siempre, en todos los estados de todos los comportamientos, A es cierto), o $A \rightsquigarrow B$ [41] (si en un estado es cierto A, en algún estado posterior, no necesariamente en el siguiente, será cierto B).

[39] En ASCII: `<>A`

[40] En ASCII: `[]A`

[41] En ASCII: `A~>B`

Recordamos de nuevo los requisitos identificados a la derecha.

Requisitos:

1. el reloj deberá mostrar sólo las horas, números naturales de 1 a 12
2. inicialmente comienza marcando la 1
3. **el reloj va marcando las horas a partir de la 1, y tras llegar a las 12 vuelve a marcar la 1**
4. va incrementando de una en una unidad la hora y nunca retrocede, salvo cuando pasa de marcar las 12 a marcar la 1.

Podemos expresar el requisito 3 de nuestro reloj de horas como una propiedad de tipo *liveness*: el reloj en algún momento estará en el estado cuyo valor de hr es 12, y posteriormente estará en el estado cuyo valor es 1. Y siempre se comportará así.

Es decir, un comportamiento bueno esperado es que el reloj llegue en algún momento a marcar las 12, y después marque la 1. Y que este comportamiento se repita indefinidamente. Podemos expresar esta propiedad en TLA+ usando lógica temporal:

```
AlwaysCycle == []<>(hr = 12 /\ (hr = 12 ~> hr = 1))
```

`~>` en la expresión `(hr = 12 ~> hr = 1)` significa que después de que `hr` valga 12, `hr` acabará valiendo 1, no necesariamente en un sólo paso[42].

Así pues podemos expresar informalmente la fórmula `AlwaysCycle` como:

> Siempre (`[]`), los comportamientos de nuestro reloj en algún momento (`<>`) alcanzarán el estado `hr = 12`, y después de este alcanzará (`~>`) el estado `hr = 1`, no necesariamente en un paso.

[42] ¿Qué significa "no necesariamente en un sólo paso"?. Si bien un reloj correcto pasará de las 12 a la 1 en un sólo paso, si hubiéramos escrito `AlwaysCycle == []<>(hr = 12 /\ (hr = 12 ~> hr = 3))`, significaría que acabará valiendo 3, pero puede que necesite pasar a través de más de un estado para llegar a ese valor. De hecho, este caso, para un reloj correcto, sería en 3 pasos o transiciones. Un reloj incorrecto que pase de marcar las 12 a marcar las 3 en un solo paso también satisfacería la fórmula `AlwaysCycle`. Hay que tener en cuenta que es una propiedad de tipo *liveness*. Cabe esperar que otras propiedades de tipo *safety* o *de acciones* comprueben que el reloj avanza de una en una hora.

En `clock_4.tla` guardamos la nueva propiedad de tipo *liveness*:

clock_4.tla

```
---- MODULE clock_4 ----
EXTENDS Naturals

VARIABLE hr

Init == hr = 1
Next == hr' = (hr % 12) + 1

Spec == Init /\ [][Next]_hr
----
TypeOK == hr \in (1..12)
AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
====
```

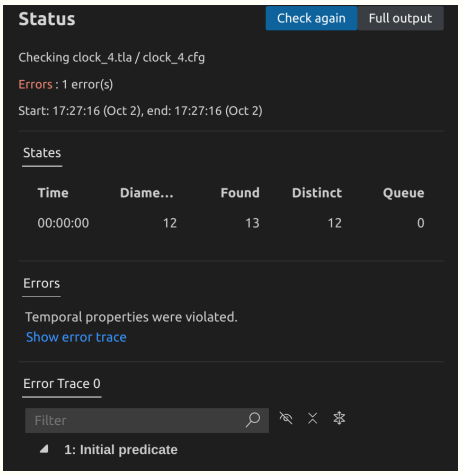
Las fórmulas temporales que definen propiedades de tipo *liveness* que queremos que compruebe TLC hay que ponerlas en la sección `PROPERTIES` del fichero de configuración. Añadimos una nueva sección `PROPERTIES` en `clock_4.cfg`:

clock_4.cfg

```
SPECIFICATION
  Spec

INVARIANTS
  TypeOK

PROPERTIES
  AlwaysCycle
```



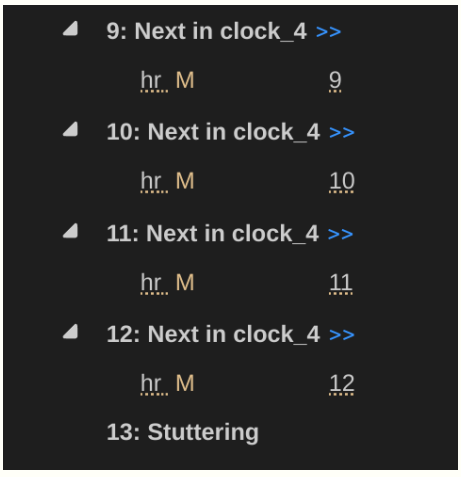
ARRANCAMOS DE NUEVO TLC. TLC ha encontrado un error[43].

En este caso nos avisa de que ha detectado la violación (no satisfacción) de una propiedad temporal, que evidentemente es `AlwaysCycle`:

```
Errors

Temporal properties were violated.
Show error trace
```

Inspeccionamos la traza que nos muestra TLC[44]. Tras el estado 12 en el que `hr = 12`, TLC ha encontrado una ejecución en la que después de este estado se pasa o transita a un estado tartamudeante (*stuttering*) numerado como 13, y después ya no muestra ningún estado más. Esta es la forma que tiene TLC de expresar que el 13 sería el primero de una secuencia infinita de estados en los que `hr` no cambia de valor, permanece fijo en su último valor, las 12.



¿Qué significa esto?

Recuerda que dijimos que es buena práctica permitir los estados *stuttering*, para lo cuál poníamos el subíndice `hr` en la fórmula temporal `[][Next]_hr` de nuestra especificación `Spec`, fórmula que también podemos escribir como `[](Next \/ hr' = hr)`.

Estos estados tartamudenates nos sirven para modelar comportamientos en los que nuestra máquina, nuestro reloj de horas, falle: tras marcar las 12 se para. Ya vimos que es uno de los comportamientos que queremos que formen parte de los que satisface la fórmula `Spec`. El problema que hemos detectado es que la nueva propiedad `AlwaysCycle` no se satiface en los comportamientos que terminan en un estado tartamudeante.

La propiedad que queremos expresar en realidad es: si el reloj no se para, siempre debe ocurrir que en algún momento marque las 12, y después, no necesariamente en un sólo paso, marque la 1.

Para ello añadiremos un nuevo término a la fórmula `Spec` que exprese que la acción `Next` siempre se acabará ejecutando tarde o temprano, evitando así precisamente las secuencias infinitas de estados tartamudeantes al final de los comportamientos del reloj. Queremos que los comportamientos de nuestra especificación puedan tener secuencias de estados tartamudeantes, pero no secuencias infinitas de estados tartamudeantes:

```
Spec == Init /\ [][Next]_hr /\ WF_hr(Next)
```

`WF` significa *Weak Fairness*[45].

[45] Hay otra variante llamada *Strong Fairness* que no estudiamos por ahora.

Con `WF_hr(Next)` estamos exigiendo que la especificación sólo incluya comportamientos en los que si la acción `Next` se puede ejecutar, tarde o temprano se acabe ejecutando y modificando consecuentemente `hr`, quizá tras algunos pasos tartamudeantes.

Fairness se puede traducir por justicia, o no discriminación. No discriminación en el sentido de que la acción `Next` debe ejecutarse alguna vez, con el consguiente cambio de estado, en lugar de permanecer en el mismo estado tartamudeante sin dar la oportunidad de ejecutar la acción `Next`.

Guardamos en `clock_5.tla` la nueva especificación, sustituyendo la definición de la fórmula `Spec` por esta nueva, y copiamos `clock_4.cfg` a `clock_5.cfg`.

Finalmente ejecutamos TLC. TLC ahora termina exitosamente: todos los comportamientos que satisfacen la especificación `Spec` satisfacen también la propiedad de tipo *safety* `TypeOK`, y también la propiedad de tipo *liveness* `AlwaysCycle`.

Podemos considerar que añadir a `Spec` *Weak Fairness* para la acción `Next` es un truco en esta fase de diseño para poder explorar si se cumple la propiedad `AlwaysCycle`. Un truco en el sentido

de que el reloj ahora especificado nunca se para, algo que en la realidad es imposible. Visto de otro modo: es la forma de poder comprobar si nuestra especificación se comporta como debe cuando el reloj no está parado.

TLC: verificación de propiedades de acciones del reloj que sólo marca las horas

Aparte de las propiedades de tipo *safety* que expresamos como invariantes que han de satisfacerse en todos los estados, y de las propiedades de tipo *liveness* que se expresan con fórmulas temporales cuyo significado son predicados sobre comportamientos, existen las propiedades de acciones.

Con las propiedades de acciones podemos expresar condiciones sobre los pasos o transiciones que se expresan en las acciones. Son una fórmula temporal pero siempre con la estructura `[][A]_var`, donde `A` es una acción, es decir, un predicado que opera sobre pasos o transiciones, es decir, una expresión en la que aparecen variables primadas y no primadas[46].

[46] `[A]_var` significa lo mismo que `A \/ UNCHANGED var`. Al ser `A` una fórmula con variables primadas, `[][A]_var` significa que siempre se cumple la fórmula `A` en todas las transiciones o pasos, o bien no cambian las variables en `var`

Recordamos de nuevo los requisitos identificados a la derecha.

Requisitos:

1. el reloj deberá mostrar sólo las horas, números naturales de 1 a 12
2. inicialmente comienza marcando la 1
3. el reloj va marcando las horas a partir de la 1, y tras llegar a las 12 vuelve a marcar la 1
4. **va incrementando de una en una unidad la hora y nunca retrocede, salvo cuando pasa de marcar las 12 a marcar la 1.**

El requisito 4 lo podemos recoger en una propiedad de acciones que exprese que el reloj, cuando cambia de hora, lo hace siempre hacia adelante, de una en una hora cada vez, no disminuyendo nunca su valor, salvo cuando `hr = 12`, en cuyo caso sí que el valor de `hr` en el siguiente estado (es decir `hr'`), pasa a ser 1.

La fórmula `AlwaysUp` define esta propiedad:

```
AlwaysUp == [][(hr /= 12 /\ hr' = hr + 1) \/ (hr = 12 /\ hr' = 1)]_hr
```

Se trata de una fórmula temporal, en la que con el operador `[]` expresamos que siempre los valores de `hr` vayan cambiando tal como indica el requisito 4.

El operador `\/` es la disyunción lógica, es decir, el equivalente a `or` o `||` en lenguajes de programación.

Añadimos esta fórmula a la especificación, que ahora guardamos en el fichero `clock_6.tla`:

clock_6.tla

```
1  ---- MODULE clock_6 ----
2  EXTENDS Naturals
3  VARIABLE hr
4
5  Init == hr = 1
6  Next == hr' = (hr % 12) + 1
7
8  Spec == Init /\ [][Next]_hr /\ WF_hr(Next)
9  ----
10 TypeOK == hr \in (1..12)
11 AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
12 AlwaysUp == [][(hr /= 12 /\ hr' = hr + 1) \/
13               (hr  = 12 /\ hr' = 1)]_hr
14 ====
```

Finalmente añadimos `AlwaysUp` como una propiedad al fichero `clock_6.cfg`:

clock_6.cfg

```
SPECIFICATION
  Spec

INVARIANTS
  TypeOK

PROPERTIES
  AlwaysCycle
  AlwaysUp
```

Ejecutamos TLC y verificamos que la propiedad `AlwaysUp` la satisfacen todos los comportamientos de Spec[47].

[47] Observa que Always

Refactorizaciones de la especificación del reloj que sólo marca las horas

Tener definidas las propiedades `TypeOK`, `AlwaysCycle` y `AlwaysUp` nos permite probar especificaciones alternativas[48].

Por ejemplo, en esta nueva versión del reloj del fichero `clock_7.tla` introducimos una forma alternativa de escribir en TLA+ la acción `Next` utilizando una nueva expresión de TLA+: `IF THEN ELSE`, que siempre debe tener la rama `THEN` y la rama `ELSE`[49]:

clock_7.tla

```
1  ---- MODULE clock_7 ----
2  EXTENDS Naturals
3
3  VARIABLE hr
4
4  Init == hr = 1
5  Next == IF hr = 12 THEN hr' = 1
6          ELSE hr' = hr + 1
7
7  Spec == Init /\ [][Next]_hr /\ WF_hr(Next)
8  ----
9  TypeOK == hr \in (1..12)
10 AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
11 AlwaysUp == [][(hr /= 12 /\ hr' = hr + 1) \/
12                (hr  = 12 /\ hr' = 1)]_hr
13 ====
```

Copiamos el fichero `clock_6.cfg` en el fichero `clock_7.cfg` y ejecutamos TLC.

Verificamos que las propiedades `TypeOK`, `AlwaysCycle` y `AlwaysUp` son satisfechas también por esta forma alternativa y equivalente de especificar el reloj de horas.

[48] Normalmente se utiliza el termino refactorizar para referirse a modificaciones del código que pretenden mejorar algún aspecto como la legibilidad sin modificar la funcionalidad.

[49] Es lógico pues es una expresión, por lo que debe evaluarse siempre a un valor booleano.

`IF THEN ELSE` es azúcar sintáctico. Podemos intentar expresar la acción `Next` de una tercera forma alternativa:

clock_8.tla

```
---- MODULE clock_8 ----
EXTENDS Naturals

VARIABLE hr

Init == hr = 1
Next == (hr = 12 /\ hr' = 1) \/ hr' = hr + 1

Spec == Init /\ [][Next]_hr /\ WF_hr(Next)
----
TypeOK == hr \in (1..12)
AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
AlwaysUp == [][(hr /= 12 /\ hr' = hr + 1) \/
               (hr  = 12 /\ hr' = 1)]_hr
====
```

TLC indica que esta especificación sin embargo viola o no satisface la invariante `TypeOK`. Piensa por qué antes de seguir y trata de modificar `Next` para que TLC no de errores. Repetimos: piensa antes de seguir leyendo.

Lo que ocurre es que la fórmula `Next` está definida como una disyunción lógica:

```
Next == (hr = 12 /\ hr' = 1) \/ hr' = hr + 1
```

Y por tanto, `Next` es `TRUE` cuando el término izquierdo `(hr = 12 /\ hr' = 1)` es `TRUE`, cuando el término derecho `hr' = hr + 1` es `TRUE` y cuando ambos términos son `TRUE`[50].

[50] Tabla de verdad de `A \/ B`:

A	B	A \/ B
FALSE	FALSE	FALSE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
TRUE	TRUE	TRUE

Por tanto, la fórmula `Next` es `TRUE` cuando se incrementa en una unidad la hora porque el término derecho de la disyunción es `hr' = hr + 1`, independientemente del valor de `hr`. Eso provoca que la especificación tenga comportamientos en los que cuando `hr` vale 12 pasa a tener en el siguiente estado el valor 13, y luego 14,... lo que hace que la propiedad `AlwaysUp` no se satisfaga en dichos comportamientos.

Arreglamos la fórmula `Next` para que sólo se incremente en una unidad `hr` cuando no vale 12:

clock_9.tla

```
1  ---- MODULE clock_9 ----
2  EXTENDS Naturals
3  VARIABLE hr
4  Init == hr = 1
5  Next == (hr = 12  /\ hr' = 1)      \/
6          (hr /= 12 /\ hr' = hr + 1)
7  Spec == Init /\ [][Next]_hr /\ WF_hr(Next)
8  ----
9  TypeOK == hr \in (1..12)
10 AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
11 AlwaysUp == [][(hr /= 12 /\ hr' = hr + 1) \/
12                (hr  = 12 /\ hr' = 1)]_hr
13 ====
```

Puedes comprobar que ahora TLC sí verifica que la especificación `Spec` cumple las 3 propiedades. Con esto damos por concluida la especificación del reloj que sólo marca las horas. Repasa los requisitos y comprueba cómo, tanto esta especificación como las anteriores que TLC verificaba exitosamente, al satisfacer las 3 propiedades definidas `TypeOK`, `AlwaysCycle` y `AlwaysUp`, satisfacen los requisitos que estas propiedades expresan formalmente:

- el requisito 1 se recoge en la propiedad de tipo *safety* `TypeOk`
- el requisito 2 se recoge en la fórmula `Init` que forma parte de la propia especificación
- el requisito 3 se recoge en la propiedad de tipo *liveness* `AlwaysCycle`
- el requisito 4 se recoge en la propiedad de acciones AlwaysUp

Reflexionamos finalmente sobre las refactorizaciones: si comparas la fórmula `AlwaysUp` con la fórmula `Next` final verás que ¡son la misma fórmula!.

No tiene mucho sentido que TLC verifique la especificación `Spec` en la que interviene la fórmula `Next` con una propiedad `AlwaysUp` que es idéntica. Siempre que sea `TRUE` `Next` también lo será `AlwaysUp`, por lo que no nos sirve para ganar confianza en que la especificación satisface los requisitos[51].

[51] Ocurre algo similar cuando para testar un programa primero ejecutas el programa y ves el resultado, y después escribes un test que comprueba que ese es el resultado que devolverá el programa. Un test así no sirve para nada.

Sin embargo, recuerda lo que nos ha traido hasta aquí: primero escribimos la propiedad `AlwaysUp` y verificamos que la especificación original y la refactorización con `IF THEN ELSE` satisfacían dicha propiedad. Nos sirvió también para detectar que la tercera refactorización tenía un fallo y no satisfacía la propiedad `AlwaysUp`. Arreglamos este fallo dejando `Next` definida finalmente igual que `AlwaysUp`.

Concluimos pues que la propiedad `AlwaysUp` está bien escrita. Pero si desde el principio, antes de las refactorizaciones, hubiésemos definido `Next` tal como ha quedado tras la última refactorización, no habría tenido sentido definir así la propiedad `AlwaysUp`.

Cambio en los requisitos del reloj que sólo marca las horas: nuevo estado inicial

Supongamos que cambian ahora los requisitos respecto al estado inicial. Marcamos en negrita el cambio en la nueva lista de requisitos que mostramos a la derecha:

Requisitos:

1. el reloj deberá mostrar sólo las horas, números naturales de 1 a 12
2. **inicialmente el reloj puede marcar una hora cualquiera, de 1 a 12**
3. el reloj va marcando las horas, y tras llegar a las 12 vuelve a marcar la 1
4. va incrementando de una en una unidad la hora y nunca retrocede, salvo cuando pasa de marcar las 12 a marcar la 1.

La siguiente especificación refleja este cambio, que afecta exclusivamente a la fórmula `Init`:

clock_10.tla

```
---- MODULE clock_10 ----
EXTENDS Naturals

VARIABLE hr

Init == hr \in (1..12)
Next == (hr = 12  /\ hr' = 1)      \/
        (hr /= 12 /\ hr' = hr + 1)

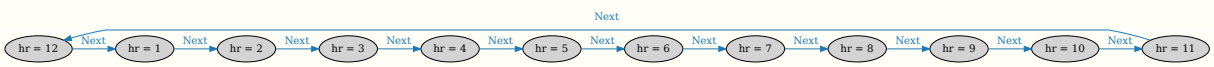
Spec == Init /\ [][Next]_hr /\ WF_hr(Next)
----
TypeOK == hr \in (1..12)
AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
AlwaysUp == [][(hr /= 12 /\ hr' = hr + 1) \/
               (hr  = 12 /\ hr' = 1)]_hr
====
```

Ejecutamos TLC y verificamos que nuestra especificación satisface también las 3 propiedades `TypeOK`, `AlwaysCycle` y `AlwaysUp`.

Cambiar la definición de `Init` hace que nuestra especificación incluya muchos otros comportamientos nuevos ya que ahora hay 12 posibles estados iniciales. Al ejecutar TLC puedes verificar que termina sin errores: todos los comportamientos generados por TLC, teniendo en cuenta que ahora ha tenido que generar comportamientos que comienzan en cualquiera de los 12 estados iniciales posibles, satisfacen las 3 propiedades `TypeOK`, `AlwaysCycle` y `AlwaysUp`.

`Es admirable la facilidad` con la que usando verificadores de modelos podemos explorar diseños alternativos: cambiar una línea de la especificación en este caso no ha requerido que hagamos nada especial para que TLC compruebe que satisface las propiedades y por tanto los requisitos. Visualizando el fichero `output.dot`[52] podemos ver el diagrama de estados en el que esta vez no aparece un único estado inicial, sino que todos los estados son estados iniciales posibles (aparecen sombreados):

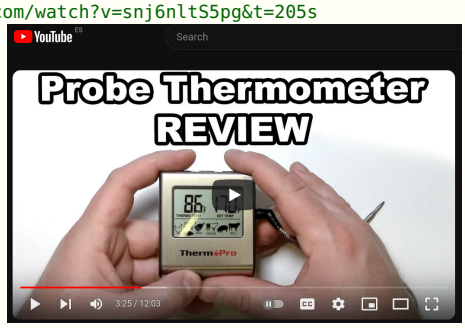
[52] Carga en VS Code el fichero `output.dot` que se genera cuando ejecutas TLC. En el panel que se abre arriba a la derecha aparece un icono de una lupa. Pincha ahí y se mostrará el diagrama del grafo de estados. Si no te sale el icono de la lupa puedes ejecutar el siguiente comando: `Ctrl-Shift-P` y escribir `Graphviz: Open Preview to the Side`



Especificación de un termómetro en TLA+

Vamos a diseñar un termómetro que muestre en una pantalla los dígitos de la temperatura medida por un sensor de temperatura[53] . Mostramos los requisitos expresados informalmente a continuación.

[53] Puedes ver un vídeo de un termómetro de este tipo: https://www.youtube.com/watch?v=snj6nltS5pg&t=205s



Requisitos:

1. El sensor de temperatura devuelve valores enteros comprendidos entre -20 y 60 grados centígrados que deberán ser mostrados en la pantalla del termómetro.

2. Ante un cambio brusco de temperatura el sensor devolverá un valor demasiado alejado del que muestra la pantalla. Por ello, se mostrará el cambio mediante varios valores intermedios, para que el usuario perciba que está cambiando bastante la temperatura en lugar de ver un único cambio del valor actual al valor medido por el sensor.

   Por ello, la pantalla no mostrará directamente el nuevo valor de temperatura devuelto por el sensor, sino que se mostrarán incrementalmente los valores de temperaturas intermedias desde el valor actual que muestra la pantalla, hasta llegar al valor leído del sensor, en incrementos lo más grandes posible; estos incrementos estarán acotados por un tamaño de incremento máximo de 5 grados en valor absoluto[54].

[54] Ejemplo: si la temperatura mostrada en la pantalla es 9 y la nueva temperatura leída del sensor es 20, suponiendo incrementos máximos de 5 grados, la pantalla mostrará los valores 14, 19, 20.

3. Hasta que no se haya mostrado en la pantalla el valor del último valor leído del sensor de temperatura no se leerá un nuevo valor del sensor de temperatura.

A la derecha mostramos una especificación para un termómetro que satisfaga los anteriores requisitos[55] y a continuación mostramos la misma especificación en ASCII, y la explicamos:

MODULE *thermometer*

EXTENDS *Integers*
VARIABLE *display*, *sensor*
$RANGE_TEMPS \triangleq (-1\,..\,1)$
$MAX_INCREMENT \triangleq 2$
$Init \triangleq \land sensor \in RANGE_TEMPS$
$\quad\quad \land display \in RANGE_TEMPS$
$ReadSensor \triangleq$
$\quad \land display = sensor$
$\quad \land \exists\, newTemperature \in RANGE_TEMPS : sensor' = newTemperature$
$\quad \land display' = display$
$UpdateDisplay \triangleq$
$\quad \land$ IF $display > sensor$ THEN
$\quad\quad$ IF $(display - sensor) > MAX_INCREMENT$ THEN
$\quad\quad\quad display' = display - MAX_INCREMENT$
$\quad\quad$ ELSE
$\quad\quad\quad display' = display - ((display - sensor)\%MAX_INCREMENT)$
$\quad$ ELSE IF $display < sensor$ THEN
$\quad\quad$ IF $(sensor - display) > MAX_INCREMENT$ THEN
$\quad\quad\quad display' = display + MAX_INCREMENT$
$\quad\quad$ ELSE
$\quad\quad\quad display' = display + ((sensor - display)\%MAX_INCREMENT)$
$\quad$ ELSE FALSE
$\quad \land$ UNCHANGED *sensor*
$Next \triangleq ReadSensor \lor UpdateDisplay$
$Spec \triangleq Init \land \Box[Next]_{(display,\, sensor,\, MAX_INCREMENT)}$
$\quad\quad \land \mathrm{SF}_{sensor}(ReadSensor)$
$\quad\quad \land \mathrm{SF}_{display}(UpdateDisplay)$

$TypeOK \triangleq \land sensor \in RANGE_TEMPS$
$\quad\quad\quad \land display \in RANGE_TEMPS$
$AllTemperatures \triangleq \forall\, t \in RANGE_TEMPS : \Diamond(sensor = t)$
$DisplayAlwaysReachesSensor \triangleq display \neq sensor \Rightarrow \Diamond(display = sensor)$
$DisplayIncrementsOK \triangleq$
$\quad \Box[$ IF $display' < display$ THEN
$\quad\quad \land display' \geq sensor$
$\quad\quad \land \neg\exists\, i \in (1\,..\,MAX_INCREMENT) :$
$\quad\quad\quad (i > display - display' \land display - i \geq sensor)$
$\quad\quad$ ELSE IF $display' > display$ THEN
$\quad\quad\quad \land display' \leq sensor$
$\quad\quad\quad \land \neg\exists\, i \in (1\,..\,MAX_INCREMENT) :$
$\quad\quad\quad\quad (i > display' - display \land display + i \leq sensor)$
$\quad\quad$ ELSE $display' = display$
$\quad]_{display}$

[55]

thermometer.tla

```
---- MODULE thermometer ----
EXTENDS Integers
VARIABLE display, sensor

RANGE_TEMPS   == (-1..1)
MAX_INCREMENT == 2

Init == /\ sensor  \in RANGE_TEMPS
        /\ display \in RANGE_TEMPS

ReadSensor ==
    /\ display = sensor
    /\ \E newTemperature \in RANGE_TEMPS: sensor' = newTemperature
    /\ display' = display

UpdateDisplay ==
    /\ IF display > sensor THEN
            IF (display - sensor) > MAX_INCREMENT THEN
                display' = display - MAX_INCREMENT
            ELSE
                display' = display - ((display - sensor) % MAX_INCREMENT)
        ELSE IF display < sensor THEN
            IF (sensor - display) > MAX_INCREMENT THEN
                display' = display + MAX_INCREMENT
            ELSE
                display' = display + ((sensor - display) % MAX_INCREMENT)
        ELSE FALSE
    /\ UNCHANGED sensor

Next == ReadSensor \/ UpdateDisplay
Spec == Init /\ [][Next]_<<display, sensor>>
             /\ WF_sensor(ReadSensor)
             /\ WF_display(UpdateDisplay)
----
TypeOK == /\ sensor \in RANGE_TEMPS
          /\ display \in RANGE_TEMPS

AllTemperatures == \A t \in RANGE_TEMPS: <>(sensor = t)

DisplayAlwaysReachesSensor == display /= sensor => <> (display = sensor)

DisplayIncrementsOK ==
    [][  IF  display' <  display THEN
            /\ display' >= sensor
            /\ ~ \E i \in (1..MAX_INCREMENT):
                 (i > display  - display' /\ display - i >= sensor)
        ELSE IF display' >  display THEN
            /\ display' <= sensor
            /\ ~ \E i \in
                 (1..MAX_INCREMENT): (i > display' - display  /\ display + i <= sensor)
        ELSE  display' = display
    ]_display
====
```

- En la línea 2 en esta ocasión importamos el módulo `Integers` en lugar del módulo `Naturals`, ya que las temperaturas que se pueden leer del sensor son números enteros positivos y negativos.
- En la línea 3 se declaran las 2 variables que se usan para modelar la abstracción del termómetro: `display` para reperesentar la temperatura que se está mostrando en la pantalla, y `sensor` para representar el valor de temperatura que devuelve el sensor.
- En la línea 4 se define `RANGE_TEMPS` como el conjunto de valores admisibles para la temperatura. Y en la línea 5 se define `MAX_INCREMENT` para representar el valor absoluto del valor máximo en el que se puede modificar la temperatura que muestra la pantalla.[56]

  Para que los diagramas de estados no sean demasiado complejos, hemos preferido utilizar el rango de temperaturas

  `RANGE_TEMPS == (-1..1)` en lugar del valor fijado por los requisitos de `(-20..60)`, y `MAX_INCREMENT = 2` en lugar del valor 5 fijado por los requisitos.

  Se pueden modificar estos valores para comprobar si la especificación se satisface también para los valores fijados por los requisitos.

- La fórmula `Init` en la línea 6 define el valor inicial de las 2 variables. Alinear en la misma columna los operadores lógicos tal como se hace en TLA+ mejora la legibilidad sin utilizar paréntesis. Podríamos haber escrito así la fórmula `Init`:

  ```
  Init == sensor  \in RANGE_TEMPS /\ display \in RANGE_TEMPS
  ```

  Veamos un ejemplo que ilustra cómo mejora la legibilidad:

  ```
  A \/ (B /\ (C \/ D))
  ```

  La alternativa aparece a la izquierda:

  ```
  \/ A                          \/ A
  \/ /\ B                       \/ /\ B
     /\ \/ C                       /\ \/ C
        \/ D                    \/ D
  ```

  Pero hay que tener cuidado con esta alternativa a los paréntesis, pues ahora el sangrado (los espacios en blanco) sí importan: la expresión que aparece a la derecha no tiene espacios en blanco antes de `D` en la última línea; Y por tanto no es equivalente a la expresión de la izquierda. La expresión de la derecha es equivalente a esta otra:

  ```
  A \/ (B /\ C) \/ D
  ```

[56] Tanto `RANGE_TEMPS` como `MAX_INCREMENT` se pueden declarar como constantes en TLA+ cuyos valores son asignados en TLC. Lo veremos más adelante.

El símbolo `\/` delante de `A` es opcional, pero hace más regular el conjunto.

- Las fórmulas `ReadSensor` y `UpdateDisplay` definidas en las líneas 8 y 12 respectivamente son ambas acciones, es decir, predicados sobre pasos o transiciones (aparecen variables primadas y no primadas). En esta especificación se ha definido la fórmula `Next` en la línea 25 como la disyunción de estas dos acciones[57]. De todos los posibles comportamientos del universo, aquellos en los que el termómetro está funcionando según esta especificación son el subconjunto de comportamientos en los que la fórmula `Next` es `TRUE`; es decir, en los que la expresión `ReadSensor \/ UpdateDisplay` es `TRUE`.

  Cuando TLC verifica la especificación, en cada estado se evalúan los predicados de todas las acciones posibles, en este caso `ReadSensor` y `UpdateDisplay`. Si los dos son `TRUE`, a partir de ese estado se generan dos posibles transiciones: una en la que los valores de las variables en el siguiente estado son los especificados en las variables primadas de `ReadSensor`, y otra transición en la que los valores de las variables en el siguiente estado son los especificados en las variables primadas de `UpdateDisplay`.

  Mostramos en la figura 1 un fragmento del diagrama de estados que debería generar la especificación del termómetro si fuese correcta. Todos los estados aparecen en gris pues todos son posibles estados iniciales. Se pueden apreciar las 2 acciones, `ReadSensor` en verde y `UpdateDisplay` en azul[58]:

[57] Esta es la forma habitual de escribir especificaciones en TLA+.

[58] Compáralo con el diagrama del reloj: allí sólo había una acción posible para transitar entre estados: `Next`.

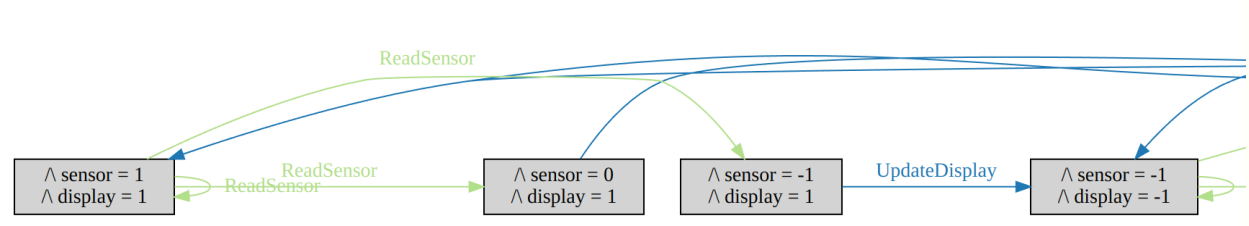


Figura 1: Diagrama de estados de un termómetro correcto.

  En el estado en el que `sensor = -1` y `display = 1`, el tercero por la izquierda, se ve cómo sólo hay una transición en azul generada por la acción `UpdateDisplay`, que hace que `display` pase de valer 1 a valer -1, usando el máximo incremento posible ya que `MAX_INCREMENT` es 2.

- En la línea 8 la acción `ReadSensor` modela la lectura del sensor: es una acción que define cómo cambia la variable `sensor`. Sólo puede cambiar cuando no se está cambiando la pantalla, para lo cuál la línea 9 especifica una condición: `display = sensor`. La llamamos condición porque al estár en conjunción junto a las expresiones que cambian el estado de la variable `sensor`, la acción `ReadSensor` no será `TRUE` si `display = sensor` es `FALSE`.

  En caso de que la especificación fuese correcta, desde cada estado sólo se deberían producir transiciones generadas por la acción `ReadSensor` o por la acción `UpdateDisplay`, pero no por ambas[59].

[59] La figura anterior muestra un diagrama de estados de una especificación correcta por lo que puede apreciarse el comportamiento descrito.

En la línea 10 se modela la lectura del sensor: con el cuantificador lógico *existe* `\E` (∃) se introduce indeterminismo en la especificación: `newTemperature` puede adoptar un valor cualquiera del rango de temperaturas, y ese valor pasa a ser el nuevo valor del sensor en el siguiente estado: `sensor' = newTemperature`. Cuando el verificador de modelos TLC procesa esta línea, genera nuevas transiciones en el grafo de estados para el siguiente estado, una para cada valor del rango `RANGE_TEMPS`.

Veámoslo en el grafo de estados. En la figura 1 se aprecia cómo desde el estado de la izquierda en el que `sensor = 1` y `display = 1` hay 3 transiciones o pasos posibles que se muestran en verde para la acción `ReadSensor`: uno para el estado destino con el valor de `sensor = -1`, otro para `sensor = -0` y otro para `sensor = 1`.

- Cada vez que se produce una transición o paso entre dos estados hay que expresar el valor que tienen todas las variables de la especificación en el nuevo estado. La acción `ReadSensor` sólo especifica el valor `sensor'` en la línea 10. Por eso en la línea 11 se especifica que el valor de la otra variable `display` no cambia en la transición de la acción `ReadSensor`.

  La línea 11 se puede sustituir por esta línea que tiene el mismo efecto:

  `UNCHANGED display`.

- En la línea 12 la acción `UpdateDisplay` modela la actualización de la pantalla. Según los requisitos, sólo se puede realizar esta acción cuando el último valor leído por el sensor (valor de la variable `sensor`) y el valor mostrado por la pantalla (valor de la variable `display`) no son iguales. Como puedes comprobar la acción `UpdateDisplay` se ha escrito utilizando `IF THEN ELSE`.

  La acción `UpdateDisplay` especifica cómo se actualiza el valor de la pantalla (variable `display`), que debe modificarse en los incrementos más grandes posibles, siempre que no sean mayores que `MAX_INCREMENT` según los requisitos. Estudia el código para ver qué algoritmo se ha utilizado para satisfacer este requisito.

  Como la acción `UpdateDisplay` sólo especifica cómo cambia el valor de la variable `display` (líneas 15, 17, 20, 22), en la línea 24 se expresa que el valor que tendrá en el nuevo estado la otra variable `sensor` no cambia en esta acción.

- Fíjate en la definición de la especificación `Spec` en la línea 26. Las dos variables para las que se permiten estados tartamudeantes para la acción `Next` aparecen entre ángulos dobles: «`display, sensor`». En una estructura de datos de TLA+ llamada secuencia o tupla que estudiaremos más adelante. Almacena una secuencia de valores, en este caso las dos variables y en este caso sirve para

indicar que cualquiera de las 2 variables pueden no cambiar de valor en estados tartamudeantes de la especificación.

- A partir de la línea 29 se definen propiedades que debe satisfacer la especificación. La línea 33 es una fórmula temporal. En ella aparece el cuantificador lógico *para todo* `\A` (∀).

  La siguiente expresión

  ```
  \A t \in RANGE_TEMPS: <>(sensor = t)
  ```

  $\forall t \in RANGE_TEMPS : \diamond(sensor = t)$

  es cierta si para todo (`\A`) valor `t` del conjunto `RANGE_TEMPS` es cierta la expresión que aparece a la derecha del símbolo `:`. Por tanto la expresión es cierta si para todo valor `t` del conjunto `RANGE_TEMPS`, eventualmente (`<>`) se acaba alcanzando un estado en el que la variable `sensor` tiene ese valor `t`. Dicho de otro modo: en todo comportamiento de la especificación se experimentan todos los valores posibles de temperatura.

- La fórmula `DisplayAlwaysReachesSensor` de la línea 34 es otra fórmula temporal. Aparece el operador lógico `=>`, que es la implicación. La fórmula expresa que si los valores de `sensor` y `display` no son iguales, entonces (`=>`), eventualmente (`<>`), lo acabarán siendo en algún momento.

- En las línea 38 y 42 aparece el operador `~`. Es la negación lógica. En ambos casos podemos leer la expresión como: no exite ningún elemento i del conjunto `(1..MAX_INCREMENT)` que haga cierta la expresión que aparece detrás del símbolo `:`.

¿Satisface la especificación las propiedades? En los ejercicios tendrás la oportunidad de descubrirlo.

Ejercicio 1: Verificación de propiedades del termómetro

Estudia con detenimiento la especificación del termómetro. No hemos explicado en detalle el algoritmo que se utiliza en la acción `UpdateDisplay` para satisfacer los requisitos. Repasa los requisitos del termómetro y asegúrate de que entiendes qué hace esta acción, y el resto de la especificación.

Tampoco hemos explicado las propiedades: parte de lo que te pedimos en el ejercicio es que estudies precisamente las propiedades, y que las utilices para descubrir los fallos que contiene la especificación.

Para entregar este ejercicio tendrás que enviarnos los ficheros de especificaciones que se mencionan en los apartados, y una memoria en la que des repuesta a todas las preguntas que se hacen. Se cuidadoso al escribir, especificando claramente los apartados a los que das respuesta y siendo claro en tus explicaciones.

Preguntas:

1. Reescribe la acción `UpdateDisplay` sin utilizar ningún `IF THEN ELSE`.
2. Reescribe la acción `ReadSensor` utilizando `IF THEN ELSE`.

   Ten en cuenta que el significado de la fórmula `ReadSensor` es un predicado sobre pasos o acciones, por lo que siempre debe evaluarse a un booleano. Por tanto tienes que especificar siempre una rama `ELSE` cuando uses `IF`. Fíjate en el uso de `IF THEN ELSE` que se hace en otros lugares de la especificación: siempre hay rama `ELSE`.
3. Añade los comentarios que consideres oportunos en el fichero de la especificación para que tanto las acciones como las propiedades queden bien documentadas.

```
(* Esto es un comentario *)

Init == v = 0    \* Comentario hasta fin de línea

(*
   Comentario en
   varias
   líneas
*)

(* Mejor comentarios en varias líneas *)
(* de esta otra forma, que mejora la  *)
(* legibilidad                        *)
```

4. Usando TLC, genera el diagrama de estados de `thermometer.tla`, pero teniendo comentadas todas las propiedades, pues de lo contrario el diagrama sólo mostrará la traza de un comportamiento

que falle en alguna de las propiedades. Estudia el diagrama de estados con atención. ¿Encuentras discrepancias con los requisitos del termómetro? ¿Encuentras diferencias con los fragmentos del diagrama de estados que hemos utilizado para explicar la especificación del termómetro?

Seguro que sí, pues hemos introducido fallos adrede. De hecho, los fragmentos de diagramas de estados que hemos usado para explicar la especificación del termómetro no son los que observarás tú cuando generes el diagrama. Presta atención al diagrama.

Ayúdate del diagrama de estados, y de la verificación de las propiedades que tienes que hacer en el siguiente apartado, para descubrir los fallos de la especificación.

5. Comenta todas las propiedades: `TypeOK`, `AllTemperatures`, `DisplayAlwaysReachesSensor`, `DisplayIncrementsOK`.

   Ahora, una a una, ve descomentándolas y ejecutando TLC. Explica para cada una de ellas si es de tipo *safety*, *liveness*, o es una propiedad de acciones.

   a) Para cada una de las propiedades, explica en lenguaje natural (castellano) qué requisitos intenta verificar, mencionando los requisitos informales del termómetro en tu explicación.

   b) Comprueba si la especificación satisface o no cada una de las propiedades.

   Si no la satisface, explica por qué crees que no la satisface, y considera si una modificación de la especificación podría hacer que se satisficiera. Si consideras que el problema está en la propiedad, explica la razón por la que consideras que dicha propiedad no tiene sentido o es incorrecta.

   A la especificación corregida que generes llámala `thermometer_correct.tla`.

   c) Cuando termines con cada propiedad, la vuelves a comentar, y pruebas la siguiente estando el resto comentadas. De este modo te será más fácil interpretar la salida de TLC.

6. No hay ninguna propiedad que compruebe el requisito de que no se puede leer el sensor mientras la pantalla está siendo actualizada. Añade una propiedad de acciones denominada `ReadSensorOK` para comprobar si la especificación satisface este requisito.

7. Se ha decidido rediseñar el termómetro, haciendo desaparecer justamente este requisito:

   - Mientras que se está actualizando la pantalla a un nuevo valor no se lee un nuevo valor del sensor de temperatura.

   Crea una nueva especificación `thermometer_2.tla` en la que sí se pueda leer un nuevo valor del sensor aunque se esté actuali-

zando la pantalla. Comprueba que esta nueva especificación no satisface la propiedad `ReadSensorOK` del apartado anterior.

Compara el diagrama de estados de `thermometer_2.tla` con el de `thermometer.tla`. Documenta cómo en el nuevo sí se realizan lecturas del sensor cuando la temperatura mostrada en la pantalla no es igual a la última devuelta por el sensor.

8. Para cada propiedad, prueba si se satisface o no cambiando en `Spec` el criterio de *fairness* para cada una de las acciones:

```
/\ WF_sensor(ReadSensor)
/\ WF_display(UpdateDisplay)
```

Prueba quitando ambos términos, dejando ambos, y dejando uno u otro. Explica el comportamiento observado ejecutando TLC en cada caso, y si tiene sentido o no para cada propiedad usar uno u otro criterio de *fairness*.

Ejercicio 2: Especificación de reloj con minutos y AM/PM

Crea una nueva especificación `clock_minutes` partiendo de una de las especificaciones del reloj que marca las horas, añadiéndole un minutero y un indicador de AM/PM.

Requisitos:

1. el reloj deberá marcar las horas, números naturales de 1 a 12
2. el reloj deberá marcar los minutos, números enteros de 0 a 59
3. el reloj deberá indicar si la hora marcada es `AM` (*ante meridiem*) o `PM` (*post meridiem*). Después de las `11:59 PM` el reloj marcará la `12:00 AM`, y seguirá indicando `AM` hasta llegar a las `11:59 AM`, tras lo cuál indicará la `12:00 PM`, tras lo cuál seguirá indicando `PM` hasta llegar a las `11:59 PM`.
4. inicialmente el reloj puede marcar una hora cualquiera, de 1 a 12, un minuto cualquiera, de 0 a 59, y puede indicar `AM` o `PM`.
5. el reloj va marcando los minutos. Al llegar al minuto 59 se incrementa en una unidad la hora y se vuelve al minuto 0; salvo al llegar a las 12:59, que pasa a marcar las 1:00.
6. se van incrementando los minutos en una unidad, y las horas se incrementan en una unidad cuando el minuto vale 59. Nunca se decrementa el valor de las horas ni el los minutos, salvo cuando pasa de marcar a las 12:59: en ese estado la hora pasa al valor 1 y el minuto al valor 0.

Preguntas:

1. Escribe la especificación `minute_clock.tla`
2. Escribe las propiedades de tipo `safety`, *liveness* y de acciones que consideres necesario.
3. Documenta con comentarios la especificación (acciones y propiedades).

Ejercicio 3: Especificación de reloj con minutos y AM/PM más termómetro

- Crea una nueva especificación `thermometer_clock_minutes.tla` para un dispositivo que satisfaga los requisitos del termómetro y del reloj con minutos e indicación de AM/PM.
- Documenta con comentarios la especificación (acciones y propiedades).

Especificación de un reloj que sólo marca las horas en PlusCal

PlusCal es un lenguaje algorítmico que se parece a los lenguajes imperativos de programación como Python, Java, C o Pascal. Hay dos variantes, una con sintaxis parecida a la de Pascal y otra parecida a la de C. Utilizaremos esta última.

Necesitas seguir sabiendo TLA+ para usar PlusCal pues las expresiones en PlusCal se escriben como expresiones TLA+, al igual que las propiedades. PlusCal añade sentencias al lenguaje. También macros, procesos y procedimientos, elementos que tratan de facilitar el uso de TLA+ a los programadores, acostumbrados a usar lenguajes imperativos que incluyen estas construcciones.

Así escribiríamos en PlusCal el reloj que sólo marca las horas:

clock_11.tla

```
1  ---- MODULE clock_11 ----
2  EXTENDS Naturals
3
4  (* --algorithm  HourClock {
5  variables hr \in (1..12);
6  {
7      Clock:
8      while (TRUE) {
9          if (hr = 12) hr := 1;
10         else hr := hr + 1;
11     }
12 }
13 }
14 *)
15 ----
16 TypeOK == hr \in (1..12)
17 AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
18 AlwaysUp == [][ \/ /\ hr /= 12
19                   /\ hr' = hr + 1
20                \/ /\ hr  = 12
21                   /\ hr' = 1
22             ]_hr
23 ====
```

- El fichero sigue siendo el de un módulo TLA+. Si lo comparas con la especificación en TLA+ `clock_10.tla` verás que las variables de TLA+, la fórmula de la acción `Next` y la fórmula de la especificación `Spec` se han sustituido por el algoritmo en PlusCal. El resto es igual, incluyendo las propiedades al final del módulo.
- El algoritmo en PlusCal se escribe dentro de un comentario que en la figura comienza en la línea 3 y termina en la línea 13. El

algoritmo comienza con **`--algorithm`** seguido del nombre del algoritmo. En este caso hemos elegido el nombre **`HourClock`**, que no tiene por qué coincidir con el del módulo TLA+ en el que hemos añadido el algoritmo PlusCal, que en este caso es `clock_11`[60].

[60] Por eso el nombre del fichero será `clock_11.tla`

- En la línea 3 comienza el algoritmo con llave abierta al final de la línea (puede ir en línea aparte) y termina con la correspondiente llave cerrada en la línea 12.
- Con la palabra reservada `variables` declaramos, y opcionalmente inicializamos, las variables globales, en nuestro caso `hr`. La declaración de variables es una expresión TLA+, por lo que usamos el símbolo `=` y no la sentencia de asignación `:=` para declarar el valor inicial que como hemos dicho es opcional.
- Tras la declaración de variables, en un nuevo bloque que comienza con llave abierta en la línea 5 y termina con la primera llave cerrada en línea 11, escribimos las sentencias del cuerpo del algoritmo[61].
- La primera sentencia del cuerpo de un algoritmo tiene que tener una **etiqueta**: una cadena seguida por el símbolo `:`. En la línea 6 escribimos la etiqueta `Clock:`. Las sentencias que se ejecutan entre dos etiquetas se ejecutarán de manera atómica: el traductor convertirá en una acción ese grupo de sentencias, y al pertenecer a una misma acción TLA+, se ejecutarán en un sólo paso o transición de TLA+. Por ello decimos que las sentencias se ejecutarán de manera atómica. En este caso, cada vuelta del bucle **`while`** que comienza en la línea 7 justo tras la etiqueta `Clock:`, se ejecutará de manera atómica, en una única acción TLA+, como un único paso o transición.
- Recuerda que ya hemos escrito especificaciones del reloj en TLA+ sin bucles `while`: no hay `while` en TLA+. En TLA+ no hay sentencias, ni `while` ni `if`, ni sentencia de asignación `:=`. En PlusCal sí las hay: esa es precisamente la razón de ser de PlusCal.
- Vemos en la línea 8 que también hay una sentencia **`if`** en PlusCal. Este `if` no es la expresión `IF THEN ELSE` de TLA+. En PlusCal las expresiones se escriben en TLA+, por lo que seguiremos utilizando `IF THEN ELSE` en las especificaciones TLA+/PlusCal, además de usar la sentencia `if`. La sentencia `if` se comporta como en los lenguajes de programación: tiene una expresión TLA+ entre paréntesis para la condición, una rama `then` para las sentencias (sí, sentencias y no expresiones), y una rama `else` opcional[62].
- En las ramas `then` (línea 8) y `else` (línea 9) usamos la sentencia PlusCal de asignación **`:=`**. En TLA+ no hay sentencia de asignación: el símbolo `=` es un comparador. Por ello, en lugar de utilizar

[61] El cuerpo del algoritmo es el equivalente a la función `main()` en C. Escribiremos más adelante algoritmos en los que no hay cuerpo del algoritmo, y en su lugar hay procesos.

[62] Recuerda que la rama `ELSE` NO es opcional cuando en TLA+ o en PlusCal usamos `IF THEN ELSE` para construir expresiones.

variables primadas y no primadas como hacemos en las acciones de TLA+ para describir los pasos o transiciones, en PlusCal utilizamos sentencias para modificar el valor de las variables que definen el estado de los comportamientos o ejecuciones. El objetivo, recuerda, es que sea más sencillo para programadores habituados a escribir código algorítmicamente.

- El resto de la especificación, fuera del algoritmo escrito en PlusCal en un comentario, es TLA+. Vemos las mismas propiedades que habíamos definido para el reloj a partir de la línea 14. La única diferencia es que hemos reescrito `AlwaysUp` para que no utilice paréntesis.

Hay que traducir a TLA+ el algoritmo PlusCal para poder verificarlo con TLC. Para ello elegimos la opción `TLA+: Parse module` del menú que desplegamos con `Ctrl+Shift+P`.

Si hay errores recuerda que el nombre del fichero en la pestaña de VS Code aparece en rojo, y con el menú `View > Problems` de VS Code puedes abrir un panel en la que se describen los problemas.

Si por el contrario no hay errores, verás que se modifica el fichero: el traductor de PlusCal a TLA+ ha insertado tras el comentario dentro del que está el algoritmo, y antes de las propiedades, la traducción a TLA+ del algoritmo PlusCal.

Mostramos el algoritmo traducido a TLA+ en la siguiente página:

clock_11.tla

```
---- MODULE clock_11 ----
EXTENDS Naturals

(* --algorithm  HourClock {
variables hr \in (1..12);
{
    Clock:
    while (TRUE) {
        if (hr = 12) hr := 1;
        else hr := hr + 1;
    }
}
}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "366b7456" /\ chksum(tla) = "8a95c7db")
VARIABLE hr

vars == << hr >>

Init == (* Global variables *)
        /\ hr \in (1..12)

Next == IF hr = 12
           THEN /\ hr' = 1
           ELSE /\ hr' = hr + 1

Spec == Init /\ [][Next]_vars

\* END TRANSLATION
----
TypeOK == hr \in (1..12)
AlwaysCycle == []<> (hr = 12 /\ (hr = 12 ~> hr = 1))
AlwaysUp == [][ \/ /\ hr /= 12
                   /\ hr' = hr + 1
                \/ /\ hr  = 12
                   /\ hr' = 1
             ]_hr
====
```

- La traducción a TLA+ del algoritmo comienza en la línea 14 y termina en la 23[63].
- El algoritmo PlusCal sigue estando ahí, como un comentario. Desde el punto de vista de TLA+, esta ahora es una especificación TLA+ como otra cualquiera: el algoritmo es un comentario.
- El traductor ha generado los siguientes elementos TLA+ a partir del algoritmo escrito en PlusCal `HourClock`:
  - Las variable TLA+ `hr` a partir de las variables del algoritmo (línea 15).

    La variable `hr` aparece entre ángulos dobles: es una estructura de datos denominada secuencia o tupla en TLA+ que aún no hemos visto, similar a las tuplas de Python.
  - Una fórmula `Init` que inicializa `hr` con el valor con el que se inicializó en la declaración del algoritmo (línea 17).
  - Una fórmula `Next` obtenida a partir del cuerpo del algoritmo (línea 19).
  - Una fórmula con la especificación `Spec` (línea 22).

  Si prestas atención verás que es prácticamente idéntica a la especificación TLA+ `clock_7.tla`. Compruébalo. Sólo se diferencia en que la fórmula `Spec` generada automáticamente a partir del algoritmo no exige *Weak Fairness*. No lo modifiques.

[63] NO debes modificar nunca el código generado. Si deseas cambiar algo, modifica el algoritmo en PlusCal y regenera la traducción.

Ahora ya podemos verificar con TLC[64] esta especificación generada por PlusCal. Si no creas el fichero `clock_11.cfg` se genera uno automáticamente al ejecutar TLC, pero en él no estarán las invariantes ni las propiedades. Las añadimos:

[64] **No olvides traducir siempre a TLA+** cuando estás usando PlusCal antes de ejecutar TLC. Si no ejecutarás la versión anterior.

clock_11.cfg

```
SPECIFICATION
  Spec

INVARIANTS
  TypeOK

PROPERTIES
  AlwaysCycle
  AlwaysUp
```

Si ejecutamos TLC vemos que, al igual que nos pasó antes de añadir *Weak Fairness*, se viola una propiedad. Para saber cuál, comentamos ambas dejando sin comentar la invariante `TypeOK` y vemos que esta invariante si se satisface.

Descomentamos `AlwaysUp` y vemos que se satisface. Pero si descomentamos `AlwaysCycle` vemos que no se satisface, al igual que nos ocurrió antes con la especificación escrita en TLA+.

No tenemos que modificar el código TLA+: En un algoritmo se exige *Weak Fairness* añadiendo antes de `algorithm` la palabra `fair`:

```
(* --fair algorithm HourClock {
```

Traduce a TLA+ y comprueba cómo se ha añadido a la especificación la expresión que exige *Weak Fairness*

```
Spec == /\ Init /\ [][Next]_vars
        /\ WF_vars(Next)
```

Ahora sí, TLC verfica que se cumple la propiedad `AlwaysCycle`. Si no recuerdas el porqué, vuelve a leer la explicación que hicimos más arriba sobre el efecto de exigir *Weak Fairness* a la especificación.

Especificación de un termómetro en PlusCal

En esta sección aprenderás a escribir en PlusCal la especificación del termómetro.

Comenzamos con una primera versión que mostramos en la siguiente página. No aparecen las propiedades para que nos quepa en la página, pero en el fichero que acompaña a estos apuntes sí están.

Repasa los requisitos del termómetro[65] para entender mejor esta explicación del código de la siguiente página:

- Mantenemos en las líneas 3 y 4 las definiciones del conjunto de temperaturas y del incremento máximo.
- Al igual que hicimos en la especificación del reloj de horas con TLA+, también exigimos *Weak Fainess* al algoritmo en PlusCal `thermometer` escribiendo `(* --fair algorithm` en la línea 5.
- En este caso tenemos dos variables globales que declaramos en la sección correspondiente, separadas por comas, con sus respectivas inicializaciones en las líneas 6 y 7.
- En la línea 8 ponemos la primera etiqueta, obligatoria al comienzo del algoritmo.
- El bucle `while` en la línea 9 contiene una única sentencia en su cuerpo. **`either`** introduce indeterminismo. En cada vuelta del bucle, indistintamente: o bien se ejecuta el código en la primera rama (línea 11), o bien se ejecuta el código en la rama **`or`** (línea 18).

  El efecto del indeterminismo introducido por `either` es que si TLC ejecuta la especificación resultado de traducir a TLA+ este algoritmo, generará en el grafo de estados tanto transiciones para una rama de either como para la otra.

  En algunos comportamientos aparecerá una transición a un siguiente estado en el que los valores de las variables serán los generados por las sentencias de una de las ramas (recuerda que un estado es una asignación de valores a variables). En otros comportamientos aparecerá una transición a un estado cuyas variables tengan los valores generados por las sentencias de la otra rama. Que TLC haga una u otra cosa dependerá de si hay sentencias **`when`** con una condición al principio de cada rama, que habiliten o no la ejecución del resto de la sentencias de esa rama:

  - La primera sentencia de la primera rama de `either` en la línea 11 es `when display = sensor;` `when` evalúa la expresión y si es `TRUE` se siguen ejecutando las siguientes sentencias. Hasta que no sea `TRUE` esa expresión no se podrán ejecutar.

[65] Requisitos del termómetro:

1. El sensor de temperatura devuelve valores enteros comprendidos entre -20 y 60 grados centígrados que deberán ser mostrados en la pantalla del termómetro.
2. Ante un cambio brusco de temperatura el sensor devolverá un valor demasiado alejado del que muestra la pantalla. Por ello, se mostrará el cambio mediante varios valores intermedios, para que el usuario perciba que está cambiando bastante la temperatura en lugar de ver un único cambio del valor actual al valor medido por el sensor.
   Por ello, la pantalla no mostrará directamente el nuevo valor de temperatura devuelto por el sensor, sino que se mostrarán incrementalmente los valores de temperaturas intermedias desde el valor actual que muestra la pantalla, hasta llegar al valor leído del sensor, en incrementos lo más grandes posibles; estos incrementos estarán acotados por un tamaño de incremento máximo de 5 grados en valor absoluto.
3. Hasta que no se haya mostrado en la pantalla el valor del último valor leído del sensor de temperatura no se leerá un nuevo valor del sensor de temperatura.

thermometer_pluscal_1.tla

```
---- MODULE thermometer_pluscal_1 ----
EXTENDS Integers

RANGE_TEMPS == (-20..60)
MAX_INCREMENT == 5

(* --fair algorithm thermometer {
    variables display \in RANGE_TEMPS,
              sensor  \in RANGE_TEMPS;
{Thermometer:
    while (TRUE) {
        either {
            when display = sensor;

            ReadSensor:
            with (newTemperature \in RANGE_TEMPS) {
                sensor := newTemperature;
            }
        }
        or{
            when display /= sensor;

            UpdateDisplay:
            if (display > sensor) {
                if (display - sensor >= MAX_INCREMENT)
                    display := display - MAX_INCREMENT
                else
                    display := display - (display - sensor)
            }
            else if (display < sensor) {
                if (sensor - display >= MAX_INCREMENT)
                    display := display + MAX_INCREMENT
                else
                    display := display + (sensor - display)
            }
        }
    } \* while
}
}
*)
----
TypeOK == /\ sensor \in RANGE_TEMPS
          /\ display \in RANGE_TEMPS
...
====
```

- La rama `or` de `either` en la línea 18 tiene también una sentencia `when`. En ambos casos comprenderás por qué se usa `when`, si recuerdas los requisitos del termómetro[66].

  En nuestra especificación vemos que en ambas ramas hay sentencias `when` al principio de cada rama de `either`, y que ambas condiciones son excluyentes: por tanto sólo se generará en los estados en lo que `display = sensor` una transición al estado generado por las sentencias de la primera rama; y en los estados en los que `display /= sensor` se generará una transición al estado generado por las sentencias de la segunda rama.

  Si no tuvieran condiciones `when`, se generarían transiciones tanto a uno como a otro estado.

[66] Y si no los recuerdas, ve a repasarlos, que los tienes justo en la nota anterior a esta que estás leyendo. ;-)

- En cuanto a lo que se hace en la rama de `either` a partir de la etiqueta `ReadSensor:` vemos que hay una sentencia **`with`** en la línea 13. Esta sentencia introduce otro tipo de indeterminismo: se genera un nuevo estado para cada uno de los valores de `newTemperature`. Si repasas la especificación del termómetro en TLA+ recordarás que para hacer esto utilizábamos el cuantificador existencial en la acción `ReadSensor`. `with` se traduce a TLA+ de hecho como un operador existencial como veremos en un momento cuando estudiemos la traducción. En el cuerpo de `either` entre llaves se asigna el valor de `newTemperature` a `sensor`.
- En la segunda rama de `either`, tras la sentencia `when` está la etiqueta `UpdateDisplay`: las sentencias que aparecen implementan el algoritmo para actualizar la pantalla del termómetro como habrás detectado.
- Recordamos por último que el código que se ejecuta entre 2 etiquetas se ejecuta de manera atómica. Por tanto todas las sentencias desde `ReadSensor:` hasta volver a `Thermometer:` en la siguiente vuelta del bucle se ejecutan de manera atómica. También las sentencias desde la etiqueta `UpdateDisplay` hasta `Thermometer:`.

Cuando traducimos el algoritmo a TLA+ se obtiene el código que aparece en la siguiente página, del que reproducimos sólo la traducción para que nos quepa en una página.

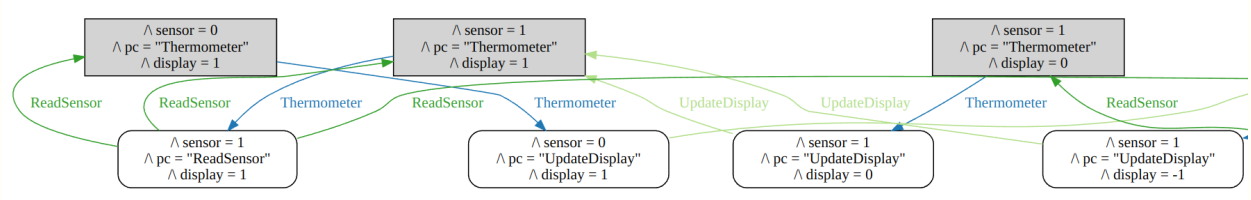


Figura 2: Grafo de estados de `thermometer_pluscal.tla`

Seguro que reconoces buena parte del código. Compara con la especificación en TLA+ que hicimos del termómetro y reconocerás casi todo.

- Además de las variables `sensor` y `display` en esta ocasión la traducción de PlusCal a TLA+ ha generado la variable **`pc`**. Esta variable se usa como un ***Program Counter*** o contador de programa.

  En `Init` vemos cómo la variable `pc` tiene valor `"Thermometer"`[67]. Por tanto todos los estados iniciales del grafo de estados tienen este valor en la variable `pc`, como puedes ver en la figura 2 (recuerda que los estados iniciales aparecen en gris).

- La acción `Thermometer` (línea 15) la ha generado automáticamente el traductor. Comienza con una condición: `pc = "Thermometer"`. Como en el estado especificado en la fórmula `Init` la variable `pc` tiene el valor `"Thermometer"`, esta acción `Thermometer` será la que se ejecutará en la primera transición desde el estado inicial. Compruébalo en la figura 2: de todos los estados iniciales en gris sale una acción en azul `Thermometer`, que a veces lleva a un estado en el que `pc = ReadSensor` y a veces a un estado en el que `pc = UpdateDisplay`, dependiendo de si `display = sensor` o no en el estado inicial de partida.

  Esta acción implementa una especie de *dispatcher* que va dando paso a `ReadSensor` o a `UpdateDisplay` siempre que se cumpla la condición que se especificó en PlusCal con `when` en cada una de las ramas de `either`, que aquí aparecen en las líneas 16 y 18. En caso de que se cumplan esas condiciones, se acaba modificando el valor de la variable `pc` en el siguiente estado, para dar paso a una u otra acción en las líneas 17 y 19.

- Al comienzo de las dos acciones del termómetro en las líneas 21 y 27 hay una condición que permite o impide que se activen las acciones en función del valor que el *dispatcher* `Thermometer` haya fijado para la variable `pc`.

- Fíjate también en cómo al final de cada acción se asigna la etiqueta `Thermometer:` del *dispatcher* a la variable `pc'` para que la

[67] Fíjate en cómo este nombre es el de la primera etiqueta que hemos puesto en el algoritmo.

thermometer_pluscal_1.tla

```
---- MODULE thermometer_pluscal_1 ----
EXTENDS Integers

RANGE_TEMPS == (-20..60)
MAX_INCREMENT == 5

(* --fair algorithm thermometer {

  ...

*)
\* BEGIN TRANSLATION
VARIABLES display, sensor, pc

vars == << display, sensor, pc >>
Init == (* Global variables *)
        /\ display \in RANGE_TEMPS
        /\ sensor \in RANGE_TEMPS
        /\ pc = "Thermometer"
Thermometer == /\ pc = "Thermometer"
               /\ \/ /\ display = sensor
                     /\ pc' = "ReadSensor"
                  \/ /\ display /= sensor
                     /\ pc' = "UpdateDisplay"
               /\ UNCHANGED << display, sensor >>
ReadSensor == /\ pc = "ReadSensor"
              /\ \E newTemperature \in RANGE_TEMPS:
                   sensor' = newTemperature
              /\ pc' = "Thermometer"
              /\ UNCHANGED display
UpdateDisplay ==
     /\ pc = "UpdateDisplay"
     /\ IF display > sensor
        THEN /\ IF display - sensor >= MAX_INCREMENT
                   THEN /\ display' = display - MAX_INCREMENT
                   ELSE /\ display' = display - (display - sensor)
        ELSE /\ IF display < sensor
                THEN /\ IF sensor - display >= MAX_INCREMENT
                       THEN /\ display' = display + MAX_INCREMENT
                       ELSE /\ display' = display + (sensor - display)
                ELSE /\ TRUE
                     /\ UNCHANGED display
     /\ pc' = "Thermometer"
     /\ UNCHANGED sensor

Next == Thermometer \/ ReadSensor \/ UpdateDisplay
Spec == /\ Init /\ [][Next]_vars
        /\ WF_vars(Next)
\* END TRANSLATION
----
TypeOK == /\ sensor \in RANGE_TEMPS
          /\ display \in RANGE_TEMPS
    ...
====
```

siguiente acción que se pueda ejecutar vuelva a ser `Thermometer`, el *dispatcher*.

CUANDO VERIFICAMOS CON TLC LA ESPECIFICACIÓN vemos que la especificación satisface todas las propiedades.

Refactorización de la especificación del termómetro en PlusCal

En lugar de escribir el código de la especificación del termómetro en el cuerpo del algoritmo, podemos sustituir este cuerpo por un proceso de PlusCal. Vamos a ver cómo hacerlo para aprender cómo escribir procesos. Además esto nos permitirá extender muy fácilmente la funcionalidad de la especificación del termómetro para que también incluya la del reloj. Te pedimos que lo hagas en un ejercicio más abajo.

En la siguiente página mostramos el código que pasamos a explicar a continuación:

- Como indicábamos, hemos sustituido el cuerpo del algoritmo por un proceso.
- Al no haber cuerpo del algoritmo, hemos quitado la palabra reservada `fair` de la línea 5 y en su lugar hemos añadido la palabra `fair` a la declaración del proceso en la línea 6.

  En la declaración ponemos entre paréntesis una asignación:

  ```
  fair process (Thermometer = "Thermometer")
  ```

  El valor que asignamos puede ser un entero o una tira de caracteres como en este caso, que servirá para diferenciar los procesos. Pero si hay varios procesos, todos ellos tienen que tener un entero, o todos una tira de caracteres como identificador.

  Dentro del código de un proceso se hace referencia al identificador no a través de `Thermometer`, sino con la palabra reservada `self`.
- Si en lugar de la línea 6 escribiésemos esta otra para el identificador, habríamos creado dos procesos distintos, cada uno con un identificador del conjunto, ambos con el mismo código:

  ```
  fair process (Thermometer \in {"Ther_1", "Ther_2"}).
  ```
- Hemos quitado las variables globales y las hemos convertido en variables locales al proceso (líneas 7 y 8).
- En la primera línea del cuerpo del proceso tiene que haber una etiqueta, que en el ejemplo es `Thermometer:`.
- El proceso creará un flujo de actividad distinto, y en este caso será el único pues sólo hay un proceso. Si añadimos más procesos tendremos varios flujos de control distintos.
- El resto del código no requiere explicación.
- Traduce tú ahora a TLA+ el algoritmo, y explora el código generado. Deberías entender prácticamente todo. Lo que no entenderás será el código del *dispatcher*, que ahora, por haber introducido el proceso, incluye construcciones de TLA+ que no hemos estudiado aún. Por ahora no es importante que entiendas esas

thermometer_pluscal_2.tla

```
---- MODULE thermometer_pluscal_2 ----
EXTENDS Integers

RANGE_TEMPS == (-20..60)
MAX_INCREMENT == 5

(* --algorithm thermometer {

fair process (Thermometer = "Thermometer")
    variables display \in RANGE_TEMPS,
              sensor  \in RANGE_TEMPS;
{Thermometer:
    while (TRUE) {
        either {
            when display = sensor;

            ReadSensor:
            with (newTemperature \in RANGE_TEMPS) {
                sensor := newTemperature;
            }
        }
        or{
            when display /= sensor;

            UpdateDisplay:
            if (display > sensor) {
                if (display - sensor >= MAX_INCREMENT)
                    display := display - MAX_INCREMENT
                else
                    display := display - (display - sensor)
            }
            else if (display < sensor) {
                if (sensor - display >= MAX_INCREMENT)
                    display := display + MAX_INCREMENT
                else
                    display := display + (sensor - display)
            }
        }
    } \* while
}
}
*)
----
TypeOK == /\ sensor \in RANGE_TEMPS
          /\ display \in RANGE_TEMPS
...
====
```

líneas que pueden extrañarte, aunque puedes imaginarte qué hacen comparando con la anterior versión del termómetro escrita en PlusCal `thermometer_pluscal_1.tla`.

Ejercicio 4: Termómetro y Reloj que marca las horas en PlusCal

Partiendo de la especificación `thermometer_pluscal_2.tla`, escribe en `thermometer_clock_pluscal.tla` la especificación de un dispositivo que es termómetro y es un reloj que marca las horas.

Escribe el código del reloj en un proceso distinto al del termómetro.

Ejercicio 5: Termómetro y Reloj que marca los minutos y AM/PM en PlusCal

Partiendo de la especificación `thermometer_pluscal_2.tla`, escribe en `thermometer_minute_clock_pluscal.tla` la especificación de un dispositivo que es termómetro y es un reloj que marca las horas y los minutos y la indicación de AM/PM.

Escribe el código del reloj en un proceso distinto al del termómetro.

Ejercicio 6: Condiciones de carrera

Depurar programas en los que hay varios flujos de ejecución[68] verificando su comportamiento correcto usando tests o pruebas es complicado por la gran diversidad de ejecuciones posibles.

Como ejemplo, el fichero `benari.c` que mostramos en la siguiente página contiene el código de un programa con varios hilos de ejecución escrito en C.

El programa principal crea 2 *threads*. Cada uno de ellos ejecuta la función `inc10`, que incrementa 10 veces en una unidad cada vez la variable global `n`. Para ello, en la función `inc10` primero se almacena el valor de `n` en la variable local reg, incrementando después esta variable, y finalmente almacenando de vuelta `reg` en `n`.

Después de cada sentencia de asignación en el cuerpo de `inc10` hay una llamada a `usleep` que provoca la suspensión de la ejecución del *thread* que la ejecuta durante un número aleatorio de microsegundos, durante los cuáles el otro *thread* puede modificar el valor de la variable `n` a la vez que lo está modificando el *thread* suspendido. Esto provoca que haya condiciones de carrera que hacen que el valor final de `n` no sea 20, como cabría esperar si tuviesen lugar correctamente los 10 incrementos de la variable `n` que pretende realizar cada uno de los dos *threads*.

[68] Puede haber varios flujos de ejecución porque hay múltiples hilos ejecutándose concurrentemente, o porque hay paralelismo con varios microprocesadores ejecutando a la vez instrucciones del programa, o porque el sistema es distribuido y hay varios procesos ejecutándose en varios ordenadores distintos a la vez.

benari.c

```c
#include <unistd.h>

char *message1 = "Thread 1";
char *message2 = "Thread 2";
const int MAXDELAY = 1000;

int n = 0;

void *
inc10 ( void *ptr )
{
     for (int count=0; count<10; count++) {
         int reg;

         reg = n;
         usleep(random_between_range(0,MAXDELAY));

         reg += 1;
         usleep(random_between_range(0,MAXDELAY));

         n = reg;
         usleep(random_between_range(0,MAXDELAY));

     }

     char *message;
     message = (char *) ptr;

     printf("%s %d \n", message, n);

     return NULL;
}

int
main()
{
     pthread_t thread1, thread2;
     int  iret1, iret2;

     iret1 = pthread_create( &thread1, NULL, inc10, message1);
     iret2 = pthread_create( &thread2, NULL, inc10, message2);

     pthread_join( thread1, NULL);
     pthread_join( thread2, NULL);

     printf("Final value: %d \n", n);
     exit(0);
}
```

Preguntas:

- Antes de ejecutar el programa, trata de predecir cuál es el valor mínimo y máximo que puede tener `n` al terminar la ejecución de los dos *threads*.

  Para ello ayúdate de una tabla como la que mostramos más abajo, y rellénala manualmente en un editor de textos, para diversas ejecuciones del programa, con el objetivo de que trates de predecir el valor de `n` al terminar los 2 *threads*.

  En el siguiente ejemplo mostramos una ejecución en la que a pesar de que cada *thread* ha realizado un incremento, el valor de `n` permanece en el valor 1, en lugar de tener el valor 2:

```
| Global   |  Thread 1:                 |  Thread 2:                 |
-------------------------------------------------------------------------
| n        |  reg  count sentencia      |  reg  count sentencia      |
-------------------------------------------------------------------------
| 0        |       0     count = 0      |                            |
| 0        |  0    0     reg = n        |                            |
| 0        |  0    0                    |       0     count = 0      |
| 0        |  0    0                    |  0    0     reg   = n      |
| 0        |  0    0                    |  1    0     reg   = reg +1 |
| 1        |  0    0                    |  1    0     n     = reg    |
| 1        |  1    0     reg = reg + 1  |  1    0                    |
| 1        |  1    0     n   = reg      |  1    0                    |
|...       | ...  ...                   | ...  ...                   |
```

  Escribe en el foro de la asignatura tu estimación de los valores posibles de `n`.

- Una vez realizado el apartado anterior, compila el programa, y ejecútalo unas cuantas veces para observar los valores de `n` de cada thread

```
$ gcc -o benari benari.c -lpthread}
$ ./benari
Thread 1 10
Thread 2 15
Final value: 15

$ for i in {1..10}; do ./benari; done
Thread 2 9
Thread 1 13
Final value: 13
Thread 1 10
Thread 2 17
Final value: 17
```

  Al finalizar cada ejecución se muestra también el valor final de `n`, que es el mismo que ha dejado el segundo thread que termina.

Es este valor final el que nos interesa observar.

¿Ha cambiado en algo tu estimación del rango de valores que puede tener el valor final de `n`? ¿Por qué?

Cambia los valores de `MAXDELAY`: prueba con valores menores como 10, y mayores como 10000 y 100000. Recompila y ejecuta de nuevo el programa cada vez que cambies el valor. ¿Ha cambiado en algo tu estimación del rango de valores que puede tener el valor final de `n`? ¿Por qué?

Escribe de nuevo en el foro de la asignatura tu estimación de los valores posibles de `n` tras haber compilado y ejecutado con diferentes valores de `MAXDELAY` el programa.

Uso de TLA+/Pluscal para analizar el programa benari.c del Ejercicio 6

Mostramos en la siguiente página la especificación TLA+/PlusCal `benari.tla`, que hemos escrito para descubrir de manera precisa el rango de valores de `n` que puede generar el programa `benari.c` una vez han concluído ambos *threads*.

Primero describimos el código de la especificación y luego te pedimos en un ejercicio que seas tú quien descubra usando TLC el rango de valores de `n`.

- Entre las líneas 5 y 10 hay un bloque **define**. Sirve para añadir definiciones TLA+, que si bien pueden ir en el módulo, fuera del comentario del algoritmo, quedan mejor situadas dentro del código PlusCal del algoritmo.

  En este caso se ha definido el conjunto `Threads`. Podríamos haber escrito esta línea antes del comentario que incluye el algoritmo PlusCal, pues se hace referencia a `Threads` en el algoritmo PlusCal.

  También hay 2 fórmulas, `Reaches20AsProperty` y `Reaches20AsInvariant`. Estas las podríamos haber escrito después del comentario del algoritmo, pues hacen referencia a la variable `n` que se define en el algoritmo PlusCal.

- Hay dos declaraciones de procesos, `inc10` en la línea 11 y `Watcher` en la línea 27. En la declaración del proceso `inc10` se crearan dos instancias, una para `"Thread_1"` y otra para `"Thread_2"`.

- Después de cada una de las etiquetas `A1, A2, A3 y A4` sólo hay una sentencia antes de encontrar la siguiente etiqueta. Por tanto sólo es atómica la ejecución de cada una de las asignaciones: después de cada una de las acciones de uno de los 2 procesos `inc10` se puede producir una acción del otro proceso. De esta forma modelamos la ejecución de los *threads* de *benari.c*, en la que tras cada sentencia de asignación hay un cambio de contexto a otro *thread*.

- El proceso `Watcher`, del que sólo se crea una instancia `"Watcher"` en su declaración, sirve para mostrar el valor de `n` una vez que han terminado los 2 procesos `"Thread_1"` y `"Thread_2"`.

  En el termómetro especificado en PlusCal sólo teníamos un proceso que no terminaba nunca, efecto que conseguíamos con `while (TRUE)`.

  Sin embargo en este caso cada uno de los dos procesos `inc10` terminan tras haber dado 10 vueltas del bucle `while`. Cuando un proceso termina, su contador de programa `pc` es la etiqueta `"Done"`.

benari.tla

```
---- MODULE benari ----
EXTENDS Integers, TLC

(* --algorithm benari {
variables n = 0;

define {
    Threads == {"Thread_1", "Thread_2"}
    Reaches20AsProperty == <> (n = 20)
    Reaches20AsInvariant == (/\ pc["Thread_1"] = "Done"
                             /\ pc["Thread_2"] = "Done") => n = 20
}

process (inc10 \in Threads)
variables reg, count = 0;
{
    Benari:
    while (count < 10) {
        A1:
        reg   := n;

        A2:
        reg   := reg + 1;

        A3:
        n     := reg;

        A4:
        count := count + 1;
    };
    \* print <<self, n>>
}

process (Watcher = "Watcher"){
    Watcher:
    when pc["Thread_1"] = "Done" /\ pc["Thread_2"] = "Done";
    print <<"Final value: ", n>>
}
}
*)
====
```

Al haber varios procesos, el contador de programa `pc` es una estructura de datos de TLA+ denominada función que veremos más adelante, cuyo uso en este caso es similar a un diccionario de Python, pudiéndose indexar por la clave del nombre del proceso el contador de programa de uno u otro proceso. Por ejemplo `pc["Thread_2"]` es el contador de programa del thread `"Thread_2"`.

La sentencia `when` en la línea 29 hace que no se muestre el valor de `n` hasta que hayan concluido los dos procesos, lo que se produce cuando sus respectivos contadores de programa almacenan la etiqueta `"Done"`.

- En la línea 30 se muestra el valor de `n` usando `print`, para lo cuál hemos tenido que importar el módulo `TLC` (línea 2). En este caso queremos mostrar 2 valores en la misma sentencia `print`, para lo cuál hemos usado una secuencia o tupla de TLA+ cuyo primer elemento es el *string* `"Final value: "` y el segundo separado por una coma es la variable `n`.
- Mira qué ocurre cuando descomentas la sentecia `print` en la línea 25. Descoméntala, traduce a TLA+ y ejecuta TLC. Observarás que la salida no es como la salida de `benari.c`: no sale el valor de `n` cuando termina el primer proceso/*thread*, luego el del segundo y finalmente el valor final de `n`. La razón es que para ahorrar tiempo, TLC explora todos los estados del grafo una sóla vez y luego genera los comportamientos que necesita. Por ello tienes que utilizar `print` con cuidado: no puedes relacionar en la salida los mensajes generados por `print` distintas.

Verificación de propiedades: Resulta evidente que las condiciones de carrera del programa `benari.c` hacen que no siempre se alcance el valor de 20 para la variable `n`.

Podemos verificar si la especificación satisface la propiedad `Reaches20AsProperty` añadiéndola en la sección `PROPERTIES` de `benari.cfg`. Si lo haces y verificas con TLC comprobarás que se produce un error ya que no se satisface, mostrando TLC un comportamiento o ejecución en la que efectivamente `n` no ha llegado a alcanzar el valor 20. Pero se trata de una ejecución en la que los dos procesos `Thread_1` y `Thread_2` no han realizado 10 vueltas cada uno: lo puedes ver porque la variable local `count` de ambos procesos no ha llegado al valor 10. También puedes verlo porque ni `pc["Thread_1"]` ni `pc["Thread_2"]` tienen el valor `"Done"`.

La traza que ha encontrado TLC que no satisface la propiedad `Reaches20AsProperty` es una en la que se queda en un estado tartamudeante.

Ya sabes que cambiando la declaración del proceso `inc10` por esta otra conseguimos que mientras haya acciones que pueden provocar una transición en el proceso, se acaben ejecutando:

```
fair process (inc10 \in Threads)
```

Haz el cambio y comprueba que ahora TLC sigue no verificando la propiedad, mostrando una nueva traza en la que `n` no llega a alcanzar el valor de 20. Sin embargo en esta traza sí ha concluido la ejecución de ambos procesos: la variable local `count` ha llegado al valor de 10 en ambos casos, y ambos contadores de programa tienen el valor `"Done"`.

Es importante que entiendas que esta traza es sólo una de las muchas que podría mostrar TLC en las que no se alcanza el valor de 20 para `n`.

Otra forma de comprobar que la especificación tiene comportamientos en los que `n` no alcanza el valor 20 es usando como invariante la fórmula `Reaches20AsInvariant`. Si la añades a la sección `INVARIANTS` de `benari.tla` y ejecutas TLC obtendrás un error y un comportamiento/ejecución en el que ambos procesos han terminado pero `n` no ha llegado a alcanzar el valor 20. El predicado de la invariante se evalúa en todos los estados, pero como el predicado tiene una implicación lógica (`=>`), el término a la derecha de la implicación (el consecuente) sólo debe satisfacerse cuando el término a la izquierda (antecedente) es `TRUE`, es decir, cuando `(pc["Thread_1"] = "Done" /\ pc["Thread_2"] = "Done")` es `TRUE`. Al encontrar una traza en la que, siendo `TRUE` el antecedente, es `FALSE` el consecuente, el predicado es `FALSE`[69] y por tanto la invariante no se cumple siempre.

[69] Recuerda la tabla de verdad de `=>`: `TRUE => FALSE` es `FALSE`

En el siguiente ejercicio te pedimos que averigües el rango de los valores de `n`:

Ejercicio 7: rango de valores de n en benari.tla

Cuando el programa `benari.c` es presentado a expertos en programación concurrente muchos de ellos no consiguen predecir el rango de `n`. Haz la prueba si quieres.

Te sorprenderá descubrir cuáles son los valores que puede adoptar la variable `n` independientemente de tus conocimientos sobre concurrencia. Usando TLC para verificar propiedades de la especificación `benari.tla` te resultará muy sencillo.

- Añade las propiedades de tipo *safety*, o de tipo *liveness*, o propiedades de acciones, que consideres oportuno para descubrir el rango de valores que puede adoptar la variable global `n`. Fíjate en las fórmulas `Reaches20AsInvariant` y `Reaches20AsProperty` como pista para realizar este ejercicio.
- Escribe una fórmula que te permita verificar con TLC que el valor de `n` no siempre es creciente.

Para entregar este ejercicio tienes que enviarnos el rango de valores de `n` que has descubierto, acompañándolo del fichero

`benari.tla` que incluya las nuevas fórmulas que has usado, y el fichero `benari.cfg` que has utilizado.

Especificación de un sistema para la evaluación de prácticas de estudiantes

Se quiere desarrollar un nuevo sistema de información para mejorar la realización de los exámenes orales de prácticas de una asignatura en la universidad. Un grupo de profesoras se encarga de corregir a los estudiantes de una asignatura.

Requisitos:

- Cada profesora seleccionará primero a un subconjunto de estudiantes a las que prefiere examinar. Fijará la fecha del examen y llegado ese día les examinará.
- Si una profesora seleccionó a un subconjunto de estudiantes para examinarles pero finalmente no se acaba celebrando el examen por cualquier razón, el sistema deberá reasignar de nuevo a esas estudiantes a alguna profesora para que finalmente sean examinadas.
- Todas las estudiantes tienen que acabar siendo seleccionadas por alguna profesora. Si alguna estudiante no ha sido seleccionada por ninguna profesora, o si ha sido seleccionada por alguna profesora pero por la razón que sea no ha podido finalmente ser examinada, tiene que haber algún mecanismo para que acaben siendo examinadas todas las estudiantes.
- Cada estudiante sólo puede ser examinada por una profesora como máximo.

Primera versión

`assessments_1.tla` es la primera versión de la especificación, que mostramos en la siguiente página. Hemos abstraido muchos detalles, centrándonos en esta versión en la selección de subconjuntos de estudiantes y en la realización del examen, sin que una estudiante pueda ser examinada por más de una profesora.

Dejamos para más adelante el que toda estudiante acabe siendo examinada en caso de que la profesora que la seleccionó no pueda hacerlo, o en caso de que no fuera seleccionada por ninguna profesora.

- En las líneas 2 y 3 definimos los conjuntos de profesores examinadores y de estudiantes que tienen que examinar.
- En la línea 6 definimos la variable `graded_students` en la que se guardará el conjunto de estudiantes que ha evaluado cada profesor. Se inicializa asignando el conjunto vació a cada uno de los profesores. Se utiliza para ello una función de TLA+, que es equivalente a una estructura de datos como el diccionario de lenguajes de programación como Python.

assessments_1.tla

```
---- MODULE assessments_1 ----
Professors == {"prof_1", "prof_2"}
Students   == {"student_1", "student_2", "student_3"}

(*--algorithm Assessments {
variables
    graded_students = [p \in Professors |-> {}]

define {
    NoDoubleGrading == \A p1, p2 \in Professors:
      graded_students[p1] \cap graded_students[p2] /= {} => p1 = p2
}

process (Professor \in Professors)
    variable selected = {};
{
    SelectStudents:
    with (s \in SUBSET Students) {
        selected := s
    };

    ExamDay:
    either {
        (* Exam done! *)
        graded_students[self] := selected
    }
    or  {
        (* Professor can't finally do the exam *)
        skip;
    }
}
}
*)
====
```

La función se expresa entre corchetes. En el dominio de la función está el conjunto de los profesores. A cada uno se le asigna con el símbolo `|->` el conjunto vacío `{}`.

- En la línea 11 se declara el proceso `Professor`, del que se creará una instancia para cada una de las profesoras. Este proceso modela las dos actividades que realiza cada profesora: seleccionar un subconjunto de estudiantes que examinará (etiqueta `SelectStudents:`) y después examinarles o no examinarles (etiqueta `ExamDay:`).
- La selección de los estudiantes que hace cada profesor se realiza en la línea 15, donde con `with` se escoge un subconjunto de estudiantes (`SUBSET`) que se guarda en la variable `selected` en la línea 16.

  Recuerda que `with` introduce indeterminismo: cuando TLC verifique la especificación, generará múltiples estados al llegar a `with`, tantos como subconjuntos distintos de estudiantes se pueden formar.
- La realización o no del examen se ha modelado con la sentencia `either` de la línea 19: la profesora puede examinar a los estudiantes que había seleccionado, almacenando los estudiantes examinados en `graded_students` (línea 21), o bien no realizar el examen (sentencia `skip` que no tiene ningún efecto, en la línea 25). De esta forma modelamos la posibilidad de que una profesora que había seleccionado un subconjunto de estudiantes finalmente no pueda examinarles.
- En la línea 8 se define una fórmula cuyo significado es un predicado sobre estados[70]. Es una propiedad de tipo *safety* para asegurarnos de que cada estudiante no es escogida más que por un profesor como máximo:

[70] Lo sabemos porque no hay variables primadas ni símbolos de lógica temporal como `[]`, `<>`, `o` ~>.

```
NoDoubleGrading == \A p1, p2 \in Professors:
  graded_students[p1] \cap graded_students[p2] /= {} => p1 = p2
```

La única novedad es el símbolo `\cap`, que es la intersección de conjuntos ∩[71]. Podemos leer la fórmula así:

[71] Para la unión de conjuntos utilizamos el símbolo `\cup` ∪.

> Para todo `p1, p2`, ambos miembros del conjunto `Professors`, el predicado es cierto si cuando la intersección de los estudiantes examinados por `p1` y por `p2` es cierta, entonces es que `p1` es igual a `p2`.
>
> Dicho de otro modo: el predicado es cierto cuando no hay intersección entre los conjuntos de estudiantes examinados por dos profesoras distintos.

Podríamos haber escrito la fórmula de este modo, teniendo el mismo significado:

```
NoDoubleGrading == \A p1 \in Professors: \A p2 \in Professors:
  graded_students[p1] \cap graded_students[p2] /= {} => p1 = p2
```

Para verificar con TLC la especificación añadimos el nombre de la fórmula `NoDoubleGrading` a la sección `INVARIANTS` de `assessments_1.cfg` y verificamos con TLC la especificación[72].

[72] Recuerda traducir a TLA+ el algoritmo PlusCal tras haber añadido la fórmula en el bloque `define`

Al hacerlo TLC da error pues la invariante no se satisface en todos los comportamientos, y nos muestra uno en el que una misma estudiante acaba siendo examinada por más de un profesor.

El problema está en que tras seleccionar un profesor a un subconjunto de estudiantes (líneas 14-17), hay otra etiqueta, y por lo tanto antes de examinar a sus estudiantes, otros profesores pueden seleccionar y examinar a un subconjunto de estudiantes entre los que está alguno que ya ha sido examinado.

Podríamos comentar entonces la etiqueta `ExamDay:`. Hazlo y comprobarás que se sigue violando la invariante: puede llegar un profesor, seleccionar a un subconjunto de alumnos, y examinarles, ejecutándose ambas operaciones de manera atómica. Pero nada impide que llegue después otro profesor y seleccione y examine, de manera atómica, a un subconjunto de estudiantes que tenga intersección con el anterior. Eso es lo que muestra TLC.

Además, no tiene sentido que la selección de estudiantes y su examen se produzcan de manera atómica: se aleja de la realidad que intentamos modelar ya que un profesor primero selecciona, y días después examina. Entre medias otro profesor puede seleccionar estudiantes. Una especificación donde ambas tareas se realizan de manera atómica no es fiel a la realidad modelada.

Segunda versión

Necesitamos por tanto que tras haber seleccionado un profesor un subconjunto de estudiantes, estos ya no pueden ser seleccionados por otro profesor. Para ello creamos la nueva especificación `assessments_2.tla` que mostramos en la siguiente página.

- Se ha declarado una nueva variable `locked` que se inicializa con un conjunto vacío (línea 7).
- Se van añadiendo a este conjunto los estudiantes que cada profesor va seleccionando (línea 20) quedando así bloqueados (*locked*) para que otros profesores no puedan seleccionarlos también para examinarlos.
- Cuando un profesor va a seleccionar estudiantes para examinarlos, escoge sólo entre los estudiantes que aún no han sido seleccionados por otros profesores: `Students \ locked` en la línea 16 es precisamente el conjunto de todos estudiantes, menos los que están en el conjunto `locked`. Estos son los que no han sido seleccionados. El operador `\` es la diferencia de conjuntos.

AL VERIFICAR CON TLC comprobamos que la propiedad sigue sin verificarse. El error nos muestra una ejecución en la que alguna estudiante ha sido examinada por dos profesoras.

El problema está en la etiqueta `LockStudents:` que hemos añadido en la línea 19. Recuerda que las sentencias que hay entre 2 etiquetas se ejecutan de manera atómica. Al añadir la etiqueta `LockStudents:`, tras seleccionar en la línea 17 un profesor sus estudiantes, y antes de que se añadan a `locked`, pueden ejecutarse las sentencias de selección de estudiantes de otro profesor, pudiendo elegir los mismos estudiantes, ya que aun no han sido añadidos a `locked` por el profesor que los seleccionó primero.

¡Necesitamos que seleccionar estudiantes y bloquearlos añadiéndolos a `locked` sea una operación atómica!. La solución es sencilla: comentamos la etiqueta `LockStudents:`. De este modo las sentencias que hay entre las líneas 16 y la 20 están todas debajo de una misma etiqueta `SelectStudents:`, por lo que se ejecutan de manera atómica.

Tras hacer este último cambio la propiedad `NoDoubleGrading` es satisfecha por nuestra especificación como puedes comprobar traduciendo el algoritmo tras haber comentado la etiqueta `LockStudent:`, y verificando con TLC de nuevo. Hazlo ahora, nos sigas leyendo hasta haberlo comprobado.

En la implementación real la selección la hará el profesor, pero el bloqueo lo hará automáticamente el sistema.

assessments_2.tla

```
---- MODULE assessments_2 ----
Professors == {"prof_1", "prof_2"}
Students   == {"student_1", "student_2", "student_3"}

(*--algorithm Assessments {
variables
    graded_students = [p \in Professors |-> {}],
    locked = {};



define {
    NoDoubleGrading == \A p1, p2 \in Professors:
          graded_students[p1] \cap graded_students[p2] /= {} => p1 = p2
}

process (Professor \in Professors)
    variable selected = {};
{
    SelectStudents:
    with (s \in SUBSET (Students \ locked)) {
        selected := s
    };

    LockStudents:
    locked := locked \cup selected;

    ExamDay:
    either {
        (* Exam done! *)
        graded_students[self] := selected
    }
    or  {
        (* Professor can't finally do the exam *)
        skip;
    }
}
}
*)
====
```

Tercera versión

Abordamos ahora el requisito de que todas las estudiantes tienen que acabar siendo examinadas.

Para ello podemos añadir la nueva fórmula `AllStudentsGraded` en el bloque `define` de la especificación:

```
AllStudentsGraded == <>(\A s \in Students: \E p \in Professors: (s \in graded_students[p]))
```

Podemos leer el predicado como:

> Eventualmente[73] para toda estudiante `s` del conjunto `Students` se cumple que (`:`) existe un profesor `p` del conjunto `Professors` tal que (`:`) la estudiante `s` está en el conjunto de las estudiantes que ha examinado el profesor `p` (`graded_students[p]`).

[73] Eventualmente = tarde o temprano = en el estado actual o en alguno posterior.

Dicho de otro modo, toda estudiante acaba estando entre las estudiantes examinadas por algún profesor.

Para verificar la propiedad `AllStudentsGraded`, añade su nombre a la sección `PROPERTIES` del fichero de configuración.

Si lo haces y verificas con TLC, comprobarás que la propiedad no se satisface. TLC muestra una ejecución en la que hay estudiantes que no están entre los examinados: no aparecen en `graded_students`. Puedes ver en la ejecución que muestra TLC en la que no se satisface la propiedad, que hay estudiantes que están en la variable `selected` de algún profesor[74], y están bloqueados en la variable `locked`, pero no aparecen entre los examinados en la variable `graded_students`.

[74] Como cada proceso profesor tiene una variable local `selected`, TLA+ ha generado una función `selected`, con dominio los profesores, y valores el conjunto de estudiantes seleccionado por cada profesor.

Esto es debido, como quizá has adivinado, a que tras seleccionar y bloquear un profesor un subconjunto de estudiantes, puede acabar no examinándoles al ejecutarse la rama `or` de `either`.

PARA SOLUCIONARLO escribimos una nueva versión de la especificación, `assessments_3.tla` que mostramos en la siguiente página.

- Añadimos el proceso `Cleanup` (línea 34) para que compruebe si, una vez terminados todos los procesos de los profesores, quedan estudiantes sin examinar. La sentencia `when` hace que el resto de sentencias sólo se ejecuten una vez que todos los procesos `Professor` hayan terminado su ejecución.

  En la variable `locked` están todos los estudiantes que han sido seleccionados por algún profesor. En la línea 37 se asignan a algún profesor todos los estudiantes que no han sido examinados aún (`Students \ locked`). Al utilizar la sentencia `with` se explorarán las ejecuciones en las que se le asignan a diferentes profesores.

  ¡Pero en la variable `locked` también están los estudiantes que fueron seleccionados por algún profesor y quedaron bloqueados, pero no llegaron a examinarse!. Por eso tenemos que hacer también este otro cambio para que estos estudiantes acaben siendo examinados:

- Si un profesor no acaba examinando a los estudiantes que seleccionó y quedaron bloqueados, hay que desbloquearlos, sacándolos de `locked`. Esto se hace en la línea 31, donde se quitan de `locked` los estudiantes que fueron seleccionados y bloqueados, pero no fueron examinados[75].

Usando TLC para verificar `assessments_3.tla` TLC falla, mostrando una ejecución en la que vemos que el nuevo proceso `Cleanup` no ha terminado: vemos que su contador de programa no es `"Done"` sino `"Finally"`, por lo que no ha llegado a ejecutar la asignación de estudiantes sin examinar a algún profesor. Si te fijas en el último estado mostrado por TLC, es un estado tartamudeante (*stuttering*).

Necesitamos que la ejecución del proceso `Cleanup` no se pare ahí. Para ello añadimos `fair` a la declaración del proceso como de costumbre[76].

Si traduces a TLA+ y vuelves a ejecutar TLC vuelve a dar un fallo. Observamos el último estado que nos muestra en la ejecución y ahora comprobamos que no ha terminado de ejecutarse de nuevo el proceso `Cleanup`. ¿Por qué, siendo que ahora si que hemos añadido `fair` a la declaración? Observamos que hay algún profesor cuyo contador de programa no es `"Done"`. ¡Esto hace que la sentencia `when` de la línea 36 no permita avanzar al proceso `Cleanup`!.

Necesitamos que los procesos `Professor` concluyan su ejecución para que el proceso `Cleanup` pueda hacer su trabajo. Añadimos para ello `fair` a la declaración del proceso `Professor`[77].

Ahora finalmente TLC verifica que la propiedad `AllStudentsGraded` es verificada por la especificación.

[75] Si bien en la especificación ponemos este código en el proceso del profesor, en la implementación real habrá que programar en el sistema este desbloqueo automático, pues no podemos esperar que el profesor recuerde desbloquear los alumnos seleccionados a los que no pudo examinar.

[76] Evidentemente en el sistema real cuando esté implementado, para que se comporte como nuestra especificación cuando añadimos `fair` al proceso `Cleanup`, tiene que ocurrir que el proceso no falle. Habrá que hacer que el sistema sea tolerante a fallos para lograrlo. Pero si no falla, un sistema implementado como nuestra especificación conseguirá que todos los estudiantes sean examinados.

[77] TLC aquí nos ayuda a darnos cuenta de que no podemos hacer que en nuestra implementación la funcionalidad modelada por `Cleanup` dependa de que los profesores acaben haciendo algo en el sistema: debemos reasignar estudiantes no examinados aun cuando un profesor no haya terminado de utilizar el sistema tras haber seleccionado a unos cuantos estudiantes. Esto lo podemos implementar dando por terminada la labor del profesor, haya hecho su trabajo o no, el día que hay que ejecutar las acciones de `Cleanup`.

assessments_3.tla

```
---- MODULE assessments_3 ----
Professors == {"prof_1", "prof_2"}
Students   == {"student_1", "student_2", "student_3"}

(*--algorithm Assessments {
variables
    graded_students = [p \in Professors |-> {}],
    locked = {};



define {
    NoDoubleGrading == \A p1, p2 \in Professors:
        graded_students[p1] \cap graded_students[p2] /= {} => p1 = p2
    AllStudentsGraded ==
        <>(\A s \in Students: \E p \in Professors: (s \in graded_students[p]))
}

process (Professor \in Professors)
        variable selected = {};
{
    SelectStudents:
    with (s \in SUBSET (Students \ locked)) {
        selected := s
    };
    locked := locked \cup selected;

    ExamDay:
    either {
        (* Exam done! *)
        graded_students[self] := selected
    }
    or  {
        (* Professor can't finally do the exam *)
        skip;
        (* unlock students *)
        locked := locked \ selected;
    }
}

process (Cleanup = "cleanup")
{   Finally:
    when \A p \in Professors: pc[p] = "Done";
    with (p \in Professors) {
        graded_students[p] := graded_students[p] \cup (Students \ locked)
    }
}
}
*)
====
```

Lecciones aprendidas: Esta especificación se ha realizado para poder pensar mejor en algunos de los aspectos del diseño de la aplicación, pero sólo especifica cómo debe comportarse el sistema en relación a los requisitos funcionales identificados para la aplicación[78].

En la especificación no se han tenido en cuenta muchos detalles[79], pero cualquiera que sea la implementación elegida, deberá comportarse como esta especificación en cuanto a los aspectos concretos que, a diferencia de los detalles que se han abstraído, sí que se han tenido en cuenta, y que hemos ido descubriendo en las sucesivas versiones que hemos generado al ir explorando alternativas de diseños de la especifiación:

- Cuando un profesor seleccione unos estudiantes para examinarlos el sistema tiene que bloquearlos, para que otro profesor no los pueda seleccionar también.
- La selección de estudiantes que hace un profesor y el bloqueo de esos estudiantes tiene que ser una operación atómica.
- Cuando todos los profesores hayan terminado de utilizar el sistema[80], hay que comprobar si alguno tenía estudiantes bloqueados que finalmente no fueron examinados, y si es así, hay que desbloquearlos y asignarlos a algún profesor.
- Cuando todos los profesores hayan terminado de utilizar el sistema, este tiene que comprobar si hay estudiantes que no han sido seleccionados por ningún profesor para asignarlos a alguno que los examine.

Tras haber verificado con TLC que la especificación satisface las propiedades que hemos escrito a partir de los requisitos, ganamos cierta confianza en que una implementación que satisfaga esta especificación permitirá que el sistema implementado satisfaga los requisitos.

En la implementación que programemos introduciremos fallos en el código que habrá que intentar descubrir con pruebas de software (*testing*). Sin embargo, haber pensado en el diseño mientras realizábamos la especificación hará que sea más probable que los aspectos de diseño no abstraídos que se han detallado en la especificación, sean tenidos en cuenta por la implementación desde el principio. Si por el contrario comenzamos a implementar el sistema sin pensar antes en estos aspectos importantes, cuando la implementación ya esté avanzada será más costoso modificarla una vez que descubramos que el sistema no satisface los requisitos porque no pensamos antes en cómo satisfacerlos en nuestra implementación.

[78] Requisitos:

- Cada profesora seleccionará primero a un subconjunto de estudiantes a las que prefiere examinar. Fijará la fecha del examen y llegado ese día les examinará.
- Si una profesora seleccionó a un subconjunto de estudiantes para examinarles pero finalmente no se acaba celebrando el examen por cualquier razón, el sistema deberá reasignar de nuevo a esas estudiantes a alguna profesora para que finalmente sean examinadas.
- Todos las estudiantes tienen que acabar siendo examinadas por alguna profesora. Si alguna estudiante no ha sido seleccionada por ninguna profesora, o si ha sido seleccionada por alguna profesora pero por la razón que sea no ha podido finalmente ser examinado, tiene que haber algún mecanismo para que acaben siendo examinadas todas las estudiantes.
- Cada estudiante sólo puede ser examinada por una profesora como máximo.

[79] Detalles no tenidos en cuenta en la especificación:

- En qué momento tienen que desbloquearse los estudiantes bloqueados pero no examinados: podría ser por ejemplo cuando tras haber seleccionado un profesor a unos estudiantes y haberlos bloqueado el sistema, llega la fecha del examen y no se realiza el examen, o podría ser cuando el profesor, previendo que no podrá realizar el examen, informa de la incidencia al sistema,...
- Cómo se avisa a los profesores a los que el sistema, tras comprobar que hay estudiantes pendientes de examinar, les acaba asignando estudiantes para examinar sin que ellos los hubieran seleccionado
- Si se ejecutará el sistema en una aplicación Web o en una aplicación en el móvil.
- Si se almacenarán los datos en una base de datos o en un fichero.
- etc.

[80] O cuando han tenido lugar todos los exámenes: el momento no se ha tenido en cuenta en la especificación.

Ejercicio 8

- En la especificación `assessments_3.tla` hemos conseguido finalmente que todas las estudiantes sean examinadas. El proceso `Cleanup` se encarga de asignar a las que no han sido examinadas a alguna profesora para que las examine.

  Pero cabe preguntarse: una vez que han terminado todos los procesos `Professor`, ¿puede ocurrir que no todos los estudiantes hayan sido seleccionados por algún profesor para ser examinados? Los requisitos parecen querer evitar que algún estudiante sea discriminado, y no sea elegido por ninguna profesora para ser examinado, aunque con el proceso Cleanup finalmente se le examinará. Crea alguna propiedad que permita detectar si efectivamente la especificación `assessment_3.tla` permite que algunos estudiantes sean discriminados y acaben no siendo seleccionados por ninguna profesora antes de ser finalmente examinados gracias al proceso Cleanup.

Especificación del sistema de control semafórico para un puente de un sólo carril

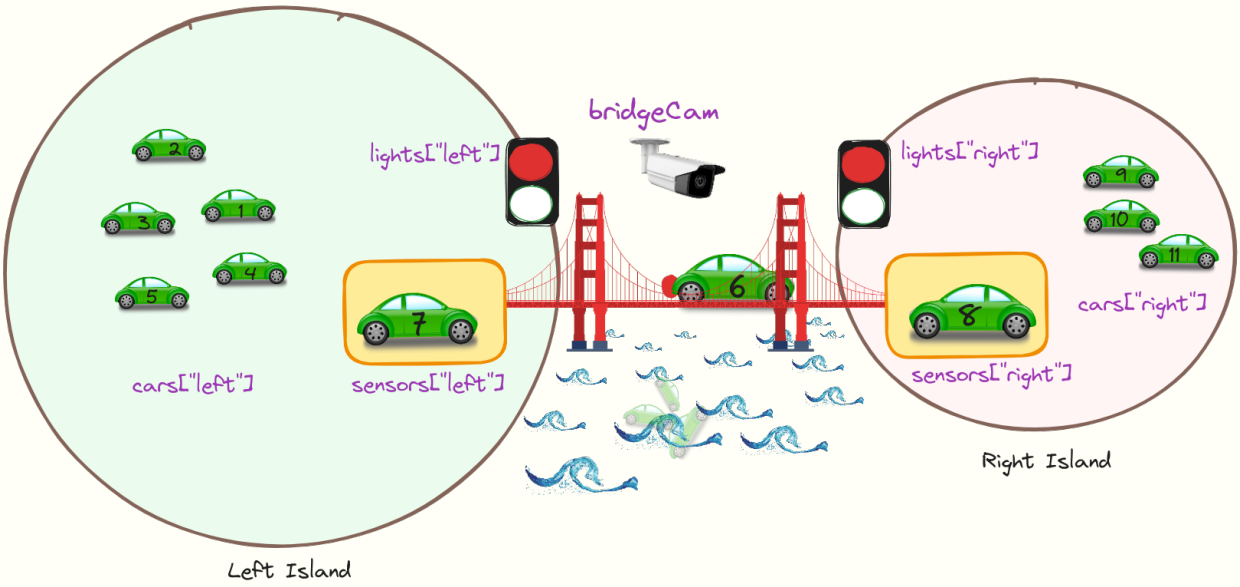


Figura 3: Diagrama del proyecto de control semafórico del puente de un solo carril

En esta sección especificaremos paso a paso un sistema semafórico para controlar el tránsito de automóviles por un puente de un sólo carril[81] que une las islas *Left Island* (en adelante isla de la izquierda) y *Right Island* (en adelante isla de la derecha).

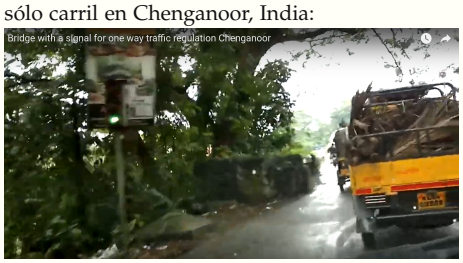
En el puente entre las dos islas ha habido muchos accidentes graves causados por el choque de vehículos que circulaban en sentido contrario ya que sólo cabe un coche en el puente y a los isleños les cuesta ponerse de acuerdo en quién tiene el derecho de paso[82].

Primero presentamos los requisitos del sistema que se desea implementar. Después introduciremos algunos conceptos que nos ayudarán a entender el proceso de crear una especificación paso a paso, y finalmente describiremos la especificación.

Requisitos:

- Las islas son pequeñas y cada una de ellas tiene una capacidad máxima de automóviles, que debe regularse a través del sistema de control semafórico: no se puede dejar acceder a un automóvil al puente si la isla destino está llena.
- En cada extremo del puente hay un semáforo que puede estar en rojo o en verde.
- Los dos semáforos nunca pueden estar en verde a la vez.
- En cada extremo del puente hay un sensor en el asfalto para detectar si hay algún automóvil esperando a cruzar el puente.

[81] Ejemplo: puente de un sólo carril en Chenganoor, India:



https://www.youtube.com/watch?v=2xgszPnF8Jo

[82] Si bien el vídeo mostrado más arriba es de la India, la necesidad del sistema semafórico ha surgido en unas islas de España, un país en el que este tipo de comportamiento incívico dentro y fuera de las carreteras es frecuente.

- Si no hay un automóvil esperando en el sensor de una isla, su semáforo está en rojo.
- El semáforo de una isla origen sólo se pone en verde para dar paso a los vehículos que viajan hacia la otra isla cuando hay algún vehículo esperando en el sensor de la isla origen, y aún caben vehículos en la isla destino.
- Hay una cámara en el puente que permite saber cuántos automóviles hay en el puente.
- Según la autoridades, el puente es muy antiguo, por lo que no soporta demasiado peso. Por ello el nuevo sistema semafórico deberá impedir que haya más de un automóvil en el puente[83]. En cuanto un vehículo entra en el puente y abandona el sensor al ponerse el semáforo en verde, este deberá pasar inmediatamente a rojo.
- Mientras que un vehículo transita por el puente el semáforo que regula el acceso a los automóviles que circulan en el sentido contrario ha de permanecer en rojo.

[83] Los noticieros de la radio local sostienen que la verdadera razón que ha llevado a las autoridades a exigir este estricto requisito es que los isleños, al igual que hacen sus compatriotas peninsulares, acostumbran a dar frenazos adrede cuando los vehículos que circulan detrás de ellos les lanzan ráfagas de luces largas para que los que van delante atraviesen más deprisa el puente. Con frecuencia esta peligrosa costumbre tiene trágicas consecuencias pues a veces los frenazos acaban produciendo choques que provocan que los coches acaben en el fondo del mar (obsérvese con atención el lecho marino en la figura 3). Tan frecuente es este tipo de incidentes que muchos peatones han adquirido la costumbre de salir a pasear cerca del puente para asistir al macabro espectáculo.

La Máquina y el Mundo

Antes de crear la especificación TLA+/PlusCal presentamos alguna terminología[84] que nos ayudará a entender las decisiones a las que nos enfrentamos al concebir una especificación de un sistema como el del control semafórico del puente de un sólo carril.

El sistema que se quiere construir pretende mejorar un sistema existente: el uso actual muy mejorable que los isleños hacen del puente de un sólo carril.

Llamamos máquina al software que tenemos que implementar y al ordenador u ordenadores en el que se ejecuta dicho software.

El sistema que se quiere construir consta de una máquina y unos elementos externos en el entorno de la máquina como sensores y actuadores, con los que la máquina interactúa directamente, monitorizando los sensores (sensores de asfalto y cámara del puente) y controlando los actuadores (los semáforos). Los elementos en el entorno de la máquina están compartidos con el mundo. En el mundo hay otros elementos externos a la máquina, no monitorizados ni controlados por la máquina: los conductores isleños y los vehículos.

El estado de los **elementos externos en el entorno de la máquina** tiene una representación en la memoria de la máquina en forma de variables: variables para representar las entradas recibidas de los sensores y de la cámara, y variables para representar las salidas que se envían a los semáforos. En caso de que el estado del semáforo no sólo se pudiera modificar desde el software de la máquina por ejemplo por un operario accediendo físicamente al semáforo,

[84] Utilizamos la terminología máquina/mundo introducida en el mundo de la ingeniería de software por el profesor Michael Jackson `https://www.theworldandthemachine.com/p/about.html`.

también tendría que estar monitorizado, para que las variables del software representen lo más fielmente su estado.

Hay otras variables en la memoria de la máquina cuyos valores son calculados, como el número de coches que hay en cada isla.

El hardware de los sensores y la cámara, y los semáforos, está fuera de la máquina, y si bien el estado interno de la máquina es una representación de su estado, su funcionamiento es autónomo, aunque monitorizado y controlado por la máquina.

Cuando diseñamos la especificación de un sistema que interactúa con el mundo fuera de la máquina, conviene siempre ser conscientes de que es distinto afirmar algo sobre el estado de los sensores reales monitorizados y sobre el estado de los actuadores reales controlados, que afirmar algo sobre la percepción que el sistema tiene del estado de los sensores y actuadores, o de otros elementos externos fuera del entorno de la máquina. Este estado percibido, almacenado en variables de la máquina, no siempre tiene por qué ser fiel a la realidad, especialmente si se considera que puede haber fallos en los elementos externos a la máquina.

Los requisitos del sistema que se quiere implementar se expresan mediante frases que son sentencias prescriptivas[85] que están enunciadas en términos de elementos y fenómenos del exterior de la máquina: si lees los requisitos verás que hay frases que afirman cómo deben comportarse los coches, los conductores, los sensores, los semáforos,... Todos estos son elementos externos a la máquina, algunos pertenecientes a su entorno y otros no. Es responsabilidad del sistema que se quiere implementar, compuesto por la máquina y por los elementos externos en el entorno de la máquina (sensores, cámara, semáforos) el que se satisfagan o no los requisitos del sistema.

[85] Que dan órdenes, que son preceptivas, que se deben cumplir.

Los requisitos del software son traducciones de los requisitos del sistema. Son por lo tanto también frases o sentencias prescriptivas, pero enunciadas en términos del estado interno de la máquina, de las variables del software, y no de los elementos externos. Es responsabilidad exclusivamente del software de la máquina el que estos requisitos software se satisfagan o no.

Las suposiciones sobre los elementos del entorno son frases o sentencias prescriptivas también, pero enunciadas en términos de los fenómenos de los elementos del entorno. Es responsabilidad exclusivamente de los elementos del entorno el que se satisfagan estas suposiciones. El software de la máquina no puede hacer nada para asegurar que dicho comportamiento correcto tiene lugar. Por ejemplo, si hay un coche sobre el sensor del asfalto este debe detectarlo; si hay un coche en el puente la cámara ésta debe detectarlo; si se envía un comando al semáforo para que cambie a rojo se debe encender su bombilla roja,...

Las propiedades del dominio son sentencias descriptivas, no prescriptivas. Son afirmaciones acerca de propiedades del mundo, cuyo valor de verdad no depende del sistema que se quiere construir:

- Los automóviles, ya sean conducidos por humanos o sean vehículos robóticos, deben iniciar su marcha cuando estando parados en la posición de un sensor se ponga su semáforo en verde. Y nunca deben iniciar la marcha si el semáforo está en rojo. La máquina que tenemos que diseñar, es decir el sistema de control semafórico, no puede hacer nada para que esto ocurra, ni para impedirlo. El funcionamiento de los automóviles, sean o no autónomos, no es responsabilidad del sistema que nosotros tenemos que diseñar.
- Los automóviles deben continuar su marcha por el puente y no detenerse una vez han accedido a él, ni por decisión del conductor ni por fallos del automóvil. Tampoco deben cambiar el sentido de su marcha mientras están en el puente, sino que avanzan hasta llegar al final del puente y lo abandonan. Tampoco la máquina que nosotros tenemos que diseñar puede hacer que esto ocurra o impedirlo. Son los elementos externos que no están en el entorno de la máquina como los automóviles y los conductores los que deben satisfacer las propiedades del dominio.

La máquina no puede hacer que se cumplan las suposiciones sobre el comportamiento de los elementos en el entorno de la máquina, como los sensores y semáforos, ni puede hacer que se satisfagan las afirmaciones de las propiedades de los elementos externos que no están en el entorno dela máquina, como los humanos y los automóviles.

No forma parte de los requisitos de la máquina ni de los requisitos del software de la máquina el satisfacer las suposiciones sobre los elementos del entorno ni las afirmaciones de las propiedades del dominio.

El que se satisfagan los requisitos del sistema requiere que se satisfagan los requisitos del software de la máquina y que se satisfagan las suposiciones sobre el comportamiento de los elementos del entorno de la máquina, y que se satisfagan las afirmaciones de las propiedades del dominio.

Sólo es responsabilidad de la especificación el satisfacer los requisitos del software de la máquina. Por lo tanto la especificación se escribe en términos de las variables de la memoria del software de la máquina que representan el estado de los elementos del entorno de la máquina.

Sin embargo, para poder verificar la especificación utilizaremos variables que modelen también elementos externos como los automóviles. Ten siempre claro qué variables forman parte del es-

tado interno que controla la máquina directamente y qué variables modelan elementos externos, controlables o no directamente por la máquina.

Abstracciones

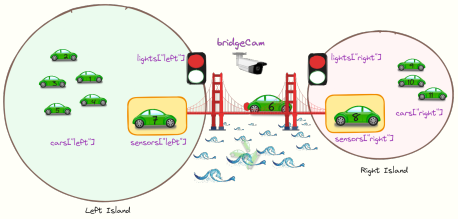
Cuando realizamos una especificación tenemos que abstraer muchos detalles del sistema que se quiere implementar, centrándonos sólo en algunos aspectos que nos interesa explorar en mayor detalle. Podemos hacer múltiples especificaciones, abstrayendo en cada una unos u otros detalles.

El objetivo de la abstracción no es ser menos concreto, sino crear un nuevo nivel semántico en el que se pueda ser absolutamente preciso[86].

[86] Esta es la traducción de una frase del discurso de Edsger W. Dijkstra en su conferencia ACM Turing Lecture, 1972: *"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."* https://www.cs.utexas.edu/~EWD/transcriptions/EWD03xx/EWD340.html

Al igual que hicimos en la especificación de las profesoras correctoras de exámenes, en esta especificación abstraemos la mayoría de los detalles relativos al funcionamiento interno de la máquina: *threads*, procesos, bases de datos, comunicación con los sensores y actuadores, y cualquier otro estado interno de la implementación del software de la máquina,... Puede haber otras especificaciones en las que algunos de estos elementos no se quieran abstraer: en ese caso la máquina no será todo el software como en este caso, sino quizá un subsistema software del mismo, y los elementos exteriores en el entorno de la la máquina serán otros subsistemas software o hardware que no forman parte de la máquina que queremos especificar. Piensa en lo que hicimos al especificar el programa con condiciones de carrera. Evidentemente el nivel de abstracción allí era muy diferente al que utilizamos en el sistema de corrección de exámenes y al que utilzaremos en el sistema de control semafórico del puente.

Variables para modelar los elementos externos que están en el entorno de la máquina: En la figura que aparece al comienzo de esta sección y que duplicamos a la derecha aparecen los nombres de las variables que usaremos para modelar los elementos externos a la máquina: sensores, cámara, semáforos y coches. Elegir cuidadosamente qué variables utilizar y qué nombre tienen es muy importante.



Las variables `lights`, `sensors` y `bridgeCam` sirven para almacenar los valores que se leen/escriben de/en los elementos externos que están en el entorno de la máquina. Recuerda que estos son los elementos que en la implementación final serán directamente monitorizables y/o controlables por la máquina. Por ello estas variables formarán parte de la implementación final de la máquina, ya se ejecute su software en uno o en múltiples procesos, en uno o en múltiples ordenadores.

- Para representar el estado de los semáforos, en la especificación usaremos la variable `lights`, una función TLA+ que almacena en `lights["left"]` y `lights["right"]` valores del conjunto `{"red", "green"}`.

  La interacción de la máquina con los semáforos permite cambiar la luz que emite un semáforo para que sea coherente con los

valores almacenados en estas dos variables. Esta comunicación se abstrae en la especificación: basta con asignar el nuevo valor del color del semáforo para modelar el control del semáforo.

- Para representar el estado que la máquina conoce sobre los dos sensores en el asfalto en los extremos del puente y la cámara en el puente utilizaremos las variables `sensors` y `bridgeCam`. `sensors["left"]` y `sensors["right"]` almacenan respectivamente el conjunto de identificadores de automóviles que hay en cada sensor, no pudiendo ser la cardinalidad del conjunto mayor de 1 pues sólo puede haber un coche en cada sensor. `bridgeCam` almacena también un conjunto de identificadores de automóviles, e igualmente la cardinalidad del conjunto no puede ser mayor de 1 pues sólo puede haber un coche en el puente.

  Suponemos que la interacción de la máquina con los sensores permite conocer si un sensor detecta la presencia de un automóvil o no, abstrayéndose la comunicación: basta con leer el valor de la variable para conocer el valor del sensor.

Variables para modelar los elementos externos que no están en el entorno de la máquina: En la especificación usamos la variable `cars` para modelar los conjuntos de identificadores de coches que hay en cada isla en cada momento: `cars["left"]` y `cars["right"]` almacenan los identificadores de los coches que la máquina cree que hay en cada isla. Los valores de esta variable se actualizan cuando la máquina detecta que los coches se desplazan de una isla origen a un sensor, de un sensor al puente y del puente a la isla destino.

Este tipo de variable es habitual en las especificaciones. Si bien estas variables no forman parte de la máquina, son necesarias para poder verificar la especificación. En la especificación tenemos que poder expresar el comportamiento de los elementos del mundo externo a la máquina y para ello tenemos que tener modelados estos elementos: los automóviles van a ser números enteros que vamos a ir moviendo de una isla al sensor en el asfalto de esa isla, del sensor al puente, y del puente a la otra isla.

En el software de la implementación final no estarán modelados los coches con identificadores, pues los coches no forman parte del estado de la máquina sino de los elementos externos en el entorno de la máquina (semáforos, sensores y cámara). Pero sí será necesario conocer el número de coches que hay en cada isla, información esta que se deducirá posiblemente contabilizando los coches que atraviesan el puente en uno u otro sentido.

En la especificación el número de coches lo abstraemos como la cardinalidad del conjunto (el número de coches en el conjunto).

Constantes y variables

En las anteriores especificaciones, cuando hemos tenido que utilizar constantes en las especificaciones hemos usado definiciones. Por ejemplo en la anterior especificaciones para las profesoras y las alumnas definíamos estas dos:

```
Professors == {"prof_1", "prof_2"}
Students   == {"student_1", "student_2", "student_3"}
```

Para este propósito existen en TLA+/PlusCal las constantes de TLA+/PlusCal. Se les da valor en el fichero de configuración de TLC y así una misma especificación se puede verificar con varios modelos en los que se dan valores distintos a las constantes. Mostramos más abajo el fichero de configuración `onewaybridge_pluscal.cfg`.

`onewaybridge_pluscal.cfg`

```
1  SPECIFICATION Spec

2  CONSTANTS
3      INITIAL_CARS_R = {"8","9","10","11"}
4      INITIAL_CARS_L = {"1","2","3","4","5","6","7"}
5      MAX_CARS_R = 5
6      MAX_CARS_L = 10
7      ISLANDS = {"left", "right"}
```

Constantes: En el fichero de la especificación `onewaybridge_pluscal.tla` que se muestra más abajo, al principio del módulo TLA+/PlusCal declaramos con la palabra `CONSTANT` la sección en la que declaran las constantes separadas por comas (línea 3).

- La constante `ISLANDS` se inicializará en el fichero de configuración de TLC con los valores `{"left", "right"}` (ver la línea 7 en el fichero `onewaybridge_pluscal.cfg`. La utilizamos para poder definir como una función TLA+[87] las variables de los sensores, los semáforos y los coches.
- `MAX_CARS_L` y `MAX_CARS_R` almacenan el número máximo de autos que cabe en cada isla.
- `INITIAL_CARS_R` e `INITIAL_CARS_L` almacenan los *strings* con los identificadores de los coches que hay en cada isla.

[87] Recuerda, las funciones las usamos como diccionarios de Python.

Con `ASSUME` escribimos en TLA+ aserciones, en este caso para asegurarnos de que los valores asignados a las constantes en el fichero de configuración de TLC son correctos.

- En la línea 9 nos aseguramos de que los identificadores de los coches son `STRING`.
- En las líneas 10 y 11 nos aseguramos de el número de coches que hay inicialmente en cada isla es menor o igual que la capacidad

onewaybridge pluscal.tla (parte 1/3)

```
---- MODULE onewaybridge_pluscal ----
EXTENDS FiniteSets, Integers

CONSTANT
    MAX_CARS_L,       \* max number of cars in left  island
    MAX_CARS_R,       \* max number of cars in right island
    INITIAL_CARS_L,   \* set of cars initially in left  island
    INITIAL_CARS_R,   \* set of cars initially in right island
    ISLANDS           \* set of names of islands

ASSUME \A car \in INITIAL_CARS_R \cup INITIAL_CARS_L: car \in STRING
ASSUME MAX_CARS_L >= Cardinality(INITIAL_CARS_L)
ASSUME MAX_CARS_R >= Cardinality(INITIAL_CARS_R)

MAX_CARS  == [island \in ISLANDS |-> IF island = "left" THEN MAX_CARS_L
                                     ELSE MAX_CARS_R],

(*--algorithm OneWayBridgeLights {
variables
    lights    = [island \in ISLANDS |-> "red"];
    sensors   = [island \in ISLANDS |-> {}],
    bridgeCam = {},

    cars      = [island \in ISLANDS |-> IF island = "left" THEN INITIAL_CARS_L
                                        ELSE INITIAL_CARS_R]
```

máxima de la isla. Para ello utilizamos el operador `Cardinality` que hay en el módulo `FiniteSets`, que importamos en la línea 2.

Variables:

- La variable `lights` almacena en una función los valores de los dos semáforos, inicializándose a `"red"` ambos.
- La variable `sensors` almacena los valores de los dos sensores, inicializándose ambos al conjunto vacío. La variable de la cámara del puente `bridgeCam` también se inicializa con un conjunto vacío.
- La variable `cars` almacena el número de coches que hay en cada isla, que se inicializa con los valores iniciales de las constantes `MAX_CARS_L` y `MAX_CARS_R`.
- Por último en la variable `MAX_CARS` almacenamos las capacidades máximas de coches de cada isla. Su propósito es poder acceder más fácilmente desde el código a los valores indexados por el nombre de la isla `"left"` o `"right"`.

Organización del código de la especificación:

En la especificación de los profesores que corrigen exámenes usamos un proceso para cada profesor sin justificar esa decisión.

Pero ahora sí queremos que tú pienses cómo modelar el sistema de control semafórico:

- ¿Cómo podríamos escribir la especificación en PlusCal utilizando las variables y constantes que hemos identificado?.
- Identifica qué procesos podemos crear: dales nombres y piensa qué tipo de código incluirías en ellos.

```
No sigas leyendo, piensa ahora en el diseño.
```

Una posibilidad es crear un proceso para cada sensor, uno para la cámara y para cada semáforo, y un proceso para cada automóvil. Esta alternativa tiene dos problemas:

- El número de procesos sería muy elevado por lo que habría muchas etiquetas en el código PlusCal: al menos una al principio de cada proceso. Esto tiene un efecto negativo en el rendimiento del verificador de modelos TLC ya que cada etiqueta de PlusCal es traducida en una acción TLA+ distinta. Cuantas más acciones, más comportamientos posibles hay que explorar, por lo que TLC tardaría mucho en ejecutar un mismo modelo. Esto haría que no pudiésemos explorar modelos en los que las constantes con el número de automóviles en cada isla fuese grande[88].
- El otro problema tiene que ver con la organización del código de la especificación. Cuando un coche intenta acceder al puente hay que consultar y/o modificar el estado del sensor y del semáforo. Cuando el coche entra en el puente hay que consultar y/o modificar el sensor, la cámara, el semáforo, y el número de coches en la isla. Y cuando un coche sale del puente hay que modificar el estado de la cámara y el número de coches en la isla destino.

  Si tenemos procesos para semáforos, sensores, cámara y coches, ¿dónde situarías el código de cada una de estas 3 actividades que hemos identificado: intento de acceder al puente, entrada en el puente, salida del puente? Acabarías teniendo código en el proceso de los semáforos que consulta los valores de los sensores, o código en los coches que modifican el estado de la cámara, etc., lo que puede hacer menos comprensible la especificación.

Para evitar los dos problemas modelamos con 3 procesos las 3 actividades que hemos identificado: `AccessToBridge`, `EnterBridge`, `ExitBridge`. Desde cada uno de estos 3 procesos se consultarán y/o modificarán las variables de los semáforos, sensores, cámara y automóviles.

[88] Verificar modelos con mayor número de elementos puede incrementar la confianza de que la especificación es correcta. No obstante, en muchos casos modelos pequeños sirven para detectar fallos en el diseño del sistema especificado.

Explicación de los procesos de la especificación: Finalmente, explicamos a continuación sólo algunos elementos nuevos que introducimos en la especificación que mostramos en la siguiente página. En un ejercicio te pedimos después que seas tú quien describa qué hace esta especificación.

Estas son las responsabilidades de los 3 procesos:

- `AccessToBridge` modela el acceso al puente: la llegada de coches a un sensor (línea 6) y el paso a verde de los semáforos (línea 12).

  `LET IN` sirve para introducir en expresiones TLA+ una definición local después de `LET` que sólo puede ser usada en el código TLA+ que hay en la parte `IN`. En la especificación se ha utilizado para definir `otherIsland` en dos de los procesos de la especificación[89].

- `EnterBridge` modela la entrada al puente de un automóvil. Cuando un coche entra en el puente se modela la detección del coche por la cámara añadiendo al conjunto `bridgeCam` una secuencia o tupla (parecida a una tupla de Python): «`car, otherIsland`» en la línea 35. De esta forma sabemos que en el puente está el coche `car` y que su destino es el valor de la isla contenido en la variable `otherIsland`.

- `ExitBridge` modela la salida del puente de un automóvil. En la línea 48 añadimos a la isla destino el coche que había en el puente. En la línea 41 `carInBridge` va adoptando como valor cada uno de los elementos de `bridgeCam`. Sólo debería haber un elemento en el puente, por lo que sólo adoptará el valor de ese elemento, que es una secuencia o tupla TLA+.

  Los elementos de las secuencias o tuplas son indexables por su posición `n`, comenzando por la posición 1 usando la notación `secuencia[n]`:

  > Mira la figura que hay al comienzo de la sección. Muestra al coche 6 en el puente, viajando hacia la derecha. Entonces `carInBridge` contiene esta secuencia TLA+: `<<"6", "right">>`. Y por tanto, en la línea 48 `carInBridge[1]` es `"6"` y `carInBridge[2]` es `"right"`.

Cuando verificamos con TLC esta especificación termina en unos segundos, dependiendo de la velocidad del ordenador en el que lo ejecutes. TLC genera cientos de miles de estados distintos.

Sin embargo, los 3 procesos tienen bucles infinitos (`while (TRUE)`). ¿Por qué termina entonces TLC y no continúa generando estados? La razón es que con los estados generados TLC ya puede generar cualquier comportamiento posible de esta especificación[90].

Exploración de los comportamientos de la especificación con TLC

En el siguiente ejercicio te pedimos que escribas propiedades de tipo *safety* y *liveness*, y propiedades de acciones.

[89] Podríamos haber creado una variable local al proceso. Sin embargo esta variable aparece luego en el diagrama de estados, quedando este menos claro. Por esta razón hemos preferido introducir LET IN en este caso para definir localmente `otherIsland`.

[90] Trata de comprender este párrafo, y si no lo consigues, pon una pregunta en el foro

onewaybridge_pluscal.tla (parte 2)

```
fair process (AccessToBridge = "accesstobridge")
{   AccessToBridge:
    while (TRUE) {
        with (anIsland \in ISLANDS) {
          either{ \* car moves into sensor
              when (sensors[anIsland] = {});
              with (car \in cars[anIsland]) {
                  sensors[anIsland] := sensors[anIsland] \union {car};
              }
          }
          or{ \* switch light into green
            when ( LET otherIsland == IF anIsland = "left" THEN "right" ELSE "left" IN
                   /\ sensors[anIsland] /= {}
                   /\ Cardinality(cars[otherIsland])
                        < MAX_CARS[otherIsland]
                   /\ bridgeCam = {}
                   /\ lights[anIsland]     = "red"
                   /\ lights[otherIsland]  = "red"
            );

            lights[anIsland] := "green";
}}}}

fair process (EnterBridge = "enterbridge")
{   EnterBridge:
    while (TRUE) {
        with (anIsland \in ISLANDS) {
            when ( /\ sensors[anIsland] /= {}
                   /\ lights [anIsland] =  "green" );
            with (car \in sensors[anIsland]) {
                sensors[anIsland] := {};
                cars[anIsland] := cars[anIsland] \ {car};

                lights[anIsland] := "red";

                bridgeCam :=
                    LET otherIsland == IF anIsland = "left" THEN "right" ELSE "left"
                    IN
                        bridgeCam \cup {<<car, otherIsland>>};
}}}}

fair process (ExitBridge ="exitbridge")
{   ExitBridge:
    while (TRUE) {
        when (bridgeCam /= {});
        with (carInBridge \in bridgeCam) {
            bridgeCam := {};
            cars[carInBridge[2]] := cars[carInBridge[2]] \cup {carInBridge[1]};
}}}}
*)
```

Sin embargo antes de hacerlo es conveniente que te familiarices con la especificación para comprender qué comportamientos debería generar si es correcta.

Podemos utilizar TLC para explorar los comportamientos de la especificación creando invariantes que, de ser correcta la especificación, no deberían ser satisfechas. De este modo TLC fallará y nos ofrecerá una ejecución en la que no se satisface la propiedad. Podremos así estudiar la ejecución mostrada por TLC para ver si el comportamiento es el esperado. Asegúrate de tener comentadas todas las propiedades salvo la que estás usando para que genere una ejecución.

Prueba por ejemplo a añadir estas invariantes para explorar el diseño:

```
Explore_1 == "1" \notin cars["right"]
Explore_2 == Cardinality(cars["left"]) /= MAX_CARS_L
Explore_3 == Cardinality(cars["right"]) /= MAX_CARS_R
Explore_4 == Cardinality(bridgeCam) /= 1
```

Exploración de los comportamientos de la especificación usando el grafo de estados

El diagrama de estados de la especificación tiene miles de estados. No es muy útil visualizarlo.

Pero si creamos un modelo en el fichero `onewaybridge_pluscal.cfg` con sólo 2 automóviles iniciales, `"1"` en la isla de la izquierda y `"2"` en la isla de la derecha, el grafo es mucho más pequeño y permite explorarlo visualmente. También fijaremos el máximo de coches de la isla de la izquierda a 2 y a 1 el de la isla de la derecha, para así poder observar comportamientos en los que se llena una isla y no pueden acceder coches[91]

Pruébalo: escribe el nuevo modelo con sólo 2 automóviles y ejecuta TLC. Sólo se generan unas decenas de estados.

Carga en VS Code el fichero `output.dot` que se genera automáticamente cuando ejecutas TLC. Arriba a la derecha en el panel del fichero `output.dot` aparece el icono de una lupa[92]. Pincha en él y se mostrará el diagrama del grafo de estados[93].

También puedes exportarlo con un botón que aparece arriba a la derecha para guardarlo en un fichero. Ábrelo con tu editor de PDF favorito y explóralo detenidamente. Recorre diferentes ejecuciones desde el estado inicial. Aprenderás mucho haciéndolo. Puedes verlo en la figura 4, pero no queremos darte el PDF para que aprendas a generarlo tú. No dejes de hacerlo: podrás verlo en grande y seguir todas las posibles ejecuciones.

[91] Prueba con diferentes valores de capacidad máxima de las islas y observa los diferentes grafos que se generan. Prueba por ejemplo con un máximo de coches de 2 en ambas islas y observa las diferencias en el grafo.

[92] 

[93] Si no te sale el icono de la lupa puedes ejecutar el siguiente comando: `Ctrl-Shift-P` y escribir `Graphviz: Open Preview to the Side`.

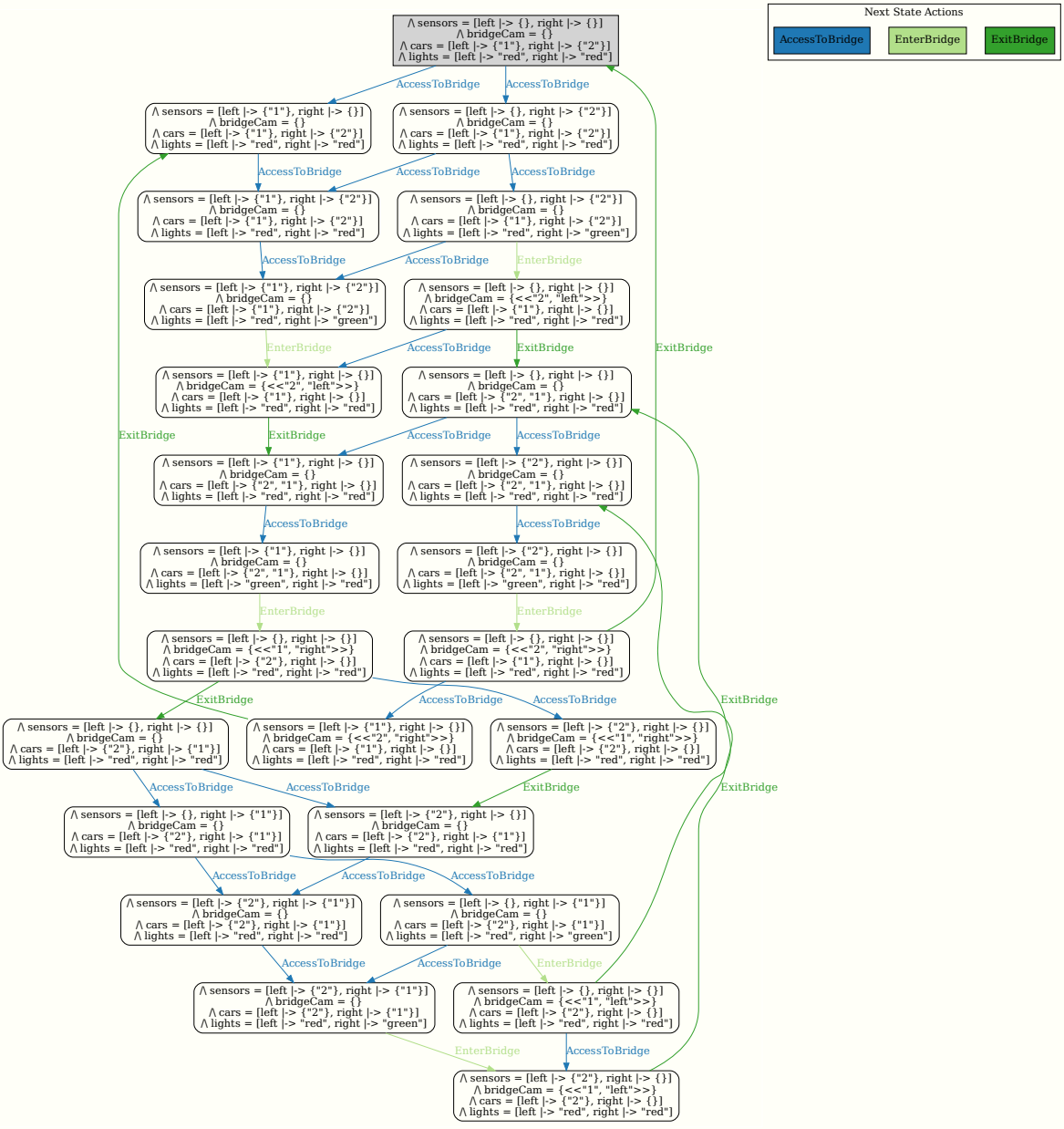


Figura 4: Grafo de estados de `onewaybridge_pluscal.tla` con sólo 2 automóviles: `"1"` en la isla de la izquierda y `"2"` en la de la derecha, con un número máximo de coches en la isla izquierda de 2 coches, y de 1 coche en la de la derecha.

Ejercicio 9

1. Describe en castellano qué hace cada uno de lo procesos `AccessToBridge, EnterBridge, ExitBridge`.

   Genera el grafo de la especificación y explóralo a la vez que lees el código. Hazlo con sólo dos coches inicialmente, uno en cada isla, y con un máximo de 2 coches en la isla de la izquierda y 1 en la de la derecha. Después prueba con diferentes valores de capacidad máxima de las islas y observa los diferentes grafos que se generan. Prueba por ejemplo con un máximo de coches de 2 en ambas islas y observa las diferencias en el grafo.

   Con modelos así de pequeños podrás fácilmente estudiar el grafo, y te ayudará a hacerte una primera idea sobre si la especificación es correcta.

2. Ahora vamos a utilizar la potencia de los verificadores de modelos para comprobar si la especificación es realmente correcta o no, es decir, si satisface o no los requisitos.

   Leyendo los requisitos, crea los nombres de propiedades de tipo *safety* y de tipo *liveness* (y quizá propiedades de acciones) para poder verificar la especificación. No escribas su código, sólo su nombre. Para cada propiedad indica su nombre, si es de tipo *safety* o *liveness* o una propiedad de acciones, y descríbela en castellano.

3. Escribe el código TLA+ para cada una de las propiedades a las que has dado nombre en el apartado anterior.

4. Describe las lecciones aprendidas con la especificación que pueden servir para guiar la implementación.

Ejercicio 10

En el fichero `onewaybridge_pluscal_2.tla` hay una nueva especificación con una implementación alternativa del proceso `EnterBridge` que no satisface los requisitos.

Verifica esta especificación con las propiedades que has escrito en el ejercicio 9 para descubrir los fallos de esta nueva especificación.

1. Para cada fallo explica cómo lo has encontrado, indicando la propiedad que has utilizado para descubrir el fallo.

2. Para cada fallo indica cómo propones corregirlo.

Secuencias o tuplas en TLA+ y módulo `Sequences`

Los elementos de las secuencias o tuplas son indexables por su posición `n`, comenzando por la posición `1` usando la notación `secuencia[n]`:

En el módulo `Sequences` hay operadores para manejar secuencias. Recuerda añadir `EXTENDS Sequences` para poderlos utilizar. Ninguno de los operadores modifica la secuencia o tupla, es decir, son inmutables (como las tuplas de Python). Si quieres por ejemplo eliminar el primer elemento de la secuencia `seq` tienes que reasignar la secuencia: `seq := Tail(seq)`

Estos son los operadores que existen en el módulo `Sequences`:

Operador	**Descripción**	**Ejemplo**
`Head`	Primer elemento	`Head(<<"3", "1", "2">>) = "3"`
`Tail`	Cola (todos menos el primero)	`Tail(<<"1", "2", "3">>) = <<"2", "3">>`
`Append`	Añadir elemento al final	`Append(<<"1", "2",>>, "3") = <<"1", "2", "3">>`
`\o`	Concatenar 2 secuencias	`<<"3">> \o <<"1", "2">> = <<"3", "1", "2">>`
`Len`	Longitud de la secuencia	`Len(<<"1", "2">>) = 2`

Ejercicio 11

Las autoridades isleñas han decidido finalmente permitir que en el puente pueda haber circulando más de un vehículo a la vez como muestra la figura 5.

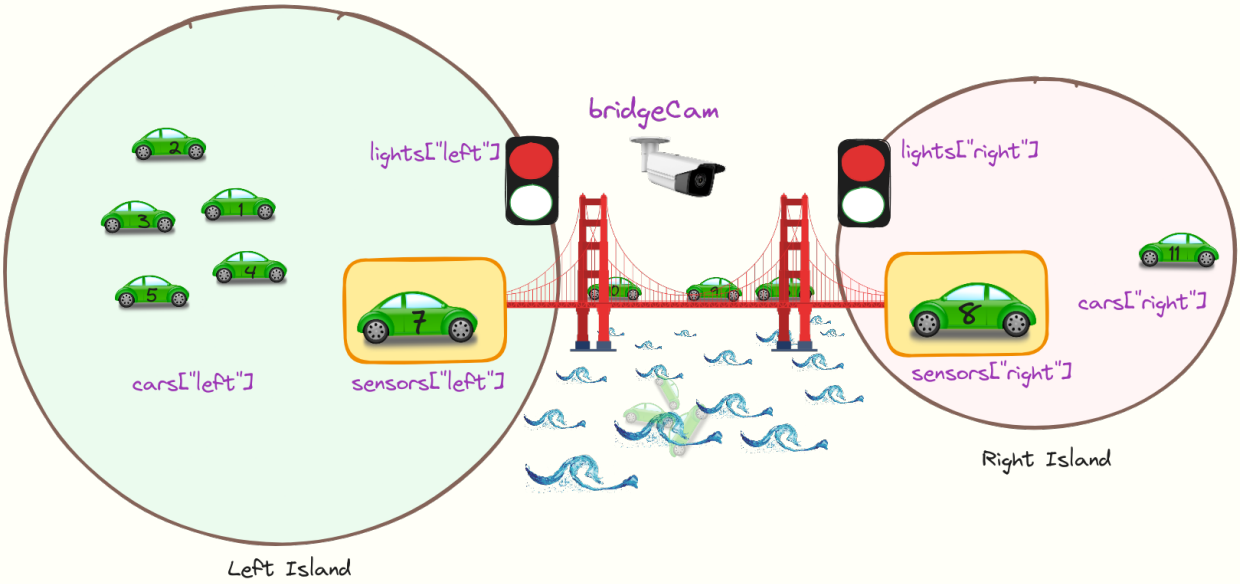


Figura 5: Ahora puede haber hasta un máximo de 3 coches en el puente circulando en el mismo sentido.

En este ejercicio tienes que escribir el código PlusCal de la especificación `onewaybridge_many_pluscal.tla`. Te proporcionamos el fichero, que incluye las variables y constantes, y las propiedades de tipo *safety* y *liveness*, y las propiedades de acciones que debe satisfacer la especificación que tienes que escribir. Estas propiedades se han escrito a partir de los nuevos requisitos que mostramos más abajo.

Aparte de estas propiedades que nosotros te damos, cuando vayas teniendo versiones de la especificación escritas, analiza tu código con TLC creando adrede otras invariantes que sabes que no deben satisfacerse si has hecho bien la versión que tengas en ese momento. Así TLC te mostrará una ejecución que no satisface la invariante y podrás observar si los estados van cambiando tal como esperas. Si por el contrario una de estas invariantes es satisfecha, es que algo has hecho mal. Asegúrate de tener comentadas todas las propiedades salvo la que estás usando para que genere una ejecución.

Nuevos requisitos (en negrita los nuevos requisitos):

- Las islas son pequeñas y cada una de ellas tiene una capacidad máxima de automóviles, que debe regularse a través del sistema de control semafórico: no se puede dejar acceder a un automóvil al puente si la isla destino está llena.
- En cada extremo del puente hay un semáforo que puede estar en

rojo o en verde.

- Los dos semáforos nunca pueden estar en verde a la vez.
- En cada extremo del puente hay un sensor en el asfalto para detectar si hay algún automóvil esperando a cruzar el puente.
- **El semáforo de una isla origen no puede pasar de rojo a verde cuando no hay algún vehículo esperando en el sensor de la isla origen, ni cuando está llena la isla destino ni cuando está lleno el puente.**
- Hay una cámara en el puente que permite saber cuántos automóviles hay en el puente.
- **En el puente puede haber más de un automóvil. Como máximo puede haber 3 automóviles en el puente, y todos ellos tienen que estar circulando en el mismo sentido. Cuando un vehículo entra en el puente porque su semáforo se ha puesto en verde, el semáforo puede seguir en verde y pueden entrar nuevos vehículos.**
- **A partir del momento en el que un coche se sitúe en el sensor para entrar al puente, en el otro sentido no puede ver pasar un pelotón de más de 5 coches seguidos antes de que su semáforo se ponga en verde**[94].
- **Si el semáforo de una isla está en verde seguirá en verde mientras que no haya vehículos esperando en el sensor de la otra isla. Por lo tanto pueden pasar pelotones mayores de 5 siempre en el otro sentido no haya coches esperando.**
- **Si el semáforo de una isla está en verde pero no hay vehículos esperando a pasar desde esa isla, se pondrá en rojo en cuanto llegue un coche al sensor de la otra isla.**
- Mientras que un vehículo transita por el puente el semáforo que regula el acceso a los automóviles que circulan en el sentido contrario ha de permanecer en rojo.

[94] Este tamaño máximo de pelotón ha de ser mayor o igual que el número máximo de coches en el puente. Por ejemplo 5 es mayor que 3. Si no, no podría cumplirse el requisito de que desde que llegue un coche a un sensor no puede ver en el sentido contrario un número mayor de coches que el tamaño máximo de pelotón antes de que su semáforo se ponga en verde: si el puente tenía coches y ese número es mayor que el tamaño máximo de pelotón, el coche que espera en el sensor verá pasar un número de coches mayor que el tamaño máximo de pelotón.

Comentarios sobre el código de `onewaybridge_many_pluscal.tla` En algunas propiedades del código aparece una nueva construcción de TLA+: `LET IN`.

Sirve para introducir en expresiones TLA+ una definición local después de `LET` que sólo puede ser usada en el código TLA+ que hay en la parte `IN`

En el siguiente ejemplo, definimos `other` para poderlo usar en la fórmula de la parte `IN`.

```
LightStaysGreenIfNoCarWantsToCome ==
    [][ \A anIsland \in ISLANDS:
          LET other == IF anIsland = "left" THEN "right" ELSE "left"
          IN
            (   /\ lights[anIsland] = "green"
                /\ sensors[other] = {}
                /\ Len(bridgeCam') + Cardinality(cars'[other]) < MAX_CARS[other]
                /\ Len(bridgeCam') < MAX_CARS_IN_BRIDGE
                /\ sensors'[otherIsland] = {}
            )
            =>  lights'[anIsland] = "green"
    ]_vars
```

Recuerda que `[A]_var` significa lo mismo que `A \/ UNCHANGED var`. Suponiendo que `A` es una acción, es decir una fórmula con variables primadas, `[][A]_var` significa que siempre se cumple la fórmula `A` en todas las transiciones o pasos, o bien no cambian las variables en `var`.

Comentarios sobre cómo modelar el nuevo código para el puente: Ve generando grafos de la especificación según vayas avanzando. Te ayudará a entender qué está haciendo. Además recuerda la técnica de añadir invariantes adrede para que fallen, como técnica para observar ejecuciones. El diagrama del grafo puede ser una técnica complementaria. Pero recuerda, genera un modelo muy pequeño para que sea visualmente posible observar un número de estados pequeño. Prueba por ejemplo con 3 coches y 2 como máximo en el puente.

Verás que hay una variable `last_platoon_length` en el trozo del algoritmo que te mostramos. Esta variable la puedes utilizar para contar cuántos coches seguidos atraviesan el puente en la misma dirección (pelotón = platoon).

Hasta ahora `bridgeCam` era un conjunto que almacenaba una secuencia que contenía en su primer elemento el coche, y en el segundo la isla destino.

Como ahora tendrás que almacenar varios vehículos en el puente, y estos deben estar ordenados para que el que primero entró sea el que primero sale del puente, es mejor que `bridgeCam` no sea

un conjunto, sino una secuencia. Como puedes ver en el fichero `onewaybidge_many_pluscal.tla` que te proporcionamos, la variable `bridgeCam` se inicializa con una secuencia vacía: `bridgeCam = <<>>`. Cada elemento de la secuencia seguirá siendo una secuencia que contenga en su primer elemento el coche, y en el segundo la isla destino.

Utiliza los operadores del módulo `Sequences` para manejar los coches que hay en el puente:

- cuando un coche `"7"` entre al puente en dirección a la isla `"right"` añádelo al final de la secuencia almacenada en `bridgeCam` usando `Append`:

  Suponiendo que `bridgeCam = << <<"2", "right">> >>`

  ```
  bridgeCam := Append(bridgeCam, <<"7", "right">>);
  ```

  dejará en `bridgeCam` la siguiente secuencia:

  ```
  << <<"2", "right"">>, <<"7", "right">> >>
  ```

- cuando quieras saber cuál es el primer coche que entró en el puente usa `Head`:

  Suponiendo que `bridgeCam = << <<"2", "right">>, <<"7", "right">> >>`

  ```
  Head(bridgeCam)
  ```

  devolverá

  ```
  <<"2", "right">>
  ```

  y `Head(bridgeCam)[1]` devolverá `"2"`

- cuando quieras eliminar del puente el primer coche que entró en el puente usa `Tail`:

  Suponiendo que `bridgeCam = << <<"2", "right">>, <<"7", "right">> >>`

  ```
  bridgeCam := Tail(bridgeCam);
  ```

  dejará en `bridgeCam` esta tupla: `<< <<"7", "right">> >>`

Comentarios sobre la verificación con TLC: Si ves que tu ordenador tarda mucho en verificar la especificación con TLC puedes crear modelos más pequeños:

- Usa un número menor de coches en cada isla.
- Usa un número menor de coches en el puente.
- Usa un número menor de coches en el pelotón máximo.

  Pero asegúrate de que el pelotón máximo siempre es mayor o igual que el número máximo de coches en el puente. Para que no puedas equivocarte hay una aserción en el fichero que lo comprueba.

Ejercicios de Conjuntos

Si en alguno de estos ejercicios tienes dudas consulta la sección 1.2 del libro Specifying Systems de Leslie Lamport[95].

[95] https://lamport.azurewebsites.net/tla/book-21-07-04.pdf

Y por supuesto, escribe en el foro de la asignatura para que podamos responderte.

1. Evalúa las siguientes expresiones. Después de haberlo hecho, utiliza un evaluador de expresiones TLA+ en VS Code [96] para comprobar si has acertado[97]:

   a) `{<<1>>, <<2,2>>, <<3,3,3>>} \cup {<<1>>, <<1,2>>, <<1,2,3>>}`

   b) `{{"a","b"}, {"a"}, {"b"}} \cap {{"a","b","c"}, {"a","b"}, {"a"}}`

   c) `{"aa", "bb", "cc"} \ {"a", "b", "c"}`

   d) `{1,1,2,2,3,3} \ {2, 3, 4}`

   e) `{1,1,1} \subseteq {1,1,2,2}`

   f) `{1,2,3} \X {"a","b","c"}`

[96] Presiona las teclas `Ctrl-Shift-P` y escribe `TLA+: Run REPL in Terminal`.

[97] Recuerda que en TLA+:
- `\cup` es el operador infijo para la unión de conjuntos ∪ y lo puedes escribir también como `\union`.
- `\cap` es el operador infijo para la intersección de conjuntos ∩ y lo puedes escribir también como `\intersect`.
- El operador para la diferencia de conjuntos es `\`.
- `\subseteq` es el operador infijo para la inclusión de conjuntos ⊆.
- `S \X T` es el producto cartesiano de conjuntos: un conjunto de tuplas cuyo primer elemento pertenece a `X` y su segundo elemento pertenece a `T`.

Ejercicios de Lógica

Si en alguno de estos ejercicios tienes dudas consulta la sección 1.1 del libro Specifying Systems de Leslie Lamport[98].

[98] https://lamport.azurewebsites.net/tla/book-21-07-04.pdf

Y por supuesto, escribe en el foro de la asignatura para que podamos responderte.

1. Evalúa las siguientes expresiones y compruébalas en el evaluador de expresiones de TLA+ de VS Code[99] para comprobar si has acertado.

   a) `~~TRUE`

   b)
   ```
   1=1 \/ 2=2
   1=1 \/ 1=2
   1=3 \/ 1=1
   1=2 \/ 3=1
   ```

   c)
   ```
   1 \in {1,2,3} \/ 2 \in {1,2,3}
   1 \in {1,2,3} \/ 4 \in {1,2,3}
   4 \in {1,2,3} \/ 1 \in {1,2,3}
   4 \in {1,2,3} \/ 5 \in {1,2,3}
   ```

   d)
   ```
   1=1 /\ 2=2
   1=1 /\ 1=2
   1=3 /\ 1=1
   1=2 /\ 3=1
   ```

   e)
   ```
   1 \in {1,2,3} /\ 2 \in {1,2,3}
   1 \in {1,2,3} /\ 4 \in {1,2,3}
   4 \in {1,2,3} /\ 1 \in {1,2,3}
   4 \in {1,2,3} /\ 5 \in {1,2,3}
   ```

[99] Presiona las teclas `Ctrl-Shift-P` y escribe `TLA+: Run REPL in Terminal`.

f)
```
1=1 => 2=2
1=1 => 1=2
1=3 => 1=1
1=2 => 3=1
```

g)
```
1 \in {1,2,3} => 1 < 4
1 \in {1,2,3} => 1 = 2
0 \in {1,2,3} => 1 = 1
0 \in {1,2,3} => 1 /= 1
```

h)
```
~ 1=2 \/ 3=1
~ (1=2 \/ 3=1)
~ 1=1 \/ 3=1
~ (1=1 \/ 3=1)
```

i)
```
~ 1=2 /\ 3=1
~ (1=2 /\ 3=1)
~ 1=1 /\ 3=1
~ (1=1 /\ 3=1)
```

2. Rellena la columna que está vacía:

P	Q	R	(P /\ Q) \/ ~R	(P /\ Q) \/ ~P	P => (Q /\ P)	~P => ~(Q \/ R)	P /\ (~Q) /\ R
FALSE	FALSE	FALSE					
FALSE	FALSE	TRUE					
FALSE	TRUE	FALSE					
FALSE	TRUE	TRUE					
TRUE	FALSE	FALSE					
TRUE	FALSE	TRUE					
TRUE	TRUE	FALSE					
TRUE	TRUE	TRUE					

3. Rellena la columna que está vacía:

P	Q	R	(~P) /\ Q /\ (~R)	(P /\ (~Q) /\ R) \/ ((~P) /\ Q /\ (~R))
FALSE	FALSE	FALSE		
FALSE	FALSE	TRUE		
FALSE	TRUE	FALSE		
FALSE	TRUE	TRUE		
TRUE	FALSE	FALSE		
TRUE	FALSE	TRUE		
TRUE	TRUE	FALSE		
TRUE	TRUE	TRUE		

4. ¿Cuáles de las siguientes fórmulas son equivalentes?

a) `~(P \/ Q)`

b) `~(P /\ Q)`

c) `(~P \/ ~Q)`

d) `(~P /\ ~Q)`

5. Supongamos que `P(x,y)` es la fórmula `x+y=4` y que `Q(x,y)` es la fórmula `x < y`.

   ¿Cuál es la tabla de verdad de la fórmulas suponiendo los valores que aparecen en la primera columna?:

	`P(x,y) /\ Q(x,y)`	`P(x,y) \/ Q(x,y)`	`P(x,y) => Q(x,y)`	`(P(x,y) <=> Q(x,y))`
`x=1, y=3`				
`x=3, y=1`				
`x=1, y=2`				
`x=2, y=1`				

Ejercicios con los cuantificadores existencial y universal

Si en alguno de estos ejercicios tienes dudas consulta la sección 1.3 del libro Specifying Systems de Leslie Lamport[100].

[100] https://lamport.azurewebsites.net/tla/book-21-07-04.pdf

Y por supuesto, escribe en el foro de la asignatura para que podamos responderte.

1. Evalúa las siguientes expresiones:

   a) `\A n \in {1,2,3}: n < 4`

   b) `\E n \in {1,2,3}: n < 4`

   c) `\A n \in {1,2,3}: \E m \in {1,4,6}: m = n * n`

   d) `\E m \in {1,4,6}: ~\E n \in {1,2,3}: m = n * n`

   e) `\A n \in {1,2,3}: \E m \in {1,4,9}: m = n * n`

   f) `\A n \in {1,2,3}: \A m \in {1,4,6}: m + n < 10`

   g) `\A n \in {1,2,3}, m \in {1,4,6}: m + n < 10`

   h) `\A n \in {1,2,3}, m \in {4,7,8}: x < y => x = y`

2. Escribe en castellano cada una de estas fórmulas, y luego reescríbela por una equivalente en la que se cambie el tipo de cuantificador (existencial / universal) que se usa, y cambiando el uso que se hace de la negación. Finalmente, escribe en castellano la nueva fórmula.

   a) `~ \A x > 0: x*x > x`

   b) `~ \E x \in {0,1}: x*x + x < 0`

   c) `~ \A x, y \in {1,2,3}: (x*y = y**2 => x = y)`

   d) `~ \E x, y \in {1,2,3}: (x > y /\ y > x)`

   e) `\A n \in {1,2,3} \intersect {10,4,6}: n = 42`

   f) `\A n \in {}: FALSE`

   g) `\E n \in {1,2,3} \intersect {10,4,6}: 1 = 1`

   h) `\E n \in {}: TRUE`

3. Escribe una fórmula TLA+ para estas afirmaciones y evalúala:

   a) Cualquier número del conjunto `{1,2,3}` elevado al cuadrado es mayor de cero.

   b) Algún número del conjunto `{-1,2,3}` elevado al cubo es menor de cero.

   c) El doble de un número del conjunto `{1,2,3}` es siempre mayor que el propio número.

4. Dados los conjuntos `{1,2,3}` y `{4,5,6}` escribe úna fórmula con el cuantificador universal y una con el cuantificador existencial para expresar cada una de estas afirmaciones:

   a) Todo elemento del primer conjunto está en el segundo conjunto.

   b) Algún elemento del primer conjunto está en el segundo conjunto.

   c) Algún elemento del primer conjunto no está en el segundo conjunto.

   d) Ningún elemento del primer conjunto está en el segundo conjunto.

5. ¿Cuáles de las siguientes fórmulas son equivalentes?

   a) `\A m \in {1,2,3}: m > 0`

   b) `~\E m \in {1,2,3}: ~ m > 0`

   c) `~\A m \in {1,2,3}: m > 0`

   d) `\E m \in {1,2,3}: ~ m > 0`

6. Evalúa estas fórmulas (`TRUE` o `FALSE`):

   a) `\A x,y,z \in {0,1,2,3}:  (x*z = y*z) =>  x=y`

   b) `\A x,y,z \in {1,2,3}:  (x*z = y*z) =>  x=y`

   c) `\E x \in {0,1,2,3}: \A y \in {1,2,3}: x + y = y`

   d) `\E x \in {1,2,3}: \A y \in {1,2,3}: x + y = y`

   e) `\E x \in {0,1,2,3}: \A y \in {1,2,3}: x + y = 0`

   f) `\E x \in {0,1,2,3}: \A y \in {1,2,3}: x * y = 0`

Verdades y falsedades por vacuidad

En los ejercicios hay 4 expresiones que te habrán sorprendido. Las analizamos en esta sección por la importancia que tienen, especialmente cuando escribas propiedades en las que por equivocación introduzcas expresiones similares.

Verdad por vacuidad

Las dos siguientes expresiones son verdad por vacuidad[101]: son verdad a pesar de que aparentemente no deberían serlo. Ambas expresiones se evalúan a `TRUE` como habrás comprobado evaluándolas con TLC:

[101] Las verdades por vacuidad y las falsedades por vacuidad tienen cierto parecido con las competiciones en las que un contrincante gana por incomparecencia del otro.

- `\A n \in {1,2,3} \intersect {10,4,6}: n = 42`
- `\A n \in {}: FALSE`

La razón es que estas expresiones tienen este significado en lógica:

```
\A n: n \in {1,2,3} \intersect {10,4,6} => n = 42
```

```
\A n: n \in {} => FALSE
```

que podemos leer respectivamente como:

> Para todo `n` en el conjunto universo, si `n` pertenece al conjunto `{}` entonces `n = 42`

> Para todo `n` en el conjunto universo, si `n` pertenece al conjunto `{}` entonces `FALSE`

Recuerda que `A => B` es equivalente a `~A \/ B`. Si `A` es `FALSE`, da igual lo que sea `B`, pues `~A \/ B = TRUE \/ B`, que es `TRUE` independientemente de lo que sea `B`, y por tanto `A => B` es `TRUE` cuando `A = FALSE`.

Por ello, en la primera fórmula, a pesar de que `n = 42` es FALSE, la fórmula es TRUE. Y en el segundo caso, a pesar de que el consecuente `FALSE` es evidentemente `FALSE`, la fórmula es `TRUE`.

A este tipo de fórmula se le denomina verdad vacua. Si escribes una propiedad con una verdad vacua, TLC la verificará siempre, dándote una falsa sensación de que la especificación cumple una propiedad que, en realidad, es una verdad vacua. Lo que ocurre es que la propiedad no tiene sentido.

Falsedad por vacuidad

Algo similar ocurre con estas otras dos expresiones. Ambas se evalúan a `FALSE` por extraño que pueda parecer:

- `\E n \in {1,2,3} \intersect {10,4,6}: 1 = 1`
- `\E n \in {}: TRUE`

La razón es que estas fórmulas tienen respectivamente este si significado en lógica:

```
\E n: n \in {} /\ 1 = 1
\E n: n \in {} /\ TRUE
```

que podemos leer en ambos casos como Existe `n` en el conjunto universo tal que `n` pertenece al conjunto `{}` y `TRUE`.

Al ser una conjunción (`/\`), esta expresión es siempre `FALSE` porque `n` no pertenece al conjunto `{}` y por tanto es `FALSE`. La conjunción de `FALSE /\ 1 = 1` en la primera expresión es `FALSE`, y la conjunción `FALSE /\ TRUE` en la segunda es también `FALSE`.

Se trata de falsedades por vacuidad. Si escribes una propiedad con una falsedad por vacuidad, TLC siempre detectará que se viola la propiedad, dándote una falsa sensación de que la especificación es incorrecta, cuando en realidad es que la propiedad no tiene sentido.

Operadores con parámetros y operadores recursivos

Hasta ahora hemos escrito definiciones para definir algo parecido a las constantes, si bien ahora ya utilizamos verdaderas constantes a las que damos un valor en el fichero `.cfg`. También hemos utilizado definiciones para crear fórmulas[102], pero todas las fórmulas que hemos creado hasta ahora no tenían parámetros. Una vez escrita la definición, cuando aparece el nombre que aparece a la izquierda de la definición se sustituye por la evaluación de la expresión que aparece a la derecha de la definición.

[102] Recuerda, una fórmula es una expresión que cuando se evalúa devuelve un valor `TRUE` o `FALSE`.

Las definiciones sin embargo pueden tener parámetros. En TLA+ llamamos a estas definiciones, tengan o no parámetros, operadores.

Cuando un operador tiene parámetros, sólo se puede llamar al operador pasándole argumentos o parámetros actuales para cada uno de los parámetros formales de la definición. El uso es como el de una función en un lenguaje de programación. La llamada al operador se sustituye por la expresión que aparece a la derecha en su definición, sustituyendo en ella los parámetros formales de la definición por los parámetros actuales que se han pasado en la llamada al operador.

En la figura 6 se definen los operadores `otherIsland(anIsland)`, `Abs(n)` y `Seconds(hours)`. Fíjate en las expresiones que definen estos operadores: su evaluación no devuelve un valor `TRUE` o `FALSE`, por lo que estos operadores no definen fórmulas.

También se pueden definir operadores recursivos. Para ello hay que hacer una declaración previa de la recursividad del operador. En la figura 6 se define el operador recursivo `Factorial(n)`, para lo cuál antes hay que declarar que es recursivo con la palabra reservada RECURSIVE, poniendo en el lugar del parámetro formal del operador el carácter _ *underscore*. El operador `Factorial(n)` no es una fórmula porque su evaluación no devuelve un valor `TRUE` o `FALSE`.

Comprueba el uso de los operadores verificando con TLC la especificación de la figura 6, que mostrará en la salida el resultado de llamar a cada uno de los 3 operadores:

operatorsExample.tla

```
---- MODULE operatorsExample ----
EXTENDS TLC, Integers



(*--algorithm operatorsExample {
define{
    Islands == {"left", "right"}

    otherIsland(anIsland) == IF anIsland = "left" THEN "right" ELSE "left"

    Abs(x) == IF x > 0 THEN x ELSE -x

    Seconds(hours) == hours * 60 * 60

    RECURSIVE Factorial(_)
    Factorial(n) == IF n = 0 THEN 1
                    ELSE n * Factorial(n-1)
}

{
    print otherIsland("left");
    print otherIsland("right");

    print Abs(-3);

    print Seconds(3);

    print Factorial(5)
}

}
*)
```

Figura 6: Ejemplos de definición y uso de operadores

Protocolo de encaminamiento basado en vectores de distancias

En esta sección estudiaremos la especificación de un algoritmo distribuido de encaminamiento, también denominado protocolo de encaminamiento dinámico[103].

Hay un conjunto de ordenadores interconectados mediante canales de comunicación. Podemos representar con un grafo la topología de los ordenadores y sus posibilidades de comunicación física directa con otros ordenadores.

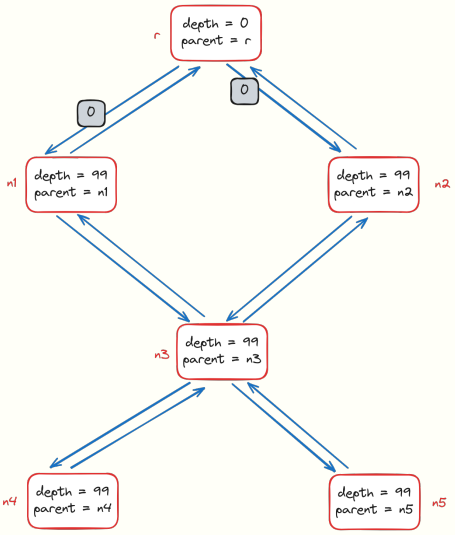
En la figura[104] vemos por ejemplo cómo el ordenador `n1` puede enviar directamente mensajes a los nodos `r, n3`. Decimos entonces que `r` y `n3` son vecinos de `n1`. La comunicación es *full duplex*: si `n1` puede enviar un mensaje a un vecino como `r`, `r` puede también enviarle un mensaje a `n1`.

Vemos en la figura que `n4` sin embargo no está conectado directamente con `r` (no son nodos vecinos). Para poderle enviar mensajes a `r`, `n4` tiene antes que descubrir una ruta que le permita encaminar mensajes destinados a `r` a través de algún nodo vecino, y que este a su vez haya descubierto cómo encaminar mensajes hacia `r`.

Los protocolos de encaminamiento basados en vectores de distancias como *RIP*[105] se han utilizado en Internet para este propósito. Presentamos ahora informalmente un algoritmo de este tipo, centrándonos sólo en el descubrimiento de rutas que les permitan a los nodos con nombres que empiezan por `n` encaminar mensajes destinados al nodo raíz `r`[106].

- Cada nodo mantiene 2 variables:
  - `depth` (profundidad) almacena la mejor distancia conocida medida en número de saltos intermedios hacia la raíz que conoce ese nodo. El nodo raíz `r` tiene el valor 0 (cero), pues él es el propio destino. Los demás nodos inicializan `depth` con un valor infinito (99) pues inicialmente no conocen una ruta hacia `r`. Utilizaremos indistintamente los términos `depth` y `distancia`.
  - `parent` almacena el nodo vecino a través del cuál tiene que encaminar los menajes destinados a la raíz `r`. Todos los nodos inicializan con su propio identificador la variable `parent`. En el caso de `r`, porque él es el destino y no importa mucho qué valor se utilice, y en el caso del resto de nodos, porque no conocen ningún nodo vecino a través del cuál encaminar los mensajes destinados a `r`, y por ello utilizan su propio identificador como valor de `parent` para indicar que no conocen ruta hacia el destino `r`.
- Inicialmente el nodo raíz `r` envía un mensaje del protocolo que incluye su distancia o profundidad conocida[107] respecto al des-

[103] La especificación puede encontrarse en el Tutorial sobre PlusCal de Leslie Lamport, en la sección 10: `https://lamport.azurewebsites.net/tla/tutorial/session10.html`. El código que mostramos es exactamente el mismo. La explicación que realizamos en esta sección está muy resumida, y utiliza un orden expositivo distinto. Es recomendable, pero no necesario, que el lector vaya a leer la fuente original de Lamport para despejar dudas que puedan surgir durante la lectura.



[104]

[105] `https://en.wikipedia.org/wiki/Routing_Information_Protocol`

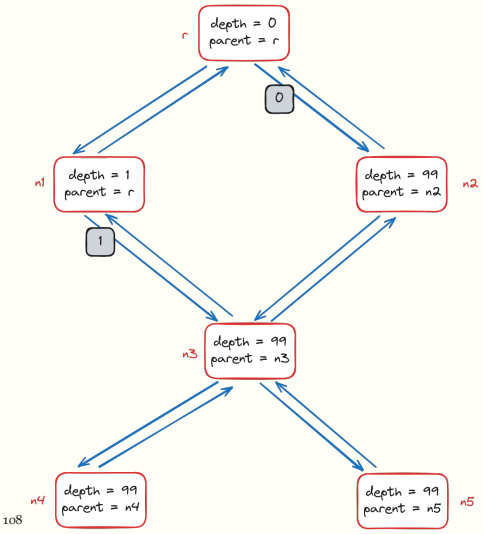
[106] Si quisiéramos descubrir rutas hacia el resto de nodos también, ejecutaríamos este algoritmo múltiples veces, fijando como nodo raíz cada uno de los posibles nodos destino.

[107] Una vez que el algoritmo ha terminado, la información de encaminamiento descubierta por cada nodo conforma un árbol con raíz en `r`, razón por la cuál denominamos profundidad a la distancia de cada nodo respecto a la raíz del árbol, medida en número de saltos intermedios que les separan de `r`.

tino (r), que es evidentemente 0. Este mensaje con un 0 es enviado por r a través de sus dos enlaces, con destinos n1 y n2.

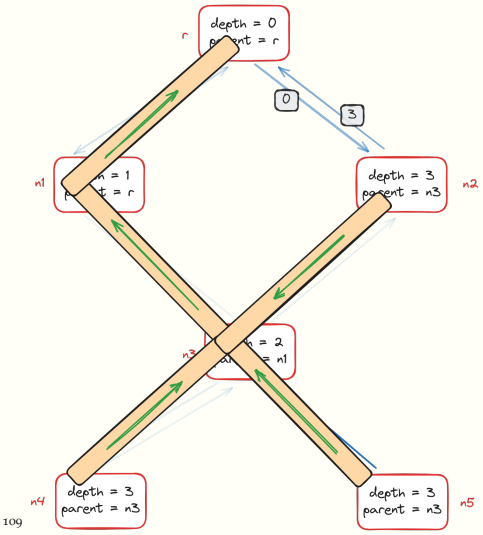
- Cuando un nodo n recibe un mensaje del protocolo, procedente de un nodo vecino v en el que se incluye la distancia de v hasta la raíz, el nodo n puede evaluar si una ruta con destino a r a través de v es mejor que la que conoce n hasta ese momento:
  - Si es mejor, n actualiza los valores de sus variables:
    - depth pasa a ser 1 + la distancia que viene en el mensaje de v.
    - parent pasa a ser v.
    - n envía un mensaje del protocolo con su distancia, a todos los vecinos salvo a v.
  - Si no es mejor, n descarta el mensaje recibido, y no envía ningún mensaje a sus vecinos.

En la figura[108] vemos cómo en n1, tras llegar el mensaje del protocolo procedente de r con valor 0, se han actualizado sus variables, y n1 entonces envía el nuevo mensaje del protocolo a todos sus vecinos salvo a r.



108

En la figura [109] se ve el árbol de expansión[110] que ha quedado configurado en las variables parent después de que el mensaje que envió r a n1 haya provocado que n1, n2, n3, n4 y n5 hayan aprendido nuevas rutas hacia r. Cuando el mensaje del protocolo con valor 3 que envía n2 llegue a r, r lo descartará porque no aporta una mejor ruta que la ya conocida por r de distancia 0.



109

[110] Árbol de expansión o recubridor o generador: https://es.wikipedia.org/wiki/%C3%81rbol_de_expansi%C3%B3n

Para ver este ejemplo extendido en mayor detalle puedes consultar el siguiente vídeo en un navegador Chrome: http://mobiquo.gsyc.es/tlapluscal/distanceVector.mp4.

Constantes

En la línea 3 de la especificación pathsToRoot_1.tla que se muestra en la siguiente figura se definen como constantes el conjunto de nodos Nodes, el nodo raíz root, el conjunto de arcos del grafo Edges, siendo cada arco un conjunto formado por dos nodos vecinos, y el número máximo de nodos MaxNodes. En el fichero pathsToRoot_1.cfg que mostramos en la página siguiente se inicializan las constantes con valores que se corresponden con la topología del ejemplo.

Procesos y variables

- Cada nodo se modela con una instancia del proceso node como puedes ver en la página siguiente, en la línea 10 del fichero

pathsToRoot_1.tla

```
---- MODULE pathsToRoot_1 ----
EXTENDS Integers, Sequences, FiniteSets, TLC

CONSTANTS Nodes, Edges, root, MaxNodes

Nbrs(n) == {m \in Nodes : {m,n} \in Edges}

QueuesFrom(n) == {<<n, m>> : m \in Nbrs(n)}
QueuesTo(n)  == {<<m, n>> : m \in Nbrs(n)}
Queues == UNION {QueuesFrom(n) : n \in Nodes}

(* --algorithm PathsToRoot {
    variable msgs = [q \in Queues |-> IF q[1] = root THEN <<0>> ELSE << >>] ;

    fair process (node \in Nodes)
      variables depth = IF self = root THEN 0 ELSE MaxNodes,
                parent = self ;
      { a: while (TRUE) {
             with (q \in {r \in QueuesTo(self) : msgs[r] /= << >>}) {
               if (Head(msgs[q] + 1) < depth) {
                  depth := Head(msgs[q]) + 1 ;
                  parent := q[1] ;
                  msgs := [r \in Queues |->
                            IF r = q
                              THEN Tail(msgs[q])
                              ELSE IF r \in QueuesFrom(self) \ {<<self, q[1]>>}
                                     THEN Append(msgs[r], depth)
                                     ELSE msgs[r] ]

               } else { msgs[q] := Tail(msgs[q]) }
             }
           }
      }
}
*)
====
```

pathsToRoot_1.cfg

```
SPECIFICATION
    Spec
CONSTANTS
    Nodes = {r, n1, n2, n3, n4, n5}
    root = r
    Edges = {{r,n1},{r,n2},{n1,n3},{n2,n3},{n3,n4},{n3,n5}}
    MaxNodes = 99
```

`pathsToRoot_1.tla`. Por ello cada nodo tiene 2 variables, `depth` y `parent`, declaradas e inicializadas en las líneas 10 y 11.

- La red la modelamos con 2 colas de mensajes entre cada pareja de nodos vecinos, para poder enviar mensajes en ambos sentidos. Desde la perspectiva de un nodo, una cola para mensajes salientes hacia un vecino y otra para mensajes entrantes de ese vecino. Cuando un nodo envía un mensaje a un vecino, lo añadirá a la cola saliente correspondiente. Cuando un nodo recibe un mensaje lo extrae de la cola entrante correspondiente.

  Nombramos cada cola con una tupla en la que el primer elemento es el nodo origen y el segundo el nodo destino. Para la topología del ejemplo mostramos los nombres de las colas en la figura de la derecha [111].

  En la especificación definimos las colas para mensajes salientes y para mensajes entrantes mediante 2 operadores, respectivamente `QueuesFrom(n)` y `QueuesTo(n)` (líneas 5 y 6).

  Cada uno de estos operadores define un conjunto de nombres de colas (recuerda, cada nombre de una cola es una secuencia o tupla con la pareja ordenada de los nombres del nodo origen primero, y del nodo destino después), utilizando la definición del operador `Nbrs(n)`, que es el conjunto de los identificadores de los nodos vecinos del nodo `n` (línea 4).

  La definición de `Queues` en la línea 7 sirve para tener todas las colas bajo un mismo nombre. Se podría haber definido igualmente así:

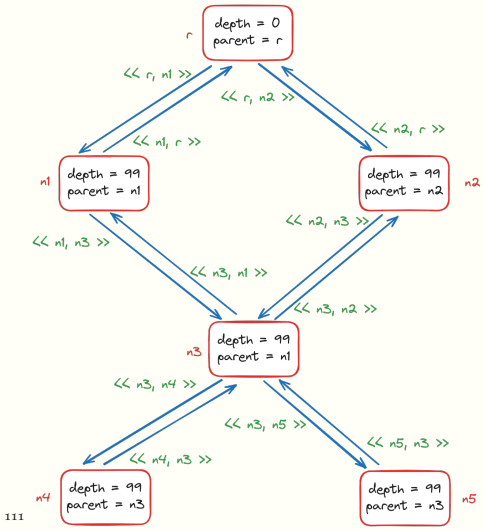
  ```
  Queues == UNION {QueuesFrom(n) : n \in Nodes}
  ```

  La expresión `{x * 2: x \in {1,2,3}}` es el conjunto de los elementos del conjunto `{1,2,3}` multiplicados por 2, y se evalúa por tanto a `{2,4,6}`[112]. Del mismo modo, `{QueuesFrom(n) : n \in Nodes}` es el conjunto formado por todos los nombres de las colas que salen de cada elemento de `Nodes`, que es igual que el conjunto de todos los nombres de las colas que llegan a cada elemento de `Nodes`.

  `UNION`[113] es un operador definido en el módulo `FiniteSets`. Siendo `S` un conjunto de conjuntos, `UNION S` [114] es el conjunto formado por la union de los conjuntos que pertenecen a `S`.

  Por lo tanto `Queues` es el conjunto de los nombres de todas las colas entrantes de todos los nodos, que es igual al conjunto de los nombres de todas las colas salientes de todos los nodos.

- `Queues`, `QueuesFrom` y `QueuesTo` son conjuntos de nombres de colas. Las colas de mensajes propiamente dichas se definen en la variable `msgs`, una función TLA+[115] que asigna a cada nombre de cola `<< m, n >>` (siendo `m` y `n` nombres de nodos) una secuencia o tupla que contendrá los mensajes del protocolo que están



111

[112] Si programas en Python esta notación te resultará familiar. Las *List comprehensions* tienen una sintaxis similar:
`[x*2 for x in range (1,4)]`
https://docs.python.org/2/tutorial/datastructures.html#list-comprehensions.

[113] $\bigcup$

[114] $\bigcup S$

[115] Recuerda que una función TLA+ se utiliza como un diccionario de Python

circulando en el canal que va desde `m` hacia `n`. Todas las colas están inicialmente vacías, salvo las colas que salen de `r` («`r,n1`» y «`r,n2`»), que contienen el primer mensaje con valor 0 que envía la raíz inicialmente a sus dos vecinos.

- El algoritmo no requiere explicaciones extensas. Si hay un mensaje en alguna cola entrante se procesa.

  Fíjate en la línea 14 que reproducimos aquí:

  ```
  with (q \in {r \in QueuesTo(self) : msgs[r] /= << >>}) {
  ```

  `{r \in QueuesTo(self) : msgs[r] /= << >>}` es el conjunto de los nombres de colas entrantes en `self` (`QueuesTo(self)`) en las que hay mensajes.

  Si hay un mensaje en una cola entrante (línea 14), o bien es una mejor ruta (líneas 16-23), o no lo es y en este caso se borra en la línea 24. En caso de que sí se haya aprendido una mejor ruta, es decir nueva distancia a la raíz mejor que la conocida, se actualizan las variables `depth` en la línea 16 y `parent` en la línea 17. En la asignación a la variable `parent` se utiliza el identificador `q[1]` como el identificador del nodo vecino del que se ha aprendido una nueva ruta: en una implementación real del algoritmo posiblemente este identificador del nodo que nos envía el mensaje formaría parte de la cabecera del mensaje. Pero en esta especificación este es un detalle que no nos interesa modelar, por lo que usamos el nombre de la cola para ello.

  El envío del nuevo mensaje informando a los vecinos se modela con la asignación a la variable `msgs` en la línea 18. Con una única asignación se elimina el mensaje entrante (línea 20), y "se envía"por todas las colas salientes (línea 22), salvo por la cola saliente hacia el nodo que nos envió el mensaje entrante (línea 23).

  ¿Por qué hacemos esto de modificar todas las colas involucradas en una única sentencia? La razón es que no podemos asignar más de una vez un valor a una variable dentro de una misma etiqueta. El algoritmo sólo utiliza la etiqueta `a:` en la línea 13. Si intentásemos dividir la asignación a `msgs` en varias sentencias tendríamos que introducir nuevas etiquetas. Esto haría que se generasen más estados y por tanto que fuese más lenta la verificación con TLC para un mismo modelo (mismo número de nodos y topología). Además, hay una **restricción en PlusCal: dentro de `with` no podemos añadir etiquetas**. Por lo tanto, para añadir una nueva etiqueta tendríamos que sacar fuera del `with` el valor de la cola entrante, para poder asignar a `msgs` fuera de `with` los múltiples valores, en etiquetas distintas. Dejamos como ejercicio esta alternativa, que sin embargo no sería una mejora por las razones antes expuestas.

Verificación con TLC

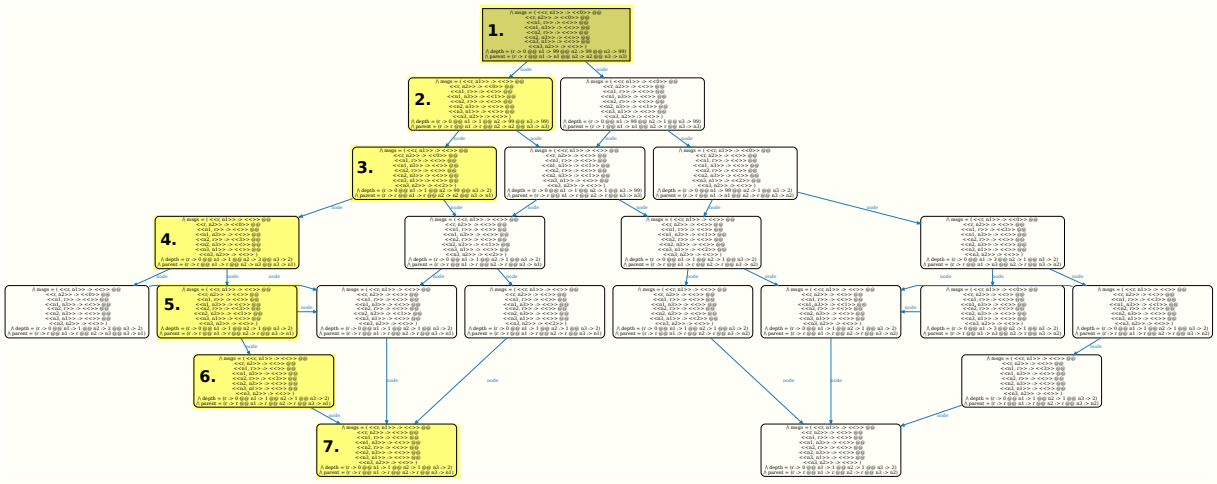
Ejecuta TLC y abre el fichero `output.dot` para que puedas inspeccionar el grafo de estados.

En la siguiente figura hemos ejecutado TLC sobre la misma topología pero eliminando los nodos `n4` y `n5` y sus correspondientes arcos, para lo cuál hemos dado los siguientes valores a las constantes:

```
Nodes = {r, n1, n2, n3}
root = r
Edges = {{r,n1},{r,n2},{n1,n3},{n2,n3}}
```

De este modo tenemos menos estados y podemos ver mejor el diagrama. Como ejemplo, hemos marcado en amarillo la secuencia de estados de una ejecución:

1. Estado inicial en el que `r` ha enviado a n1 y a n2 el mensaje 0
2. n1 recibe el mensaje 0 de r y aprende una mejor ruta a través de 4 de coste 1; n1 envía el mensaje 1 a n3
3. n3 recibe el mensaje 1 de n1 y aprende una mejor ruta a través de n1 de coste 2; n3 envía el mensaje 2 a n2
4. n2 recibe el mensaje 2 de n3 y aprende una mejor ruta a través de n3 de coste 3; n2 y envía el mensaje 3 a r
5. n2 recibe el mensaje 0 de r y aprende una mejor ruta a través de r de coste 1; n2 envía el mensaje 1 a n3
6. n3 recibe el mensaje 1 de n2 y NO aprende una mejor ruta; n3 NO envía mensajes.
7. r recibe el mensaje 3 de n2 y NO aprende una mejor ruta; r NO envía mensajes.



Propiedades y aserciones (encaminamiento basado en vectores de distancias)

En la figura de la página siguiente mostramos una nueva versión de la especificación a la que se han añadido propiedades y aserciones.

Propiedades de tipo safety*: `TypeOK`*

En las líneas 24-26 se define la propiedad de tipo *safety* `TypeOK`.

```
TypeOK  ==  /\ depth  \in [Nodes -> 0..MaxNodes]
            /\ parent \in [Nodes -> Nodes]
            /\ msgs   \in [Queues -> Seq(0..(Cardinality(Nodes)-1))]
```

Cuando `TypeOK` se use como una invariante en el fichero de TLC `pathsToRoot_2.cfg`, verificará que en todos los estados los valores almacenados en `depth, parent` y `msgs` son los adecuados en cuanto al tipo de los datos.

- En la primera expresión de la conjunción [116] se comprueba que la variable `depth` de cada nodo siempre tiene valores comprendidos entre 0 y `MaxNodes`. En el algoritmo PlusCal `depth` es una variable local de cada proceso `node`, por lo que cada uno tiene su propio valor. El traductor de PlusCal a TLA+ genera una función TLA+ (como un diccionario) que asigna el valor de `depth` a cada nodo. Por eso escribimos `TypeOK` teniendo en cuenta cómo se habrá traducido la variable PlusCal `depth`.

  `[Nodes -> 0..MaxNodes]` es el conjunto de las funciones que tienen como conjunto dominio el conjunto `Nodes` y como conjunto imagen los valores comprendidos entre 0 y `MaxNodes`[117].

  No es casualidad que a este tipo de fórmulas cuyo significado es un predicado sobre estados, por convenio, las llamemos siempre `TypeOK`, pues es la forma de construirnos un sistema de tipos en TLA+, que es un lenguaje no tipado.

- Del mismo modo, la segunda expresión de la conjunción[118] se asegura de que los valores de `parent` para cada nodo siempre son elementos del conjunto `Nodes`.

- Finalmente, la tercera expresión de la conjunción[119] se asegura de que la variable global `msgs` siempre es un conjunto de funciones cuyo conjunto dominio son nombres de colas (elementos de `Queues`[120]) y su conjunto imagen son elementos del conjunto `Seq(0..(Cardinality(Nodes)-1))`[121], que es el conjunto de las secuencias de elementos del conjunto de números `0..(Cardinality(Nodes)-1)`. Tiene sentido pues recuerda que los mensajes son simplemente los números de las profundidades [122] que envía cada nodo, por lo que la cola de mensajes está formada por secuencias o tuplas de números.

[116] `/\ depth \in [Nodes ->0..MaxNodes]`

[117] Fíjate en la flecha `->`. La expresión `[S -> T]` siendo `S` y `T` conjuntos, es el conjunto de las funciones que tienen en su dominio un valor de `S` y en su imagen un valor de `T`. Teniendo en cuenta la analogía con los diccionarios de Python, podemos interpretar la expresión `depth \in [Nodes -> 0..MaxNodes]` como una afirmación de que `depth` contendrá siempre un diccionario con claves que son valores del conjunto `Nodes` y valores asociados a los nodos que son elementos del conjunto `0..MaxNodes`.

[118] `/\ parent \in [Nodes ->Nodes]`

[119] `/\ msgs \in [Queues ->Seq(0..(Cardinality(Nodes)-1))]`

[120] Recuerda que `Queues` es el conjunto de los nombres de las colas, nombres como « `r, n1` »

[121] `Seq(S)` es un operador del módulo `Sequences` que devuelve el conjunto de todas las secuencias cuyos elementos son elementos del conjunto `S`.

[122] Sería un buen título para una novela: *Números de las profundidades*.

pathsToRoot_2.tla

```
---- MODULE pathsToRoot_2 ----
EXTENDS Integers, Sequences, FiniteSets, TLC

CONSTANTS Nodes, Edges, root, MaxNodes

HasTwoElts(S) == \E a,b \in S : (a # b) /\ (S = {a,b})

ASSUME /\ \A e \in Edges : (e \subseteq Nodes) /\ HasTwoElts(e)
       /\ root \in Nodes
       /\ MaxNodes >= Cardinality(Nodes)

Nbrs(n) == {m \in Nodes : {m,n} \in Edges}

SetNbrs(S) == UNION {Nbrs(n) : n \in S}

RECURSIVE ReachableFrom(_)
ReachableFrom(S) == LET R == SetNbrs(S)
                    IN  IF R \subseteq S THEN S
                                        ELSE ReachableFrom(R \cup S)
ASSUME ReachableFrom({root}) = Nodes

----
QueuesFrom(n) == {<<n, m>> : m \in Nbrs(n)}
QueuesTo(n)  == {<<m, n>> : m \in Nbrs(n)}
Queues == UNION {QueuesFrom(n) : n \in Nodes}

(* --algorithm PathsToRoot {
  ...
*)
----
Terminated  ==  \A q \in Queues : msgs[q] = << >>

TypeOK  ==  /\ depth  \in [Nodes -> 0..MaxNodes]
            /\ parent \in [Nodes -> Nodes]
            /\ msgs \in [Queues -> Seq(0..(Cardinality(Nodes)-1))]

RECURSIVE PPath(_)
PPath(n) == IF n = root THEN <<n>> ELSE <<n>> \o PPath(parent[n])

RECURSIVE DistanceFrom(_, _)
DistanceFrom(n, S) == IF n \in S THEN 0
                                 ELSE 1 + DistanceFrom(n, S \cup SetNbrs(S))

Dist(n, m) == DistanceFrom(n, {m})

Postcondition == \A n \in Nodes : Len(PPath(n)) - 1 = Dist(n, root)

RoutesFound == Terminated => Postcondition
====
```

Propiedades de tipo safety*:* `RoutesFound`

En la línea 34 se define `RoutesFound`:

```
RoutesFound == Terminated => Postcondition
```

- `Terminated` es una fórmula que se evalúa a `TRUE` cuando todos los mensajes de `msgs` han sido recibidos (línea 23):

  ```
  Terminated  ==  \A q \in Queues : msgs[q] = << >>
  ```

- `Postcondition` es una fórmula que compureba si las rutas calculadas por el algoritmo son las mejores. Por eso la propiedad `RoutesFound` está escrita como una implicación, para que compruebe si las rutas calculadas son las mejores, una vez que el algoritmo ha terminado, es decir, una vez que las colas de mensajes están todas vacías.

Analizamos ahora la fórmula `Postcondition` definida en la línea 33:

```
Postcondition == \A n \in Nodes: Len(PPath(n)) - 1 = Dist(n, root)
```

Esta fórmula tiene como significado un predicado que comprueba que para todos los nodos, si la longitud del camino desde el nodo a la raíz que ha calculado el algoritmo (**`Len(PPath(n)) - 1`**) es igual a la distancia desde el nodo a la raíz, calculada sobre el grafo de la topología de la red (**`Dist(n, root)`**). Es importante que entiendas que la distancia calculada sobre el grafo de la topología de la red (`Dist(n, root)`) es información externa al algoritmo que utilizamos para verificar si el algoritmo es correcto, para lo cuál la comparamos con la longitud del *Parent Path* (`Len(PPath(n)) - 1`)[123].

En las líneas 27 y 28 se define `PPath(n)` como una fórmula recursiva que devuelve la secuencia de nodos desde `n` hasta `r`, incluyendo ambos. Llamamos a este camino calculado por el algoritmo *Parent Path* o `PPath`. Esta secuencia se obtiene siguiendo desde `n` las variables `parent` de cada nodo, que se van concatenando por el principio a la secuencia con el operador `o`.

```
RECURSIVE PPath(_)
PPath(n) == IF n = root THEN <<n>> ELSE <<n>> \o PPath(parent[n])
```

El valor devuelto por `PPath` para cada nodo es la secuencia de nodos calculada por el algoritmo, y por tanto es este camino o ruta la que queremos verificar si ha sido calculada correctamente por el algoritmo.

Por ello en `Postcondition` se comprueba si la longitud de este camino generado por el algoritmo es igual a la longitud mínima que debería haberse obtenido analizando el grafo que devuelve el operador `Dist(n, m)` que se define en la líne 32:

```
Dist(n, m) == DistanceFrom(n, {m})
```

[123] Cuando escribimos "información externa al algoritmo" queremos decir que ninguno de los nodos que está ejecutando el algoritmo posee esa información. Ningún nodo conoce la topología de la red que aparece en los dibujos. Sólo un informador omnisciente que estuviera en todas partes tendría acceso a dicha información. Nosotros, diseñadores de la especificación, somo pequeños diosecillos omniscientes que hacemos uso de ese privilegio para poder verificar si las rutas calculadas por el algoritmo son las que nosotros conocemos de antemano en nuestra condición de diosecillos omniscientes. Repetimos: es muy importante que entiendas el modo en el que estamos verificando el algoritmo: como diosecillos omniscientes conocemos qué rutas debe calcular el algoritmo, y usamos dicho conocimiento para escribir la propiedad `Postcondition`.

Y en las líneas 29-31 se define el operador recursivo `DistanceFrom`:

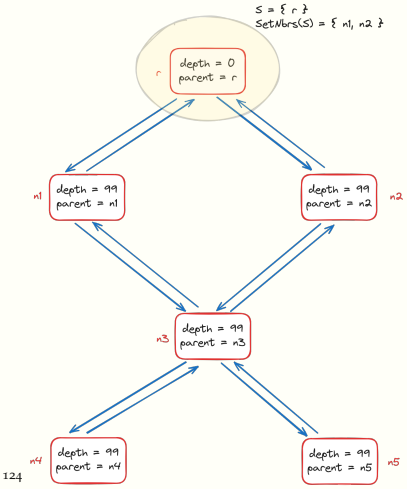
```
RECURSIVE DistanceFrom(_, _)
DistanceFrom(n, S) == IF n \in S THEN 0
                      ELSE 1 + DistanceFrom(n, S \cup SetNbrs(S))
```

`SetNbrs(S)` está definido en la línea 9: es el conjunto de todos los nodos vecinos de los nodos que hay en `S`. Se obtiene como la unión de los conjuntos de vecinos de cada uno de los nodos que hay en `S`:

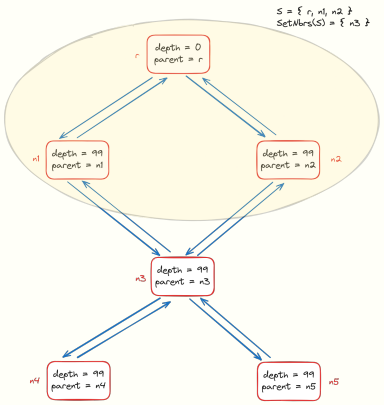
```
SetNbrs(S) == UNION {Nbrs(n) : n \in S}
```

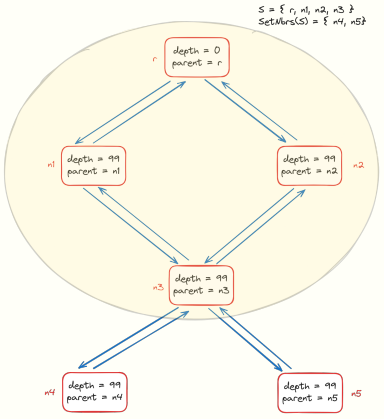
Lo que hace `DistanceFrom` es calcular, a partir de la topología de la red, la distancia desde el nodo `n` hasta `S`, siendo `S = {r}` cuando se llama desde `Dist(n, root)`. Las llamadas recursivas terminan cuando `n \in S`.

En las figuras de la derecha[124] se muestra una secuencia temporal de gráficos que pueden ayudarte a comprender el funcionamiento de estas llamadas recursivas a `DistanceFrom` para un ejemplo en el que se llama a `Dist(n5, root)`, que llama entonces a `DistanceFrom(n5, {r})`. En la figura de más arriba se muestra en un óvalo amarillo el conjunto `S` formado por `{r}`. También se muestra `SetNbrs(S)` que en esta primera figura es `{n1, n2}`. En las siguientes figuras hacia abajo se muestra la ejecución paso a paso de las sucesivas llamadas recursivas a `DistanceFrom(n, S)`, hasta que el nodo destino `n5` que se le pasó en el primer parámetro forma finalmente parte del conjunto `S` que se le pasa en el segundo parámetro, lo que finaliza la llamada recursiva, devolviendo la distancia, que en este caso es 3.



124





Aserciones para las constantes

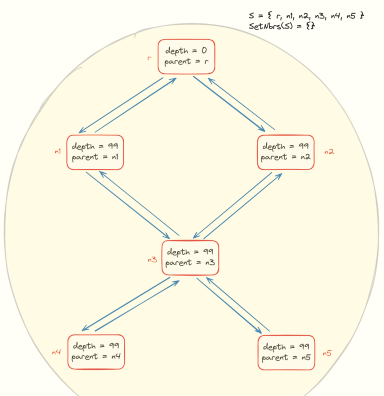
En esta especificación Leslie Lamport ha añadido dos aserciones para comprobar que los valores para las constantes son adecuados. Merece la pena que nos detengamos a estudiarlas.

Esta es la primera aserción, que junto al operador `HasTwoElts(S)` se definen en las líneas 4-7:

```
HasTwoElts(S) == \E a,b \in S : (a # b) /\ (S = {a,b})

ASSUME /\ \A e \in Edges : (e \subseteq Nodes) /\ HasTwoElts(e)
       /\ root \in Nodes
       /\ MaxNodes >= Cardinality(Nodes)
```

En la asercción `ASSUME` el conjunto `Edges` debe tener como elementos conjuntos de dos nodos. La primera expresión de la conjunción comprueba primero que todos los elementos `e` de `Edges` (recuerda que son conjuntos de dos nodos como `{r,n1}`) son subconjuntos de nodos `(e \subseteq Nodes)`, y que cada uno tiene dos elementos `HasTwoElts(e)`.



`HasTwoElts(S)` comprueba que el conjunto `S` tiene dos elementos. Fíjate en cómo es necesario comprobar que `a` y `b` son distintos `a # b`[125]. El operador existencial `\E a, b` puede devolver en `a` y `b` el mismo valor. Si no se comprobase que son distintos, podríamos haber añadido erróneamente un arco (*edge*) a la constante `Edges` en el fichero `.cfg` como `{n1}`, y sin embargo `HasTwoElts({n1})` devolvería `TRUE`.

[125] `a # b` es otra forma de escribir `a /= b`

Las otras dos expresiones de la conjunción con la que está construida la aserción comprueban respectivamente que la raíz es uno de los nodos del conjunto `Nodes`, y que la constante `MaxNodes` es mayor o igual al número de nodos; de esta forma podemos definir como valor infinito para la profundidad o distancia desde cualquier nodo hasta la raíz el valor de `MaxNodes`, pues en el caso peor de que la topología sea lineal (cada nodo tiene como máximo dos vecinos) la longitud del camino desde el nodo más alejado de la raíz sería el número de nodos menos 1. Por tanto, es seguro inicializar con `MaxNodes` el valor infinito de profundidad o distancia de cada proceso `node`, salvo el de la raíz que se inicializa a cero[126].

[126] `depth = IF self = root THEN 0 ELSE MaxNodes`

La segunda aserción y el operador recursivo `ReachableFrom(S)` se definen en las líneas 10-14:

```
RECURSIVE ReachableFrom(_)
ReachableFrom(S) == LET R == SetNbrs(S)
                    IN  IF R \subseteq S THEN S
                                        ELSE ReachableFrom(R \cup S)

ASSUME ReachableFrom({root}) = Nodes
```

Esta aserción se asegura de que en la topología definida en `Edges` todos los nodos tienen un camino para llegar a la raíz, es decir, que el grafo definido por `Edges` es conexo. Si no lo fuera, habría nodos para los que no podríamos encontrar una ruta hasta la raíz usando nuestro algoritmo.

La aserción compara el conjunto de nodos alcanzables desde la raíz, que calcula el operador `ReachableFrom(S)`, con el conjunto de todos los nodos. Como los enlaces son simétricos, si desde la raíz hay un camino hasta cada nodo, desde cada nodo habrá un camino hasta la raíz, que es lo que queremos asegurar con esta aserción.

En cada llamada recursiva al operador `ReachableFrom(S)` se va añadiendo a `S` el conjunto de nodos vecinos de `S`, hasta que no quedan más nodos que añadir, momento en el que termina la recursión.

La misma secuencia de gráficos que usamos en la página anterior ilustra cómo va creciendo el conjunto `S` desde que inicialmente es `{ r }` hasta que finalmente incluye todos los nodos.

La aserción simplemente se asegura de que el conjunto de no-

dos devueltos por `ReachableFrom(S)` es igual a `Nodes`. Si el grafo contuviese nodos que no son alcanzables desde la raíz, la igualdad de la aserción no produciría y fallaría TLC, haciéndonos ver que la topología inducida por `Edges` no es la de un grafo conexo en el que todos los nodos pueden alcanzar la raíz[127].

[127] Puedes comprobarlo quitando por ejemplo el elemento `{n3,n4}` de la constante `Edges` en el fichero `pathsToRoot_2.cfg`, y ejecutando TLC. No dejes de hacerlo para comprobar la utilidad de las aserciones

Reflexiones sobre la especificación

Usando la terminología máquina/mundo[128], en esta especificación las variables `depth` y `parent` son evidentemente parte de la máquina, que en este caso está distribuida, con una copia de ambas variables en cada nodo.

[128] `https://www.theworldandthemachine.com/p/about.html`.

Sin embargo, la red, modelada con las colas de mensajes, sería un elemento externo en el entorno de la máquina: necesitamos modelar la red en la especificación para poder enviar y recibir mensajes a través de los canales, y para verificar las propiedades de la especificación[129].

[129] Los diosecillos omniscientes necesitan tener toda la información para escribir la propiedad `Postcondition`.

Ejercicio 12

Modifica el algoritmo para que la asignación a `msgs` en la línea 18 de `pathsToRoot_1.tla` se haga en varias sentencias. Tal como indicábamos en la explicación, tendrás que sacar el valor de la cola entrante en la que se ha recibido un mensaje (línea 14) fuera del `with`, asignándola a una variable local nueva, y después, usando varias etiquetas, modificar `msgs` para borrar el mensaje de la cola entrante, y enviar el nuevo mensaje por todas las colas saliente salvo la que tiene como destino al nodo del que hemos recibido un mensaje.

Ejercicio 13

Añade una invariante a la especificación que compruebe que se cumple que, si para un nodo `n` `depth[n] < MaxNodes`, entonces el camino calculado por el algoritmo (*Parent Path*) para el nodo n termina en `r` y tiene una longitud menor o igual que `depth[n]`.

Ejercicio 14

Analiza esta alternativa de implementación del operador `ReachableFrom(S)`. Explica en castellano su funcionamiento:

```
RECURSIVE RF(_, _)
RF(B, T)  ==  IF B = {} THEN T ELSE RF(SetNbrs(B) \ T, B \cup T)

ReachableFrom(S) == RF(S, {})
```

Comprueba que es equivalente sustituyendo la definición original por esta otra y verificando la especificación con TLC.

Ejercicio 15

El algoritmo no depende de que los mensajes se reciban en orden FIFO. Modifica el algortimo para que los mensajes de las colas entrantes se reciban en cualquier orden. Para ello define un operador `RemoveElt` que devuelva un elemento cualquiera de una cola de mensajes.

Ejercicio 16

Si, como en el ejercicio anterior, podemos extraer mensajes en cualquier orden, ya no es necesario utilizar secuencias (tuplas) para modelar cada cola de mensajes. Modifica la especificación para que cada cola de mensajes sea un conjunto en lugar de una secuencia.

Ejercicio 17

Modifica la especificación para que el nodo raíz sepa cuándo todos los nodos han encontrado una ruta hasta él.

Para ello, haz que los nodos envíen mensajes a la raíz a través de las rutas calculadas.

Este ejercicio te puede llevar un buen rato, pero merece la pena que lo hagas: aprenderás y te divertirás haciéndolo.

Protocolo de inundación controlada usado en el encaminamiento basado en el estado de los enlaces

En esta sección estudiaremos la especificación de la inundación controlada que se utiliza en los algoritmos distribuidos de encaminamiento basados en el estado de los enlaces. El protocolo OSPF[130] es de este tipo.

[130] https://en.wikipedia.org/wiki/Open_Shortest_Path_First

En este tipo de protocolos los nodos se dan a conocer a sus vecinos utilizando mensajes `HELLO`. De este modo cada nodo acaba conociendo cuáles son sus nodos vecinos. Periódicamente cada nodo difunde por inundación controlada a todos los demás nodos de la red un mensaje `LINKSTATE` con su identificador y el conjunto de los enlaces con sus nodos vecinos[131]. De este modo todo nodo acaba conociendo la topología de la red, y puede calcular rutas más cortas hacia cualquier destino utilizando por ejemplo el conocido como algoritmo de Dijkstra para encontrar la ruta más corta entre dos nodos de un grafo[132].

[131] De ahí el nombre en inglés de esta familia de protocolos: *Link State*

En nuestra especificación sólo nos interesa modelar la inundación controlada. Abstraeremos por tanto los detalles del envío de mensajes `HELLO`: inicialmente cada nodo inicializa la topología que conoce de la red con los enlaces a sus vecinos.

Tampoco nos interesa el cálculo de rutas más cortas usando el algoritmo de Dijkstra una vez se conoce la topología de la red, por lo que no hay nada en la especificación relativo a este aspecto del protocolo de encaminamiento. Sólo nos interesa modelar el algoritmo de inundación que debe permitir a cada nodo conocer de manera distribuida la topología de la red[133].

[132] Dijkstra escribió muchos algoritmos y este, que lleva su nombre, no es ni mucho menos el más interesante aunque sí el más conocido: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm



Edsger W. Dijkstra

[133] Los nodos también quieren ser diosecillos quasi-omniscientes, y para ello intercambian mensajes que les permiten descubrir a todos ellos cuál es la topología de la red de la que forman parte ;-)

Algoritmo de inundación controlada

La especificación `linkState.tla` que mostramos en la siguiente página modela el algoritmo de inundación controlada con el que se difunden los estados de los enlaces, haciendo que todos los nodos conozcan la topología de la red.

Se modela de manera muy similar a la anterior especificación, si bien en este caso no existen las constantes `root` ni `MaxNodes` ya que no forman parte del algoritmo.

Tan sólo hay un elemento nuevo de TLA+ en el código: en la aserción de la línea 13 se elige un nodo cualquiera del conjunto `Nodes` y se comprueba si todos los demás son alcanzables desde él en la topología. En la anterior especificación comprobábamos en su lugar si todos los nodos eran alcanzables en la topología desde el nodo raíz, para garantizar que la topología corresponde a un grafo conexo. Como en esta especificación no hay nodo raíz, elegimos uno cualquiera con el operador TLA+ `CHOOSE`, que devuelve un elemento que satisfaga el predicado que aparece detrás del carácter `:`,

linkState.tla

```
---- MODULE linkState ----
EXTENDS Integers, Sequences, FiniteSets, TLC
CONSTANTS Nodes, Edges
----
HasTwoElts(S) == \E a,b \in S : (a # b) /\ (S = {a,b})
ASSUME /\ \A e \in Edges : (e \subseteq Nodes) /\ HasTwoElts(e)
Nbrs(n) == {m \in Nodes : {m,n} \in Edges}
SetNbrs(S) == UNION {Nbrs(n) : n \in S}
RECURSIVE ReachableFrom(_)
ReachableFrom(S) == LET R == SetNbrs(S)
                    IN  IF R \subseteq S THEN S
                            ELSE ReachableFrom(R \cup S)
ASSUME ReachableFrom({CHOOSE n \in Nodes: TRUE}) = Nodes
----
QueuesFrom(n) == {<<n, m>> : m \in Nbrs(n)}
QueuesTo(n)   == {<<m, n>> : m \in Nbrs(n)}
Queues        == UNION {QueuesFrom(n) : n \in Nodes}

(* --algorithm LinkState {
    \* All nodes have already sent LINKSTATE on all its links
    variable msgs = [q \in Queues |-> << <<q[1], {edge \in Edges: q[1] \in edge}>> >>];

fair process (node \in Nodes)
  variable
    \* HELLO protocol already run: every node knows its links with neighbors
    learntTopo = {edge \in Edges: self \in edge},
    \* Initialize with our own LINKSTATE msg that has already been sent in msgs
    linkStateMsgsSeen = {<< self, learntTopo >>};
{a:while (TRUE) {
    with (q \in {r \in QueuesTo(self) : msgs[r] /= << >>}) {
        if (Head(msgs[q]) \in linkStateMsgsSeen)
            msgs[q] := Tail(msgs[q]) \* LINKSTATE msg already seen, delete it
        else {                       \* Process new LINKSTATE msg
            learntTopo        := learntTopo \union Head(msgs[q])[2];
            linkStateMsgsSeen := linkStateMsgsSeen \union {Head(msgs[q])};
            msgs :=
                [r \in Queues |->  \* forward rcvd LINKSTATE msg and delete it
                    IF r = q
                    THEN Tail(msgs[q]) \* delete LINKSTATE msg
                    ELSE IF r \in QueuesFrom(self) \ {<<self, q[1]>>}
                        THEN Append(msgs[r], Head(msgs[q])) \* forward LINKSTATE
                        ELSE msgs[r] ] \* don't forward to node that sent it
}}}}}
*)
----
Terminated == \A q \in Queues: msgs[q] = <<>>
Postcondition == \A n \in Nodes: learntTopo[n] = Edges
AllToposEqual == Terminated => Postcondition
EventuallyTerminated == <>Terminated
====
```

linkState.cfg

```
1  SPECIFICATION
2      Spec
3
3  CONSTANTS
4      \* Nodes = {n0, n1, n2, n3, n4, n5}
5      \* Edges = {{n0,n1},{n0,n2},{n1,n3},{n2,n3},{n3,n4},{n3,n5}}
6
6      \* Nodes = {n0, n1, n2, n3, n4}
7      \* Edges = {{n0,n1},{n0,n2},{n1,n3},{n2,n3},{n3,n4}}
8
8      Nodes = {n0, n1, n2, n3}
9      Edges = {{n0,n1},{n0,n2},{n1,n3},{n2,n3}}
10
10 INVARIANTS
11     AllToposEqual
12
13
12 PROPERTIES
13     EventuallyTerminated
```

en este caso, `TRUE`. El efecto es que devuelve un nodo cualquiera[134].

[134] `CHOOSE` es determinista: devuelve un elemento cualquiera, pero siempre el mismo: no es escoger uno al azar.

Se han reutilizado las aserciones de la anterior especificación, y se han modificado las propiedades, que comentaremos más abajo.

Inundación controlada de los mensajes LINKSTATE

En la figura 7 mostramos una topología como ejemplo con el que luego verificaremos la especificación, mostrándose en cada nodo los valores en el estado inicial de las variables que usa cada nodo para implementar el algoritmo de inundación controlada.

Cada nodo envía inicialmente un mensaje `LINKSTATE` con una secuencia TLA+ o tupla que contiene su identificador y el conjunto de los enlaces con sus vecinos (línea 20).

Variables en cada nodo:

- `learntTopo`: almacena el conjunto de enlaces que se van conociendo cuando se van recibiendo mensajes `LINKSTATE`. Cada nodo inicializa esta variable con los enlaces que le unen a sus vecinos, que debería haber conocido gracias a los mensajes `HELLO`. En esta especificación se abstrae completamente el envío de estos mensajes, inicializando con los vecinos la variable `learntTopo` de cada nodo (línea 24).
- `linkStateMsgsSeen`: almacena el conjunto de mensajes `LINKSTATE` que ya se han recibido previamente en un nodo, para no reen-

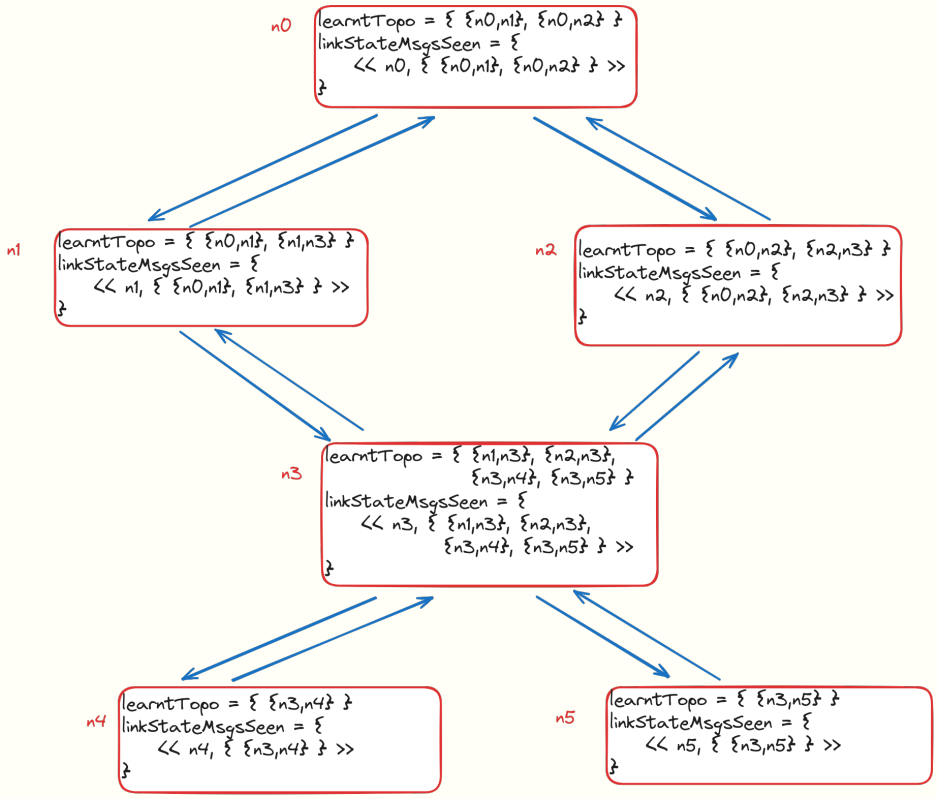


Figura 7: Topología del ejemplo

viarlos si se vuelven a recibir (inundación controlada). Se inicializa con los mensajes `LINKSTATE` iniciales que ha enviado cada nodo antes de comenzar el algoritmo de inundación controlada, conteniendo los enlaces a sus vecinos (línea 26).

CUANDO UN NODO RECIBE UN MENSAJE `LINKSTATE`:

- si ya lo había recibido antes: lo descarta (línea 30)
- si no lo había recibido antes:
  1. guarda en `learntTopo` los enlaces que vienen en el mensaje (línea 32).
  2. guarda el mensaje en `linkStateMsgsSeen` (línea 26).
  3. reenvía el mensaje por todos sus enlaces salvo por el que lo ha recibido, y borra el mensaje de la cola entrante (líneas 34-40).

Verificación de propiedades

- La propiedad de tipo *safety* `AllToposEqual` comprueba que, una vez se han recibido en todos los nodos los mensajes LINKSTATE enviados por inundación desde cada nodo, todos los nodos conocen la topología completa de la red (línea 46). Para ello se compara con `Edges` la topología aprendida en cada nodo gracias a los mensajes `LINKSTATE` recibidos (línea 45).

- La propiedad de tipo *liveness* `EventuallyTerminated` sirve para aseguarse de que finalmente las colas de los mensajes se vacían (línea 47).

En el fichero `linkState.cfg` aparecen en comentarios varias configuraciones de la red del ejemplo. Ten en cuenta que dado que en la especificación todos los nodos envían inicialmente un mensaje `LINKSTATE`, el número de estados es muy alto. Por eso es conveniente que ejecutes TLC con un modelo con pocos nodos. Comprueba la diferencia en tiempo de ejecución cuando se usa una u otra topología en la verificación con TLC: tarda pocos segundos en verificar la que está descomentada pero las otras le llevan decenas de minutos (depende de la CPU y la memoria de tu ordenador) y varias horas respectivamente. Y eso que de nuevo este algoritmo tiene una sola etiqueta y por lo tanto, como puedes comprobar el código TLA+ producido al traducir el algoritmo PlusCal, una sola acción `node(self)`.

Ejercicio 18

Puedes probar modelos más grandes si en lugar de que todos los nodos envíen inicialmente su correspondiente mensaje inicial `LINKSTATE` haces que sólo alguno lo envíe. Modifica la especificación para que sólo un nodo cualquiera elegido con `CHOOSE` sea el que envíe el mensaje. Asegúrate de que la inicialización de `msgs` y de `linkStateMsgsSeen` es la adecuada. Y modifica las propiedades: ahora al terminar los nodos no habrán aprendido toda la topología (`Edges`), sino sólamente los enlaces de uno de los nodos.

Verifica con TLC modelos más grandes tras realizar estas modificaciones.

Ejercicio 19

Crea otra especificación para modelar el envío de los mensajes `HELLO`. Cuando un nodo comience su ejecución enviará un mensaje `HELLO` por cada una de sus colas salientes. El mensaje `HELLO` está compuesto por una tupla con el identificador del nodo que lo envía.

Cuando un nodo recibe un mensaje `HELLO` actualiza su variable `learntTopo` con una descripción de ese enlace que ha descubierto.

Ejercicio 20

Integra las especificaciones de los dos anteriores ejercicios.

Lecturas recomendadas

- *PlusCal / TLA+: An Annotated Cheat Sheet*. Stephan Merz [135].

  Este pequeño documento contiene todo lo que puedes necesitar saber de PlusCal/TLA+, contado de manera sintética.

  Puedes utilizarlo como un manual de referencia para cuando dudes algo sobre la sintaxis de PlusCal o de TLA+. Al tener sólo 15 páginas, puedes leerlo una primera vez por encima para saber todo lo que incluye y luego recurrir a él sólo cuando tengas dudas sobre sintaxis o uso de expresiones y construcciones TLA+/PlusCal.

[135] Merz, S. (Last Updated: 2021). PlusCal / TLA+: An annotated cheat sheet. http://mobiquo.gsyc.es/tlapluscal/pluscal-cheatsheet.pdf

- Chuletas (Cheat Sheet) de TLA+:
  - Summary of TLA. Leslie Lamport[136].

    Un resumen extendido de TLA+ que puedes usar junto al anterior documento.

  - *The TLA cheat-sheet*. OCamlPRO[137].

    Chuleta muy resumida de TLA+.

[136] Lamport, L. (Last Updated: 2022). Summary of TLA+. https://lamport.azurewebsites.net/tla/summary-standalone.pdf

[137] OCamlPRO (Last Updated: 2021). The TLA cheat-sheet. https://ocamlpro.com/assets/pdf/tla-cheat-sheet-v1.pdf

Recursos sobre TLA+/PlusCal

- *A PlusCal User's Manual C-Syntax*. Leslie Lamport[138].

  Este es el manual de usuario de PlusCal. Lo cuenta todo sobre el lenguaje.

  Échale un ojo pero sólo por encima, ya que el documento PlusCal / TLA+: An Annotated Cheat Sheet, de Stephan Merz, puede ser suficiente para consultar la sintaxis y uso de TLA+/PlusCal. Si no, consulta este manual.

[138] Lamport, L. (Last Updated: 2021b). A PlusCal user's manual c-syntax. https://ocamlpro.com/assets/pdf/tla-cheat-sheet-v1.pdf

- *PlusCal Tutorial*. Leslie Lamport[139].

  Con los conocimientos adquiridos en estos apuntes este tutorial puede ser especialmente valioso pues debería entender la mayoría de lo que se explica aquí. La penúltima especificación que explicamos en estos apuntes es la de la sesión 10 de este tutorial de Leslie Lamport.

  Sin embargo es extenso y va paso a paso introduciendo múltiples elementos, por lo que requiere bastante tiempo seguirlo. Merece la pena hacerlo, quizá en verano ;-)

[139] Lamport, L. (Last Updated: 2021a). PlusCal tutorial. https://lamport.azurewebsites.net/tla/tutorial/intro.html

- *Learn TLA+*. Hillel Wayne[140].

  A pesar del nombre, es un tutorial de TLA+ y de PlusCal que te aconsejamos que consultes.

[140] Wayne, H. (Last Updated: 2023). Learn TLA+. https://learntla.com/

- *Practical TLA+: Planning Driven Development*. Hilel Wayne[141].

[141] Wayne, H. (2018). *Practical TLA+: Planning Driven Development*. Apress

Libro en papel sobre TLA+ y PlusCal del mismo autor que el anterior tutorial. Leslie Lamport ha publicado erratas[142].

[142] https://lamport.azurewebsites.net/tla/practical-tla.html

Está disponible en electrónico en Safari Books accediendo desde IPs de la URJC.

- *The TLA+ Video Course*. Leslie Lamport[143].

  Tutorial sólo sobre TLA+, no PlusCal, muy interesante y divertido.

[143] Lamport, L. (Last Updated: 2021c). The TLA+ video course. https://lamport.azurewebsites.net/video/videos.html

- *Specifying Systems*. Leslie Lamport[144].

  Este es "el libro de TLA+". Su contenido íntegro está legalmente disponible en formato PDF. No busques aquí información sobre PlusCal. Sólo recomendamos su lectura a los estudiantes con interés en profundizar en el conocimiento de TLA+.

[144] Lamport, L. (2002). *Specifying systems: the TLA+ language and tools for hardware and software engineers*. Addison-Wesley Professional

- Blog de Murat Dermibas[145].

  El profesor Murat Dermibas de la Universidad de Búfalo está enseñando actualmente (septiembre 2023) TLA+/PlusCal a empleados de Amazon Web Services, donde trabaja actualmente.

  Sus entradas en el blog relacionadas con TLA+/PlusCal merecen la pena pues tienen una orientación pedagógica interesante.

[145] Demirbas, M. (Last Updated: 2023). TLA+ posts in blog. https://muratbuffalo.blogspot.com/search/label/tla

- *TLA+ in Practice and Theory*. Ron Pressler[146].

  Estos 4 artículos de Ron Pressler sobre TLA+ son muy buenos. Interesantes para el estudiante que quiera profundizar en el estudio de TLA+.

[146] Pressler, R. (Last Updated: 2017). TLA+ in practice and theory. https://pron.github.io/posts/tlaplus_part1

- Grupo de usuarios de TLA+/Pluscal[147].

[147] TLA+ community (Last Updated: 2023a). TLA+ and PlusCal discussion group. https://groups.google.com/g/tlaplus

- Colección de ejemplos escritos en TLA+/PlusCal[148].

[148] TLA+ community (Last Updated: 2023b). TLA+ and PlusCal specification examples. https://github.com/tlaplus/Examples

Referencias

Demirbas, M. (Last Updated: 2023). TLA+ posts in blog. https://muratbuffalo.blogspot.com/search/label/tla.

Lamport, L. (2002). *Specifying systems: the TLA+ language and tools for hardware and software engineers*. Addison-Wesley Professional.

Lamport, L. (Last Updated: 2021a). PlusCal tutorial. https://lamport.azurewebsites.net/tla/tutorial/intro.html.

Lamport, L. (Last Updated: 2021b). A PlusCal user's manual c-syntax. https://ocamlpro.com/assets/pdf/tla-cheat-sheet-v1.pdf.

Lamport, L. (Last Updated: 2021c). The TLA+ video course. https://lamport.azurewebsites.net/video/videos.html.

Lamport, L. (Last Updated: 2022). Summary of TLA+. https://lamport.azurewebsites.net/tla/summary-standalone.pdf.

Merz, S. (Last Updated: 2021). PlusCal / TLA+: An annotated cheat sheet. http://mobiquo.gsyc.es/tlapluscal/pluscal-cheatsheet.pdf.

OCamlPRO (Last Updated: 2021). The TLA cheat-sheet. https://ocamlpro.com/assets/pdf/tla-cheat-sheet-v1.pdf.

Pressler, R. (Last Updated: 2017). TLA+ in practice and theory. https://pron.github.io/posts/tlaplus_part1.

TLA+ community (Last Updated: 2023a). TLA+ and PlusCal discussion group. https://groups.google.com/g/tlaplus.

TLA+ community (Last Updated: 2023b). TLA+ and PlusCal specification examples. https://github.com/tlaplus/Examples.

Wayne, H. (2018). *Practical TLA+: Planning Driven Development*. Apress.

Wayne, H. (Last Updated: 2023). Learn TLA+. https://learntla.com/.